DEGREE PROJECT IN INFORMATION AND COMMUNICATION TECHNOLOGY,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2017

Performance Optimization of Signal Processing Algorithms for SIMD Architectures

SHARAN YAGNESWAR

KTH ROYAL INSTITUTE OF TECHNOLOGY
SCHOOL OF ELECTRICAL ENGINEERING

KTH Information and
Communication Technology

Performance Optimization of Signal Processing Algorithms for SIMD Architectures

Masters Thesis submitted in partial fulfillment of the requirements for the Degree of Master of Science in Embedded Systems at KTH Royal Institute of Technology, Stockholm

SHARAN YAGNESWAR

Master's Thesis at MIND Music Labs
Supervisor: Dr. Stefano Zambon
Examiner: Dr. Carlo Fischione

Acknowledgment

I would like to express my sincere gratitude and respect towards my supervisor Dr. Stefano Zambon, MIND Music Labs - Stockholm for being of immense help, guidance and a source of inspiration during the period of this thesis. I would also like to thank my Examiner Dr. Carlo Fischione for his patience, guidance and support. This project has given me a lot of knowledge and has sparked my interest. I am eternally grateful for this opportunity.

I would also like to sincerely thank my parents and my sister. I would not be here if not their unwavering support and love all throughout this journey.

Stockholm, June 2017
Sharan Yagneswar

Abstract

Digital Signal Processing(DSP) algorithms are widely implemented in real time systems. In fields such as digital music technology, many of these said algorithms are implemented, often in combination, to achieve the desired functionality. When it comes to implementation, DSP algorithms are performance critical as they have tight deadlines. In this thesis, performance optimization using Single Instruction Multiple Data(SIMD) vectorization technique is performed on the ARM Cortex-A15 architecture for six commonly used DSP algorithms; Gain, Mix, Gain and Mix, Complex Number Multiplication, Envelope Detection and Cascaded IIR Filter. To ensure optimal performance, the instructions should be scheduled with minimal pipeline stalls. This requires execution time to be measured with fine time granularity. First, a technique of accurately measuring the execution time using the cycle counter of the processor's Performance Management Unit(PMU) along with synchronization barriers is developed. It was found that the execution time measured by using the operating system calls have high variations and very low time granularity, whereas the cycle counter method was accurate and produced reliable results. The cost associated with the cycle counter method is 75 clock cycles. Using this technique, the contribution by each SIMD instruction towards the execution time is measured and is used to schedule the instructions. This thesis also presents a guideline on how to schedule instructions which have data dependencies using the cycle counter timing execution time measurement technique, to ensure that the pipeline stalls are minimized. The algorithms are also modified, if needed, to favor vectorization and are implemented using ARM architecture specific SIMD instructions. These implementations are then compared to that which are automatically produced by the compiler's auto-vectorization feature. The execution times of the SIMD implementations was much lower compared to that produced by the compiler and the speedup ranged from 2.47 to 5.11. Also, the performance increase is significant when the instructions are scheduled in an optimal way. This thesis concludes that the auto-vectorized code does poorly for complex algorithms and produces code with a lot of data dependencies causing pipeline stalls, even with full optimizations enabled. Using the guidelines presented in this thesis for scheduling the instructions, the performance of the DSP algorithms have significant improvements compared to their auto-vectorized counterparts.

Keywords: SIMD, ARM, Vectorization, DSP, NEON, IIR, Envelope, Complex, Performance Optimization.

Sammanfattning

Digitala signalbehandlingsalgoritmer(DSP) implementeras ofta i realtidssystem. Inom fält som exempelvis digital musikteknik används dessa algoritmer, ofta i olika kombinationer, för att ge önskad funktionalitet. Implementationen av DSP-algoritmer är prestandakritisk eftersom systemen ofta har små tidsmarginaler. I det här examensarbetet genomförs prestandaoptimering med Single Instruction Multiple Data(SIMD)-vektorisering på en ARM A15-arkitektur för 6 vanliga DSP-algoritmer; volym, mix, volym och mix, multiplikation av komplexa tal, amplituddetektering, och seriekopplade IIR-filter. Maximal optimering av algoritmerna kräver också att antalet pipeline stalls i processorn minimeras. För att kunna observera detta krävs att exekveringstiden kan mätas med hög tidsupplösning. I det här examensarbete utvecklas först en teknik för att mäta exekveringstiden med hjälp av en klockcykelräknare i processorns Performance Management Unit(PMU) tillsammans med synkroniseringsbarriärer. Tidsmätning med hjälp av operativsystemsfunktioner visade sig ha sämre noggrannhet och tidsupplösning än metoden med att räkna klockcykler, som gav tillförlitliga resultat. Den extra exekveringstiden för klockcykelräkning uppmättes till 75 klockcykler. Med den här tekniken är det möjligt att mäta hur mycket varje SIMD-instruktion bidrar till den totala exekveringstiden. Examensarbete presenterar också en metod att ordna instruktioner som har databeroenden sinsemellan med hjälp av ovanstående tidsmätningsmetod, så att antalet pipeline stalls minimeras. I de fall det behövdes, skrevs koden till algoritmerna om för att bättre kunna utnyttja ARM-arkitekturens specifika SIMD-instruktioner. Dessa jämfördes sedan med resultaten från kompilatorns automatgenererade vektoriseringkod. Exekveringstiden för SIMD-implementationerna var signifikant kortare än för de kompilatorgenererade och visade på en förbättring på mellan 2,47 och 5,11 gånger, mätt i exekveringstid. Resultaten visade också på en tydlig förbättring när instruktionerna exekveras i en optimal ordning. Resultaten visar att automatgenererad vektorisering presterar sämre för komplexa algoritmer och producerar maskinkod med signifikanta databeroenden som orsakar pipeline stalls, även med optimeringsflaggor påslagna. Med hjälp av metoder presenterade i det här examensarbete kan prestandan i DSP-algoritmer förbättras betydligt i jämförelse med automatgenererad vektorisering.

Nyckelord: SIMD, ARM, Vektorisering, DSP, NEON, IIR, Kuvert, Komplex, Prestandaoptimering.

Contents

List of Figures i

List of Tables iii

List of Abbreviations v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 3
- 1.3 Goals . . . 4
- 1.4 Approaches taken . . . 4
- 1.5 Outline . . . 5

2 Literature Review 7
- 2.1 Real Time Systems . . . 7
  - 2.1.1 Characteristics . . . 8
  - 2.1.2 Events in a real time system . . . 8
  - 2.1.3 Hard and Soft Real Time Systems . . . 9
  - 2.1.4 Embedded Hardware and Processors . . . 9
- 2.2 The ARM Architecture . . . 11
- 2.3 The ARM Cortex-A15 . . . 13
  - 2.3.1 Pipeline . . . 14
  - 2.3.2 Advanced SIMD(NEON) Unit and Instruction Set . . . 15
  - 2.3.3 Load and Store Operations with the Advanced SIMD Unit . 20
  - 2.3.4 The VFP Unit . . . 26
  - 2.3.5 ARM Performance Management Unit . . . 27
  - 2.3.6 Odroid XU4 . . . 28
- 2.4 Execution Time Measurement . . . 29
  - 2.4.1 Static Timing Analysis . . . 30
  - 2.4.2 Dynamic Timing Analysis . . . 31
- 2.5 DSP Algorithms . . . 32
  - 2.5.1 NE10 Library . . . 32
  - 2.5.2 Gain . . . 32
  - 2.5.3 Mix . . . 33

2.5.4 Gain and Mix . . . 33
2.5.5 Complex Number Multiplication . . . 34
2.5.6 Cascaded Infinite Impulse Response Filter . . . 36
2.5.7 Peak Program Meter . . . 40

3 Development and Testing Methodology 43
3.1 Methodology of Research . . . 43
3.2 Development Methodology . . . 43
3.2.1 Programming Language and Packaging of Functions . . . 44
3.2.2 Development cycle . . . 44
3.2.3 Folder Structure . . . 44
3.2.4 List of Functions Developed . . . 45
3.2.5 General Code Structure . . . 46
3.3 Testing Methodology . . . 46

4 Timing measurement and Benchmarking Methodology 47
4.1 Calculation of WCET . . . 47
4.2 Instruction Scheduling Methodology . . . 48
4.2.1 Guidelines for Scheduling SIMD Instructions With Timing Information . . . 49
4.3 Timing Measurement . . . 51
4.3.1 Puslar.Webshaker Cycle Counter for Cortex A8 . . . 51
4.3.2 GEM5 Simulator . . . 51
4.3.3 C++ Chronos Library . . . 53
4.3.4 Performance Management Unit . . . 53
4.3.5 Development Platform Details . . . 55
4.4 Accuracy Evaluation of PMU cycle counter and Chronos . . . 56
4.4.1 Measuring Cycles per Instruction . . . 58
4.4.2 Cost of using PMU Cycle Timer with Barriers . . . 60
4.5 Performance Metrics . . . 61
4.5.1 Speed Up . . . 61

5 SIMD Vectorization of DSP Functions 63
5.1 Input and Output Audio Buffers . . . 63
5.2 Gain . . . 63
5.2.1 Architectural Optimization and Implementation . . . 63
5.2.2 Results . . . 66
5.3 Mix . . . 68
5.3.1 Architectural Optimization and Implementation . . . 68
5.3.2 Results . . . 71
5.4 Gain and Mix . . . 72
5.4.1 Architectural Optimization and Implementation . . . 72
5.4.2 Results . . . 75
5.5 Complex Number Multiplication . . . 76

5.5.1 Architectural Optimization and Implementation 76
5.5.2 Results 79
5.6 Peak Program Meter 80
5.6.1 Algorithmic and Architectural Optimization 80
5.6.2 Results 85
5.7 Four Band Equalizer with Cascaded Biquad Filters 86
5.7.1 Initial State Coefficients 86
5.7.2 Cascade To Parallel 87
5.7.3 Architectural Optimization And Implementation Of The Algorithm 88
5.7.4 Results 97

6 Conclusion **99**
6.1 Conclusions 99
6.2 Future Work 101

Bibliography **103**

Appendices **107**

A Build System used for Development **109**

B Unit Testing with Google Test **111**

C Execution Time Measurement with Chronos Library **115**
C.1 Measuring code with Chronos High Precision Timer 116

D Execution Time Measurement with Chronos Library **117**
D.1 Enabling User Space Access to PMU Registers 117
D.2 Using PMU Cycle Counter with Barriers 119

List of Figures

2.1 Typical real time audio processing system. 8
2.2 Typical embedded system 10
2.3 ARMv7 register file 12
2.4 Performance to Code Density comparison 13
2.5 ARM Cortex-A15 Pipeline stages 14
2.6 ARM Cortex-A15 pipeline execution units 15
2.7 Quadword and doubleword register mapping in the Advanced SIMD unit 16
2.8 Two examples of register packing. 17
2.9 Internal structure of the Cortex-A8 NEON unit 17
2.10 SIMD interleaved store example 22
2.11 SIMD interleaved load instruction example 22
2.12 Single n element from memory into multiple lanes example 23
2.13 Multiple n elements from memory to multiple lanes example A 25
2.14 Multiple n elements from memory to multiple lanes example B 25
2.15 VFP register mapping 26
2.16 Performance Management Unit 27
2.17 Odroid XU4 29
2.18 Typical probability distribution of execution times 30
2.19 Arrangement of complex numbers in memory 35
2.21 Cascaded realization of biquads 38

3.1 Flow chart outlining the approach towards development. 45
3.2 General SIMD code structure 46

4.1 Instruction scheduling guidelines 50
4.2 Execution time measurement technique 54
4.3 Probability histogram of execution times measured with PMU cycle timer for the SIMD vectorized gain algorithm 56
4.4 Probability histogram of execution times measured with Chronos library for the SIMD vectorized gain algorithm 57
4.5 Measurement of Cycles per SIMD instruction using PMU cycle counter 59
4.6 Measurement of Cycles per SIMD instruction using Chronos cycle counter 60
4.7 Probability histogram of PMU cycle counter latency. 60

5.1 Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Gain algorithm. 67
5.2 Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Mix algorithm. 72
5.3 Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Gain and Mix algorithm. 75
5.4 Normalized probability histograms for the SIMD Vectorized implementation and auto-vectorized implementation of the Complex Number Multiplication algorithm. 79
5.5 Normalized probability histograms for the SIMD Vectorized implementation and auto-vectorized implementation of the Peak Meter Algorithm algorithm. 85
5.6 Parallel realization of biquads 87
5.7 Vectorized implementation of parallel biquad filters function 89
5.8 Normalized probability histograms for the vectorized parallel filter implementation, vectorized biquad implementations in cascade form and auto-vectorized C implementation. 97

A.1 `CmakeLists.txt` is placed in all directories having source code as well as the top directory. 109

List of Tables

2.1 Single element from memory into single lane list fields 21
2.2 Single n element from memory into multiple lanes list fields 23
2.3 Multiple n elements from memory to multiple lanes list fields 24

5.1 Gain algorithm execution time results. 67
5.2 Mix algorithm execution time results. 71
5.3 Gain and Mix algorithm execution time results. 76
5.4 Complex number multiplication algorithm execution time results 80
5.5 Peak program meter algorithm execution time results 86
5.6 Peak program meter algorithm execution time results 98

List of Abbreviations

AC ..... Attack Coefficient

APSR ..... Application Program Status Register

BCET ..... Best Case Execution Time

CFA ..... Control Flow Analyses

CISC ..... Complex Instruction Set Computer

CPI ..... Cycles per Instruction

DSP ..... Digital Signal Processing

FIR ..... Finite Impulse Response

FPSCR ..... Floating Point Status Control Register

GCC ..... GNU Compiler Collection

GPU ..... Graphics Processing Unit

IIR ..... Infinite Impulse Response

IoT ..... Internet of Things

LR ..... Link Register

OS ..... Operating System

PC ..... Program Counter

PCMR ..... Performance Monitors Control Register

PMCCNTR ..... Performance Monitors Cycle Count Register

PMCNTENSET .... Performance Monitors Count Enable Set Register

PMOVSR ..... Performance Monitors Overflow Flag Status Register

PMSELR ..... Performance Monitors Event Counter Selection Register

PMU ..... Performance Management Unit

PMUSERENR ..... Performance Monitors User Enable Register

PMXEVCNTR ..... Performance Monitors Event Count Register

PMXEVTYPER ... Performance Monitors Event Type Select Register

PPM ..... Peak Program Meter

RISC ..... Reduced Instruction Set Computer

RTL ..... Register Transfer Logic

RC ..... Release Coefficient

SIMD ..... Single Instruction Multiple Data

SISD ..... Single Instruction Single Data

SoC ..... System on Chip

SP ..... Stack Pointer

VFP ..... Vector Floating Point

WCET ..... Worst Case Execution Time

Chapter 1

Introduction

When it comes to hard real time systems, the execution time of the program plays an important role in determining the stability and failure rate of a system. This is because certain real time systems are characterized by hard deadlines, which they have to meet for proper functionality[1]. Furthermore, precise determination of these deadlines play a crucial role during the design phase of a system. In a real time system that carries out DSP, such as an audio processing system, the DSP algorithms are generally performance critical, since they involves high number of floating point calculations[2]. Hence there is a need to improve the performance of such performance critical sections in a real time system, in order to meet the tight deadlines. There are many ways by which a given algorithm can be optimized for performance. These methods can be either architecturally independent or dependent[3][4].

1.1 Background

Architecturally independent optimizations involve using programming guidelines and tricks to ensure that the compiler produces improved code, i.e, parallelization using multiple threads, code which reuses registers, reduced memory access, unrolled loops etc[3][5]. On the other hand architecture dependent optimizations involves writing code which exploits the potential of the underlying hardware architecture. This includes code which is cache friendly, uses extra hardware computational units such as vectorization units, eliminates pipeline hazards and so on[3]. A combination of the two yields well optimized code.

Parallelization involves using multiple cores of the hardware platform with the help of the operating system(OS) threads. The performance can be improved, in an ideal sense, by dividing the computational load of the algorithm and distributing it evenly amongst the various cores of the system. Though this technique is widely used, in practicality, there is a time cost associated with synchronization

and scheduling of the threads. This cost is significant when dealing with performance critical code whose execution time ranges from 10^{-7} to 10^{-9} seconds, such as those in a real time audio processing system[6]. Enter vectorization, a feature set offered by many embedded System on Chip(SoC) manufactures. Vectorization offers the capability of working on multiple points of data, i.e, vectors using a single instruction, using SIMD type of instructions[5].

Unlike traditional Single Instruction Single Data(SISD) type of instructions which can perform an operation only on a single data element at a time, SIMD instructions can be used to improve execution speed of performance critical code by exploiting the data level parallelism of the algorithm. The SIMD unit usually exists as a co-processor alongside the main cores in a SoC and shares other hardware resources with it[5]. For this thesis, the ARM NEON SIMD instruction set which runs on the ARM Advanced SIMD hardware unit[7] is used. It offers various arithmetic (both integer and floating point), logical and various memory transfer operations and can work on 128 bits of data at a time[8]. This instruction set can be incorporated in the code by either using the auto-vectorization feature offered by many compilers, such as GNU Compiler Collection(GCC)[7], or manually vectorizing by hand. When the auto-vectorization feature is used, the compiler identifies hot spots in the code and vectorizes them with SIMD instructions during the code generation stage of compilation. However, hand optimized vectorization usually yields better performance[3][9]. Another feature provided by ARM is the Vector Floating Point(VFP) Unit[8], which, as the name suggests, can work with vectors of floating point data. This is also a possibility explored in this thesis.

There are various hardware factors which can affect the performance of code. Some general ones are:

- Pipeline Hazards - When an instruction waits for the result of a previous one, the pipeline is stalled until the hardware unit produces the result. These stalls increase execution time of the code and is detrimental to performance. Software pipelining techniques and instruction scheduling can combat this issue[10].

- Cache friendliness - The data accessed by the code should have maximum possible cache locality and minimum number of cache misses for optimal performance. Ensuring spatial locality of data along with aligned memory accesses can be used to improve the performance[9].

- Alignment of Data - Memory alignment refers to the way data is arranged in the memory and this affects memory transfer speeds. This because some architectures like ARM are designed to access 32 bits in a single fetch, so it is optimal if the data is aligned to 32 bits in the memory. If that is not the case, the processor has to carry out more than one memory transfer operation[4][5].

- Code Size - The smaller the code size, the better it fits into the instruction cache. It is essential that software running on real time systems have good cache locality and the number of main memory accesses is minimal[9].

Benchmarking a real time DSP application is essential for getting the execution time and other characteristic information needed during the design phase of a real time system. The best case and worse case execution times, along with memory transfers and cache performance data will help aide in the designing of the system[6]. In ARM the Performance Management Unit(PMU) comes in hand for gaining useful statistics such as execution time, cache and TLB performance etc. It has 6 event counters, a clock cycle counter and has support for 67 different events[11]. Since benchmarking with fine time granularity is necessary in this thesis, the ARM PMU is explored as a solution.

1.2 Problem Statement

DSP algorithms such as recursive filters and envelope detectors are an essential part in any digital music technology or application. In a real time audio processing system, some of these algorithms can be performance critical because they deal with a large number of floating point calculations. Therefore, their performance has to be improved so that they can meet the hard deadlines imposed by the system requirements. Parallelization is a solution, but the time cost involved with synchronization and data transfer between cores is significant. Hence vectorization techniques which exploits the underlying architecture needs to be explored. GCC offers the auto vectorization feature, but it does not always produce optimal results[9][12][13] and so manual implementation is necessary. Different algorithms for a particular DSP functionality can yield different performance results, depending on vectorization potential. So an analysis and comparison between algorithms and their vectorized implementations is required.

While comparing the performance of different implementations, accurate measurement of execution time gives an idea of how well it performs. Also, improper scheduling of instructions can lead to pipeline stalls. Proper scheduling of instructions can be performed by measuring the contribution of each instruction towards the execution time. Operating system calls or standard libraries do not provide accurate and stable results and often have a lot of variance between tests, since they also operate in the same time scale[14] as the code they measure. Other factors which likely contribute to inconsistent measurement results are context switching, cache misses and background processes running[14][6]. Hence a more accurate execution time measurement technique with high time granularity and low overhead is needed, which provides more accurate results and is independent of said factors.

1.3 Goals

This thesis explores the various SIMD vectorization possibilities of improving the performance of DSP algorithms which run in a real time embedded system on-board a digital musical instrument. The goal was to deliver a set of algorithms for frequently used DSP functionality, which are chosen by comparing different implementations of vectorization. The deliverables are

- A study and comparison of the following DSP functions and approaches to vectorize them.
    1. Gain.
    2. Mix.
    3. Gain and Mix.
    4. Complex Number Multiplication.
    5. Peak Program Meter (Envelope Detector).
    6. Cascaded IIR filter.
- Deliver a library of optimized implementations of the algorithms.
- Best case and worst case execution time data for the optimized implementations.
- Guidelines and workflow to be followed when using ARM NEON instruction set and the Advanced SIMD unit to get optimal performance results.
- An accurate execution time measurement technique to aide in instruction scheduling and measurement of program execution times.

1.4 Approaches taken

To solve the above mentioned problem, quantitative analyses was the preferred method of research used in this thesis. The outline of tasks taken were.

- Study and familiarization with ARM NEON instruction set and the Advanced SIMD unit.
- Setting up of build system to develop the implementations.
- Study of the said algorithms and the various approaches that have been taken to vectorize them.
- Study of the ARM PMU unit to develop an execution time measurement technique using it.

- Sequential implementations of the said algorithms written in C, to evaluate the compiler's auto-vectorization performance.
- Vectorization using ARM NEON instruction set whilst reducing pipeline hazards and improving cache performances.
- Comparison and analyses amongst different implementations and choosing the optimal one.
- Establishing a work flow and guideline to aide in SIMD Optimization in ARM Cortex-A15.

1.5 Outline

This thesis report is divided into six major chapters.

1. **Introduction**: The first chapter deals with giving an introduction to the thesis work. It contains the problem statement and relevant background information, the goals of the thesis and what approaches were taken.
2. **Literature Review**: This section deals with all the necessary background literature. It contains introductory information about real time systems, the ARM architecture, different techniques for measurement of execution times and related literature on optimization of the DPS algorithms using SIMD vectorization.
3. **Development and Testing Methodology**: This chapter describes what research methodology was used in this thesis. It also briefly describes how the library was developed and tested.
4. **Timing and Benchmarking Methodology**: This chapter explores different execution times measurement techniques and evaluation of their accuracy. It also describes the instruction scheduling methodology and the performance metrics used to describe the results.
5. **SIMD Optimizations of DSP functions**: This chapter describes in detail how each algorithm was optimized using SIMD vectorization as well as how the instructions were scheduled. It also contains detailed explanations of the results obtained.
6. **Conclusion**: This chapter contains the conclusions of the thesis. It also describes its limitations and how the project can be expanded in the future.

Chapter 2

Literature Review

As mentioned earlier, to improve the performance of DSP algorithms in a real time system, vectorization using SIMD instructions was the chosen method and is explored in this thesis. This chapter gives some background and information about:

- What a real time system is and the constraints with which DSP algorithms run such systems.
- Information about the ARM architecture.
- Different methods of execution time measurement.
- Description of each algorithm and relevant work on vectorizing it with SIMD instructions.

2.1 Real Time Systems

A real time system, by definition, is that which reacts to stimuli within a finite and predictable amount of time[15]. Proper functionally of such systems not only require logical correctness of the output, but also, the amount of physical time within which these outputs are produced[16]. These stimuli can come from an external source such as the environment, e.g, a thermostat to control the air temperature, or can be periodical as well. A real time system is usually designed to perform a dedicated task or function, usually with high reliability requirements, and can accept inputs needed for that particular task or function. One of the main characteristics of real time systems is that they are aimed to operate continuously on the stimuli with minimal downtime. The amount of time taken to respond to the stimuli is known as the latency or lag, and usually occurs due to the processing time[1][15]. Certain systems require that the incoming stimuli or signal needs to be processed within a specified amount of time, to ensure smooth and stable operation of the system by the user.

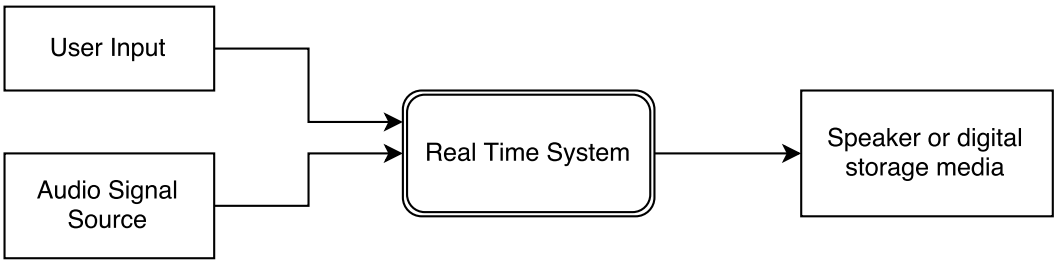

Figure 2.1: Typical real time audio processing system.

An audio or media processing system as shown in Fig. 2.1 is usually classified as real time system[1], where the incoming audio signal represents the stimuli and the output is the processed audio signal. Certain real time systems perform digital signal processing on these incoming signals. The computation is required to be completed within a certain time period, failure of which will result in a non reactive system with improper functionality. For example, a speech recognition system or vocal processing system should have very good response times otherwise it will be quite unusable.

2.1.1 Characteristics

A **task** or **thread** is a single unit of work which operates on an input and produces an output. A typical real time system carries out one or more of such tasks. The **response time** of a real time system is the amount of time within which the system has to process and produce the output corresponding to the stimuli. This is often dictated by the requirement and needs of the system. The time instant within which a meaningful and correct output should be produced is called a **deadline**[16]. Usually, these systems are run on dedicated hardware with a supporting operating system which helps in the scheduling of such tasks. Real time systems are ideally deterministic in nature, where all relevant details and characteristics of the tasks, their deadlines and their schedules are known before the system is run. In summary, real time systems have dedicated functionality and tight timing constraints[1].

2.1.2 Events in a real time system

The events or stimuli to which real time systems respond to can be classified in three categories depending in the nature of occurrence. They are

- **Synchronous Events**[15] : These events occur in regular time intervals and are periodical in nature. Tasks handling such events can be easily scheduled due to the fact that the periodicity of such events are known in advance. As in the case of the thesis, the audio processing system has synchronous events, since the audio source produces samples of audio data in periodic intervals.

- **Asynchronous Events**[15] : These events are irregular in nature and can occur at any point in time. Since they are unpredictable, much effort is put into the design phase to ensure these events are handled and scheduled properly. An example would be a keyboard interrupt in a computer.

- **Isochronous Events**[15] : These events are periodic in nature, but only within a certain time window.

2.1.3 Hard and Soft Real Time Systems

Since real time systems operate with tight timing constraints, they can be classified into two categories depending on their usage and the consequences which occur when they fail to meet the required response times. They are:

- **Hard Real Time Systems**: When a real time system fails to meet a deadline and as a consequence, leads to the failure of the system, then these systems are classified as Hard Real Time systems[15][16]. Deadlines in a real time system are referred to as hard deadlines. An extension to hard real time systems are safety critical systems, failure of which can have catastrophic results; e.g, an autopilot system in an aircraft.

- **Soft Real Time Systems** : When a failure to meet the required deadline does not result in total system failure but only improper functionality, then these systems are classified as soft real time systems. Such deadlines are also knows as **soft deadlines**[15][16].

Usually, real time systems can have one or more hard deadlines as well as soft deadlines. Certain tasks which are critical to the functionality are delegated with hard deadlines. Tasks which support the system in general and do not have direct effect on its stability have soft deadlines. If there are one or more hard deadlines, then the system is classified as a hard real time system, otherwise it is a soft real time system. In the design phase, it is important to accurately predict the best case and worst case execution times of these tasks, so that the stability of the system can be guaranteed and the tasks can be scheduled effectively[15].

Audio or media processing systems which carry out DSP fall into the category of hard real time systems[1][15]. These algorithms usually carry out a large number of floating point calculations periodically and can take significant amount of time and CPU resources. Hence the tasks processing it might not have the response time required which can result in system failure.

2.1.4 Embedded Hardware and Processors

Embedded processors are those which are usually dedicated to run a particular functionality. Unlike their general purpose counterparts which are used in desktop systems, they are usually smaller in size, have lesser processing capability, smaller

memory footprint and low power requirements. However, coupled with software written for their architectures, embedded processors are designed to execute the task at hand with high efficiency, reliability and with minimal requirements. The embedded processor, along with its software, input sources and outputs are commonly referred to as an **Embedded System**[1]. For a real time system, embedded processors are the most suitable computational units for implementation. It can be readily interfaced with inputs and outputs as well as dedicate all of its resources to the needs of the system.

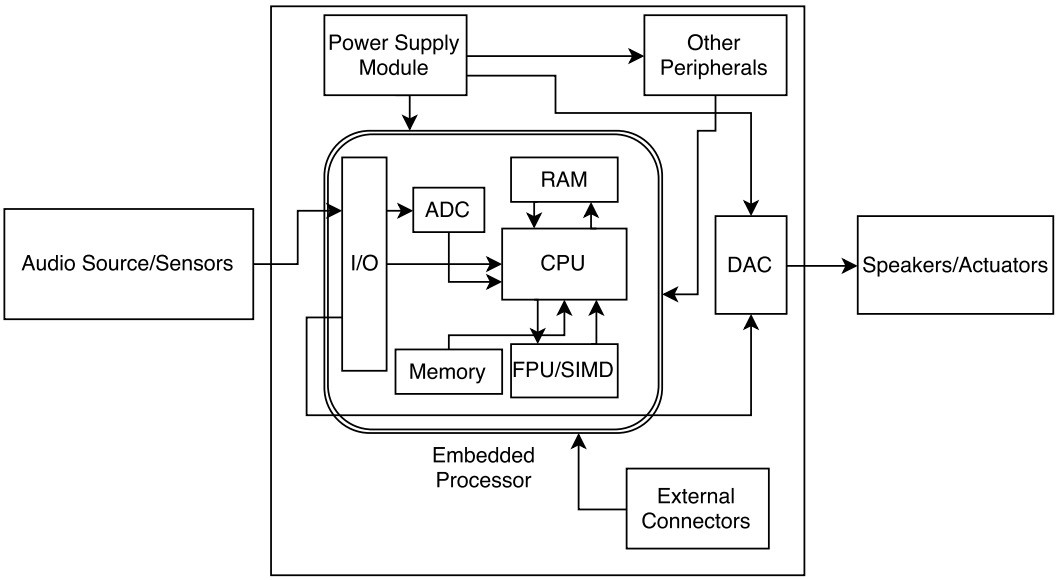

Figure 2.2: Typical embedded system

As shown in Fig. 2.2, a typical embedded system consists of:

- **Processor or SoC**: This is the main component of the system. It has the CPU and Caches, RAM and the Graphics Processing Unit(GPU) in a single die. Apart from these, embedded processors nowadays also come with a myriad of supporting hardware units for faster computation, storage and communication. Some examples are controllers for I2C, Ethernet, USB, analogue to digital conversion and vice versa, PWM etc. The CPU is also assisted by co-processors such as a Floating point unit or a SIMD unit. Since all required hardware units exist on a single die, they are often referred to as **System on Chip**(SoC) devices.

- **Memory Storage Device**: Usually embedded processors come outfitted with an off-chip memory storage device. This is usually comprised of flash memory and serves to store program memory and files.

- **GPIO Pins**: GPIO stands for General Purpose Input/Output. There pins are used to interface the SoC to the outside world.

- **System Peripherals**: Some embedded boards come with peripherals such as an LCD display, Bluetooth, Zigbee radio and Wifi network cards. These add to the overall functionality of the system.

- **Power Supply Module**: As the name suggests, the power supply module delivers power to the SoC and all other peripheral units. Embedded devices usually have low power requirements in the range of 5 to 25 watts.

- **External Connectors**: These pins and connectors help to connect other peripherals to the SoC for expandability. The most common ones are JTAG and the serial port, which aide in debugging and direct communication to the SoC with an external device such as a laptop.

- **Input sources/Sensors**: In a typical use case, an embedded processor used in a real time system accepts inputs from input sources. These sources can include sensors which give signals or data, a storage media from which the SoC reads input data or interactions from the user such as button presses.

- **Output devices/Actuators**: Output devices or actuators are connected either to the GPIO pins or other peripherals. In a real time system, the SoC processes the input data or signals from the input source and sends the required output through the actuators. Some examples are motors or relay switches in a control system, speakers for an media processing system or even network interfaces in the case of Internet of Things(IoT) applications.

- **Application Software**: The software plays a huge role in any embedded system and is comprised of tasks needed for the particular functionality. Embedded software development is quite different from that of mainstream desktop software. It is tuned for the underlying architecture, and is usually accompanied by a real time operating system which acts as an interface between the application software and the hardware.

2.2 The ARM Architecture

The architecture used in this thesis is from ARM, specifically the ARMv7-A architecture. It is an evolution of earlier ARM models and is widely used for real time embedded systems. It is a Reduced Instruction Set Computer(RISC) and has the following relevant features[17]:

- It is a 32 bit machine, meaning it has 32 bit wide internal registers for computation and has an address space of 2^{32} bytes. There are 16 accessible internal registers as shown in Fig. 2.3. The register file contains 12 general purpose registers used for general integer calculations, one program counter(PC) register which holds the address of the next instruction, one stack pointer(SP), one link register(LR) which holds the return address when a function call is made

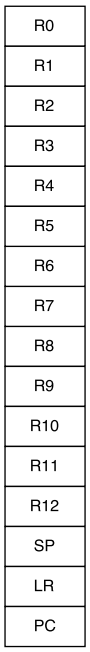

Figure 2.3: Graphical representation of the ARMv7 register file as illustrated in [17].

and an Application Program Status Register(APSR) which gives information about status of the task or thread currently executing in that particular core[17].

- It provides supports conditional instructions. They are added as suffixes to the actual instructions and the core only executes them if the conditions are met. This is aided by four APSR flag bits **N,Z,C,V**, which represent Negative, Zero, Carry and Overflow occurrences respectively[17]. A combination of these four results in multiple condition codes which can aide in the flow of the program. When a comparison instruction is executed, these bits are automatically affected. If the following instruction is suffixed by condition codes, its execution depends on these flags. This technique eliminates the need for branching, which can lead to pipeline hazards and flushes.

- It supports two instruction sets, **ARM** and **Thumb2**. Being a RISC machine, ARM instructions which are 32 bits long, need more instructions to complete a function unlike their Complex instruction Set Computer(CISC) counterparts. This results in more memory and lesser code density, which can be a problem in embedded systems[19]. Hence the Thumb instruction set was introduced. It is only 16 bits wide, and is a subset of ARM instructions[20]. It still can work on 32 bit data since the instructions are expanded into a 32 bit equivalent in the hardware, thus ensuring full functionality. Not all ARM instructions exist on the Thumb side and registers R8 - R12 are not accessible[20]. The ARM and Thumb states can be interchanged from one to another in the code.

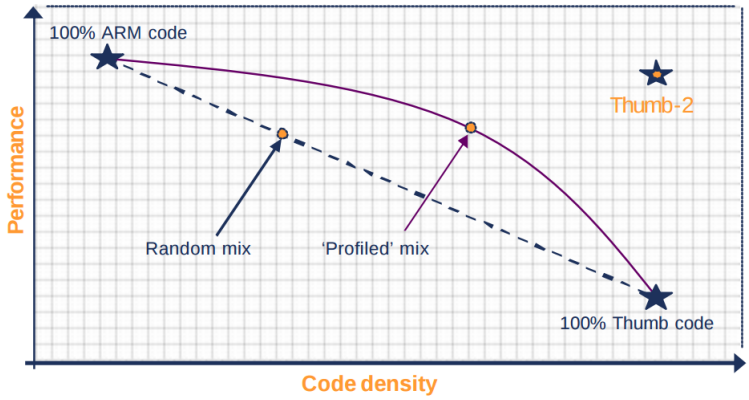

Figure 2.4: Performance to Code Density comparison between ARM and Thumb states as illustrated in [18].

Figure 2.4 shows the performance to code density ratio as the program code mixture changes from ARM to Thumb. Usually performance critical parts of the code are programmed with ARM instructions whereas the others are in Thumb, to help in higher code density which can result in better code caching. The Thumb2 instruction set is an extension to Thumb, providing 32 bit instructions as well and some few instructions to enhance functionality.

- It has the Advanced SIMD and Vector Floating Point(VFP) units which support the **NEON** and **VFPv3** instruction sets respectively. The Advanced SIMD unit, as the name suggests, is a true SIMD capable co-processor which can be used to accelerate signal processing intensive algorithms such as graphics, media encoding and decoding and audio processing. Being a co-processor, it can run in parallel to ARM, thus saving power and time[7]. The VFP, on the other is a floating point hardware acceleration unit. It provides support for IEEE-754 single and double precision floating point units and capable of working on vector data sets. However, ARM has officially deprecated the use of VFP for vector operations and recommend the newer Advanced SIMD unit for such purposes[21][22]. The hardware sequentially executes the operations and now the intended usage of VFP was to improve code density in lower end ARM models.

2.3 The ARM Cortex-A15

One of the commonly used implementations of the ARMv7 architecture is the ARM Cortex-A series. Each variant in this series have slightly different implementations of this architecture. It consists of a group of 32 bit processors which run the ARMv7 and Thumb instruction sets, with various features and capable of running complex operating systems. The following subsections deal with relevant information about

one of the processors from this family, the **Cortex-A15**, which were used for this thesis. The ARM Cortex-A15 has a 15 stage integer pipeline[22] out of order pipeline along with a 2 way set associative 32KB L1 data and instruction caches and a 16 way set associative L2 cache shared by all the cores in the system. It offers a myriad of features such as Advanced SIMD Unit, VFPv4 and PMUv2[22].

2.3.1 Pipeline

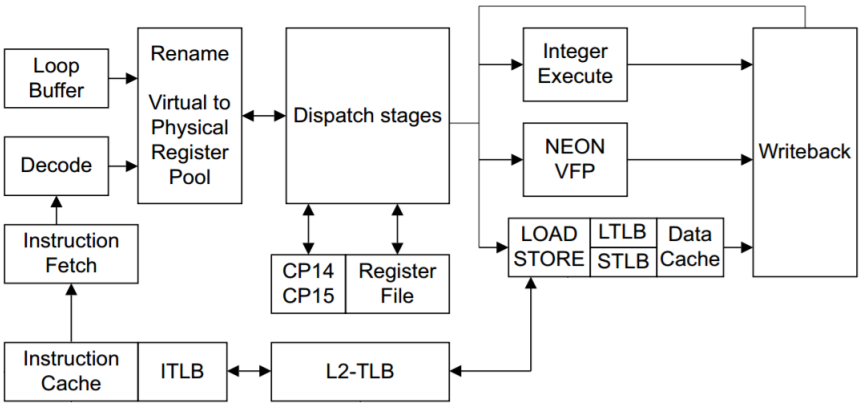

Figure 2.5: The ARM Cortex-A15 pipeline stages illustrated in [22].

The ARM Cortex-A15 pipeline is an Out of Order pipeline. As shown in Fig. 2.5, the Instruction Fetch stage consists of 5 substages where the instructions are fetched from the L1 cache. It can issue 3 instructions in a single clock[22]. Following this are 7 more stages of the pipeline which include the instruction decoding stage where ARM, THUMB and NEON/VFP instructions are decoded, the register renaming stage for both ARM and NEON register files and instruction dispatch stage, which contains all the registers and the instructions are sent from here to the execution cluster[23]. The Out of order capability in the decoding and execution stage greatly helps in utilizing the instruction level parallelism.

The execution cluster has the following execution units as shown in Fig. 2.6. Instructions to each unit can be issued simultaneously from the issue queue, thus increasing instruction parallelism[23]. Below is a description of each unit.

- Two single stage integer ALU and shifter pipelines.
- Two 9 stage NEON/FPU pipelines with 8 entries each. Hence two instructions can be issued and executed in the same cycle. The NEON pipeline is fully

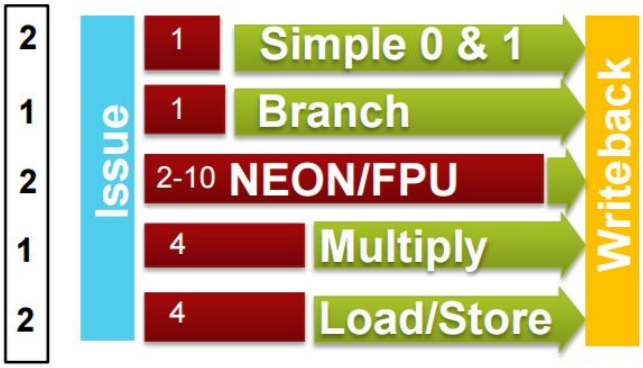

Figure 2.6: The ARM Cortex-A15 pipeline execution units as illustrated in [23].

integrated with the ARM pipeline[23], unlike previous iterations of Cortex-A series such as Cortex-A9, where the NEON unit has is its own decoupled Decode and Execution Stages. This allows for the NEON instructions to be scheduled out of order. This is the longest path an instruction can take in this processor, also known as **critical path**.

- Four stages of load and four stages of store. It supports a 128 bit wide data path. Loads can be issued out of order but cannot bypass a store. A store is always issued in order[24]. This results in a very valuable feature later used in the execution time measurement section.
- A single stage of branching logic and condition code logic.
- A 4 stage **integer** MAC and divide unit.

2.3.2 Advanced SIMD(NEON) Unit and Instruction Set

The Advanced SIMD(NEON) unit is a media processing engine capable of yielding very high performance through vectorized operations. It has support for performing SIMD operations on integer and single precision floating point data[17]. It supports operations on 8, 16, 32, 64 or 128 bits of data and can do various logical and mathematical operations, data-type conversions and memory transfer operations. It has its own 2048 bit register file, which can be viewed (in the perspective of the programmer) as thirty two doubleword registers (D0-D31) or sixteen quadword registers (Q0-Q16)[25][17].

As shown in Fig. 2.7, any register D_{2n} can be mapped to the least significant half of Q_n and $D_{(2n+1)}$ maps to most significant half of Q_n[17]. This helps in the versatility of operations and suits cases where it is not possible to work with the entire quadword. Figure 2.8(A) shows quadword register Q_0 filled with four single precision floating point numbers and Fig. 2.8(B) shows quadword register Q_0 filled with sixteen 8 bit unsigned integers.

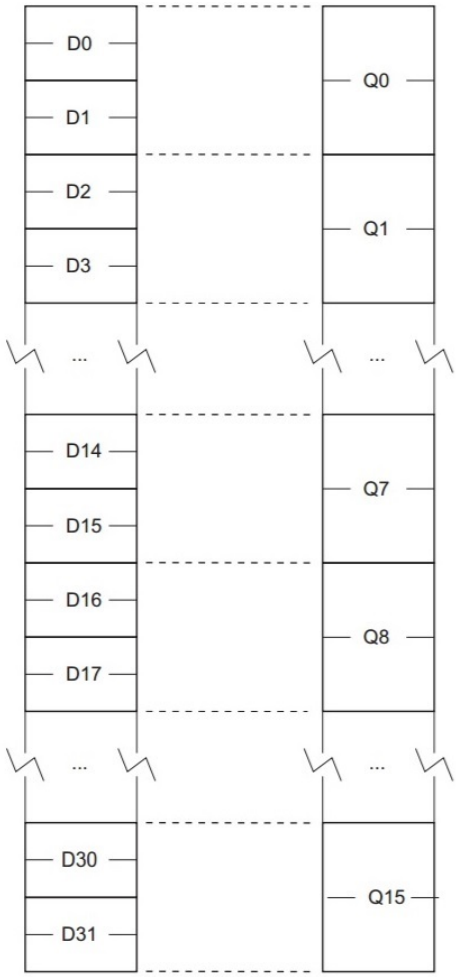

Figure 2.7: Quadword and doubleword register mapping in Advanced SIMD unit as illustrated in [25].

As mentioned earlier, The NEON unit is completely integrated into the ARM pipeline, as opposed to previous iterations in the Cortex-A series where it was decoupled and had its own fetch, dispatch, execute and load store operations. Also, the Cortex-A9 could perform operations only on 64 bits at a time.

No documentation for the pipeline structure and architecture of the Advanced SIMD unit is available for the ARM Cortex-A15. Nonetheless, the decoupled pipeline implementation of the NEON unit in the ARM Cortex-A8 gives an idea of the internal structure as shown in Fig. 2.9. It has three integer pipelines (integer MAC, Shifter and ALU) and two Floating point pipelines. The Advanced SIMD unit's pipeline in the ARM Cortex-A15 also sports an FMAC unit[23]. One important point to note is that the LOAD/STORE unit can work in parallel to the execution units, a feature which is utilized in this thesis. NEON provides in-

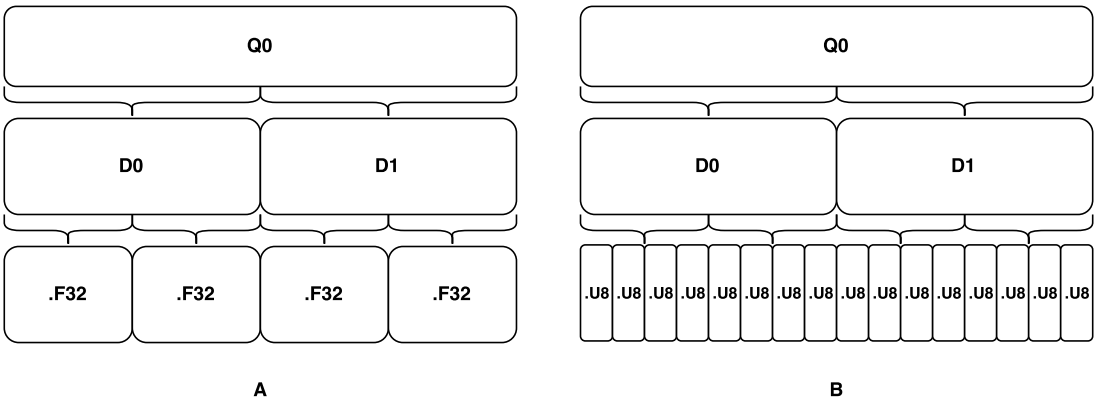

Figure 2.8: Two examples of register packing.

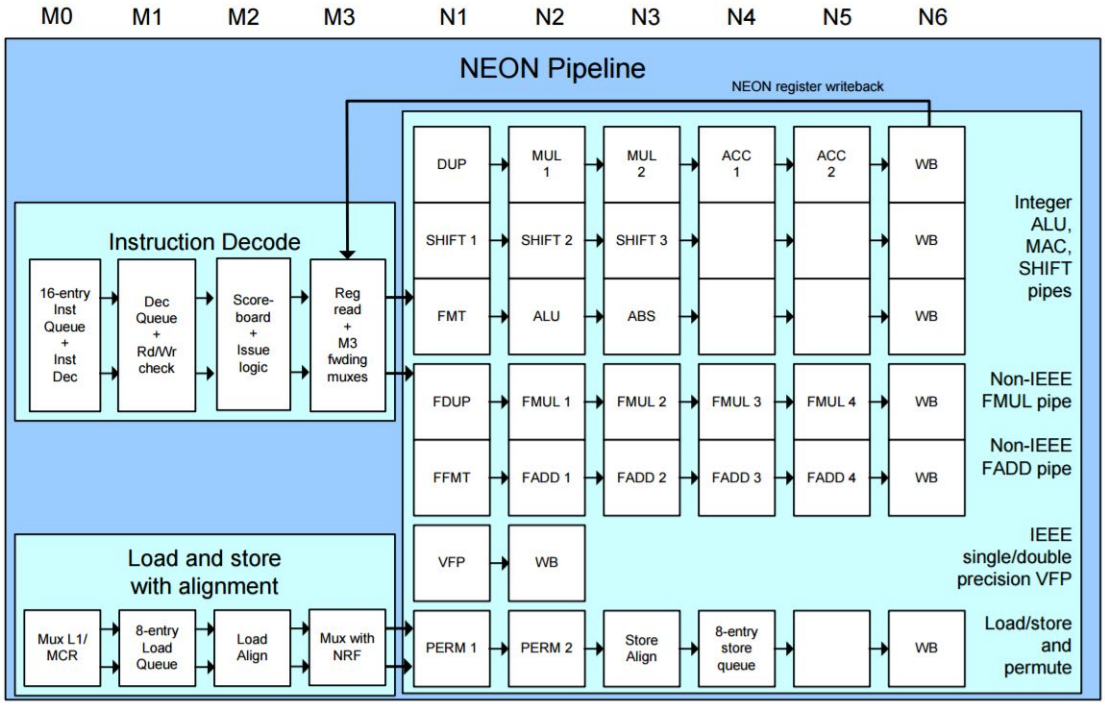

Figure 2.9: The internal structure of the Cortex-A8 NEON unit illustrated in [26].

structions for Integer MUL, ALU and shifting operations as well as non IEEE 754 compliant Floating point MUL and ADD support. ARM has, by default, enabled **Flush to Zero** on the Advanced SIMD unit[17]. This means that floating point numbers which are denormal in nature are flushed to zero by default, rather than executing support code to handle them. This helps to improve performance speed with the cost of sacrificing a little bit of accuracy, which usually presents itself as rounding of noise. Moreover, it follows the policy of rounding to the nearest avail-

able floating point number. Hence they are not fully IEEE 754 compliant.

The NEON instructions support operations between two vectors as well as between a vector and a scalar. These instructions can be placed under ARM or THUMB mode. Description of instructions which are relevant to this thesis are described in the subsections below. NEON instruction follow the format as shown below:

```
Vop{cond}.datatype {Qd}, Qn, Qm
                or
Vop{cond}.datatype {Dd}, Dn, Dm
                or
Vop{cond}.datatype {Qd}, Qn, Dm[x]
                or
Vop{cond}.datatype {Qd}, Qn, #immediate
```

where:

- All NEON instructions start with the prefix "`V`"
- "`op`" stands for operation. E.g. MUL, ADD, MAC, ABS.
- Q_d is the destination quadword register, Q_n and Q_m are the source registers. For example a VMUL instruction can be written as follows:

  $$\begin{aligned} \texttt{VMUL.F32} \quad & Q_0, Q_1, Q_2 \\ & Q_0 = Q_1 \odot Q_2 \end{aligned} \tag{2.1}$$

  Equation 2.1 shows the equivalent mathematical representation of the operation; vectorized multiplication is represented by the symbol $\odot$. Q_0 is the destination vector and Q_1 and Q_2 are the source vectors. Since the suffix "`.F32`" is added to the instruction, it implies that each source register is filled with four single precision floating point numbers.

- The instructions can be also used to work on doubleword registers. D_d is the destination doubleword register. D_n and D_m are the source doubleword registers. A similar example would be:

  $$\begin{aligned} \texttt{VADD.U8} \quad & D_0, D_1, D_2 \\ & D_0 = D_1 \oplus D_2 \end{aligned} \tag{2.2}$$

  Equation 2.2 shows the equivalent mathematical representation of the operation; vectorized addition is represented by the symbol $\oplus$. Here doubleword registers D_1 and D_2 are filled with eight 8 bit unsigned integers and the result of the multiplication operation is placed in D_0.

- "`x`" represents the either the upper or lower half of the doubleword register D_m. `x` can be either 0 or 1 to represent the upper or lower half respectively. This notation is helpful to multiply vectors with scalars. An example would be:

$$\begin{aligned} \texttt{VMUL.F32} \quad & Q_0, Q_1, D_4[0] \\ & Q_0 = Q_1 \odot D_{(4,0)} \end{aligned} \tag{2.3}$$

  Here quadword register Q_1 is a vector containing four single precision floating point numbers and it is multiplied with one single precision floating point number in the upper half D_4, and the result is placed in Q_0. the scalar can also be specified as an immediate using the "`#`" symbol.

- "`cond`" stands for condition code. As explained earlier, condition code eliminates the need for branches by checking if the condition holds true before executing the instruction. However, with the exception of VFPv4 instructions and NEON instructions which are also common to VFP instructions, most NEON instructions do not affect any of the condition flags[25][17]. Since it is not applicable to NEON, it is ignored when writing the instruction.

- "`datatype`" refers to the type of data this operation is performed on. It can be signed/unsigned integer or floating point. It is written in the format `datatype<size>`. For example, the operation `VMUL` can operate on vectors having the following data types[8][7]:

  1. `U8, U16, U32` and `U64` to represent unsigned integer of length 8,16,32 and 64 bits respectively.
  2. `S8,S16, S32` and `S64` to represent signed integer of length 8,16,32 and 64 bits respectively.
  3. `I8,I16, I32` and `I64` to represent signed or unsigned integer of length 8,16,32 and 64 bits respectively.
  4. `F16` and `F32` to represent half precision and full precision floating points respectively.
  5. `P8` and `P16` to represent polynomials over 0,1[17].

There are two system registers which the NEON unit is associated with. They are the **FPSCR and FPEXC**[17] registers. FPSCR stands for Floating Point Status Control Register. It is responsible for the control of the Advanced SIMD and VFP unit and denoting whether exceptions or saturation's have occurred after a floating point or integer calculation is performed. It is accessible through user space mode. The FPEXC register is responsible for enabling/disabling of the advanced SIMD unit/VFP and denoting how much of information is needed to be saved for performing a context switch.

2.3.3 Load and Store Operations with the Advanced SIMD Unit

The Advanced SIMD unit is capable of up to 128 bit wide memory transfers. It shares the same Load/Store unit with the ARM integer pipeline in the ARM Cortex-A15, as shown in Fig. 2.5. The NEON instruction set provides the `VLD` instruction for load operations and `VST` instruction for store operations. These instructions are very powerful since they offer fast aligned memory accesses along with the capability of interleaving and deinterleaving[8] data. The format of NEON load is:

```
Vopn{cond}.datatype list, [Rn{@align}]{!}
e.g. : VLD1.F32 {D0,D1,D2,D3}, [R0@128]!
```

where:

- `op` refers to the operation, which is either `LD` for load and `ST` for store.
- `n` is an integer which represents how many elements of the specified datatype should be fetched or stored in the memory. It can be `1,2,3 or 4`.
- `list` denotes the list of registers the data is loaded from or stored too. Depending on the value of `n`, this list can help deinterleave data when loading from memory and interleave them back when stored. Detailed description is explained in the following subsection.
- R_n is the ARM integer register which contains a pointer that points to the memory location.
- Adding the `"!"` increments the base address R_n by the number of bytes fetched or stored, after the operation is completed. Otherwise it remains the same.
- It is also possible to offset the base address in R_n with a value in another register R_m, so that the new address would be $R_n = R_n + R_m$. An example would be:

  ```
  VLD1.F32 {D0,D1,D2,D3}, [R0@128],R1
  ```

- `alignment` is an optional argument that specifies how the data is aligned in the memory when performing the load or store. Improperly specifying the alignment or trying to access unaligned memory with alignment specified in the instruction is not permitted and will cause an exception.
- The instruction in the above example loads 8 elements of the datatype `.F32` and stores them in doubleword registers D_0, D_1, D_2 and D_3.

The NEON instruction set provides options for dealing with interleaving and deinterleaving of data. These also help arrange the data in registers in a such a way that favors more vectorization possibilities. Since this thesis deals with single precision floating point numbers, instructions dealing with the datatype `.F32` along

with relevant operational capabilities are explained below. The NEON instruction set offers three ways to load or store data. Their syntaxes are the same as explained in the previous subsection with the exception being the "`list`" field. They are:

1. **Single element from memory into single lane**[8] type of LOAD/STORE operation. This instruction takes data from memory and slots it into the upper or lower half of the doubleword registers. Table 2.1 below gives an idea of the available list options[7]:

Number of Elements n	*list* **field**
1	Dd[x]
2	Dd[x], D(d+1)[x] Dd[x], D(d+2)[x]
3	Dd[x], D(d+1)[x], D(d+2)[x] Dd[x], D(d+2)[x], D(d+4)[x]
4	Dd[x], D(d+1)[x], D(d+2)[x], D(d+3)[x] Dd[x], D(d+2)[x], D(d+4)[x], D(d+6)[x]

Table 2.1: Possible list fields for the any data type of 32 bits in length.

```
Example A (n=4):
    VST4.F32 {D0[0], D1[0], D2[0], D3[0]}, [R0@128]!
    VST4.F32 {D0[1], D1[1], D2[1], D3[1]}, [R1@128]!

Example B (n=4):
    VLD4.F32 {D0[0], D2[0], D4[0], D6[0]}, [R0@128]!
    VLD4.F32 {D0[1], D2[1], D4[1], D6[1]}, [R0@128]!
    VLD4.F32 {D1[0], D3[0], D5[0], D7[0]}, [R0@128]!
    VLD4.F32 {D1[1], D3[1], D5[1], D7[1]}, [R0@128]!
```

Example A deals with a case when 32 bit data are operated on in pairs by interleaving them in registers, as shows in Fig. 2.10, but the arrays X and Y have to be stored in continuous fashion in the memory. The memory locations for X and Y are addressed by the pointers in R_0 and R_1 respectively. The store of X elements is carried out by the first instruction. It stores the data in $D_{(0,0)}, D_{(1,0)}, D_{(2,0)}$ and $D_{(3,0)}$. Y array is stored by a similar fashion by the next instruction which acts on the lower half of the doubleword registers. Hence these instructions took 4 single elements and stored them into 4 single slots in the memory. Figure 2.10 shows the graphical representation of the operation. Note that the memory is shown as blocks of 4 bytes each.

Example B deals with the case when data is present in an interleaved fashion in the memory, as shown in Fig. 2.11, and they need to be deinterleaved for maximum vectorization potential. So the first instruction loads four 32 bit

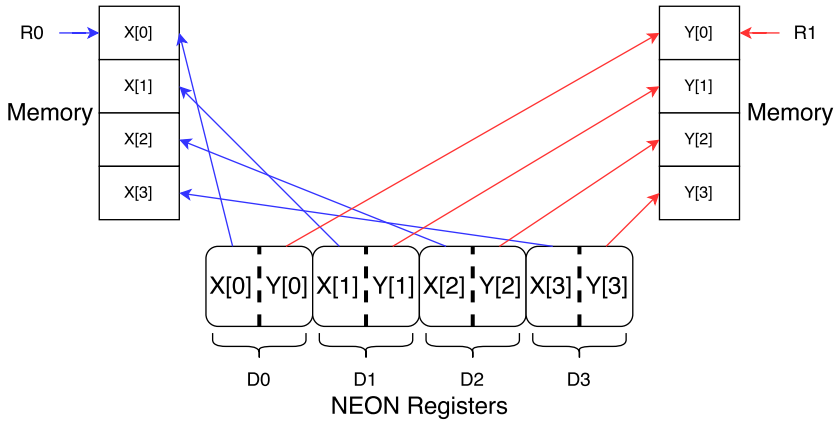

Figure 2.10: Storing operation as carried out by instructions in Example A. The value in $R0$ and $R1$ point to the start addresses. The **blue** arrows represents the operation performed by the first instruction in Example A and the **red** arrows represents the operation of the second instruction.

elements from the memory and slots them into the upper halves of D_0, D_2, D_4 and D_6, as represented by the black arrows. The second instruction takes the next four 32 bit elements from the memory and slots them into the lower half of the same registers, as represented by the blue arrows. The third instruction loads the next four elements into the upper halves D_1, D_3, D_5 and D_7 (represented by the red arrows) and the fourth instruction loads the next four elements into their lower halves (represented by the green arrows). In this way, quadword registers Q_0, Q_1, Q_2 and Q_3 are filled with elements of the arrays P, Q, R and S respectively.

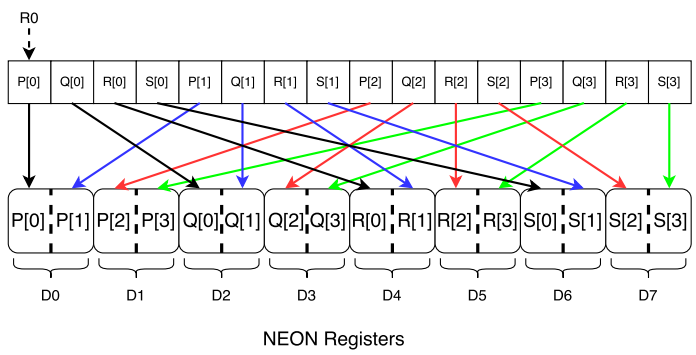

Figure 2.11: Loading operation as carried out by instructions in Example B. Note that the memory is represented in blocks of 4 bytes each. The value in $R0$ points to the start addresses.

2. **Single n element from memory into multiple lanes:** This instruction copies a single n element structure into all the lanes[7] mentioned by the list field, thereby creating multiple copies. It is also possible for interleaving them. This feature only exists for load operation and is useful when dealing with constants. Table 2.2 shows the possible values for the *list* fields. Each

doubleword register will have, in the case of 32 data, two copies in both the upper and lower halves.

Number of Elements n	*list* **field**
1	Dd[ ] Dd[ ],D(d+1)[ ]
2	Dd[ ], D(d+1)[ ] Dd[ ], D(d+2)[ ]
3	Dd[ ], D(d+1)[ ], D(d+2)[ ] Dd[ ], D(d+2)[ ], D(d+4)[ ]
4	Dd[ ], D(d+1)[ ], D(d+2)[ ], D(d+3)[ ] Dd[ ], D(d+2)[ ], D(d+4)[ ], D(d+6)[ ]

Table 2.2: Possible list fields for the any data type of 32 bits in length. Each element will be slotted into both the halves of the doubleword register.

```
Example : VLD4.F32 {D0[ ],D1[ ],D2[ ],D3[ ]},[
    R0@128]
```

In the example, four 32 bit elements are fetched from the memory. The first element is duplicated in the both lanes of D_0, the second one is duplicated in both the lanes of D_1 and so on. The graphical representation is shown in Fig. 2.12.

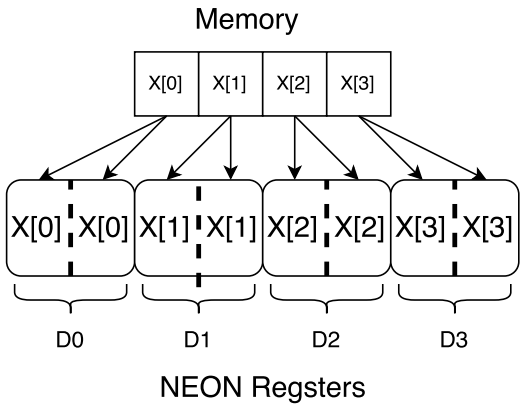

Figure 2.12: Loading operation as carried out by instructions in the example. Note that the memory is represented in blocks of 4 bytes each. Each element of data in the memory is duplicated in both the lanes of each doubleword register.

3. **Multiple n elements from memory to multiple lanes**. As the name describes, the lists also offer the capability of loading multiple data elements from the memory and slotting them into the doubleword registers. Conversely it is also capable of storing this assorted data present in the registers back to

the memory. This type of instruction is very commonly used when no interleaving is necessary, especially just to load a chunk of data from the memory into one or more quadwords, work on it and then store it back.

The value of $\boldsymbol{n}$ decides what elements in the memory are grouped together and slotted into the doubleword register. To understand this, it can be viewed as number of strides. For example, if $\boldsymbol{n = 2}$ then every second element in the memory is considered as one multi-element structure and is slotted into a doubleword register. If $\boldsymbol{n = 1}$ then it groups elements in a contiguous manner. ***list*** denotes the destination doubleword registers of these structures and how many of them are to be filled. Conversely this concept applies in reverse when performing a store operation. Examples below explain with greater detail. Table 2.3 below shows the ***list*** fields.

Number of Elements n	***list* field**
1	Dd Dd, D(d+1) Dd, D(d+1), D(d+2) Dd, D(d+1), D(d+2), D(d+3)
2	Dd, D(d+1) Dd, D(d+2) Dd, D(d+1), D(d+2), D(d+3)
3	Dd, D(d+1), D(d+2) Dd, D(d+2), D(d+4)
4	Dd, D(d+1), D(d+2), D(d+3) Dd, D(d+2), D(d+4), D(d+6)

Table 2.3: Possible list fields for the any data type of 32 bits in length. Every two $\boldsymbol{n^{th}}$ element will be grouped together and slotted into both the doubleword register specified by the list field.

```
Example A (n=1) : VLD1.F32 {D0,D1,D2,D3}, [R0@128]!
Example B (n=2) : VLD2.F32 {D0,D1,D2,D3}, [R0@128]!

Example C (n=4) : VLD4.F32 {D0,D2,D4,D6}, [R0@128]!
                  VLD4.F32 {D1,D3,D5,D7}, [R0@128]!
```

In **Example A**, since $\boldsymbol{n = 1}$ and four doubleword registers are specified in ***list***, every pair of adjacent elements in the memory form a 2 element structure. So four sets of these structures are loaded in $\boldsymbol{D_0, D_1, D_2}$ and $\boldsymbol{D_3}$ as shown in Fig. 2.13. This is used quite often in the implementation of this thesis to load chunks of contiguous data into quadword registers. In **Example B**, since $\boldsymbol{n = 2}$, every other element is considered a 2 element structure and four of these are slotted into the specified doubleword registers. A typical use case (including this thesis) is when an array of complex numbers are contiguously

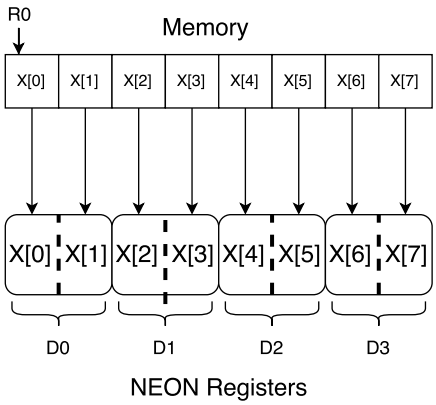

Figure 2.13: Loading operation as carried out by instructions in the example A. Note that the memory is represented in blocks of 4 bytes each.

stored in the memory as shown in Fig. 2.14. This instruction pairs 2 real parts and 2 imaginary parts together and slots them into the doubleword registers. The result of the operation is that quadword register $\boldsymbol{Q_0}$ is filled with real part values and quadword register $\boldsymbol{Q_1}$ is filled with imaginary part values. **Example C** gives the same result as that shown in Fig. 2.11. Since

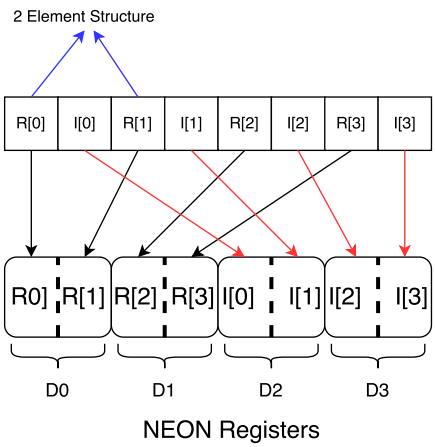

Figure 2.14: Loading operation as carried out by instructions in the example B. Note that the memory is represented in blocks of 4 bytes each.

$\boldsymbol{n = 4}$, every fourth element is paired together. The first instruction pairs $\boldsymbol{(P_0, P_1)}$, $\boldsymbol{(Q_0, Q_1)}$, $\boldsymbol{(R_0, R_1)}$ and $\boldsymbol{(S_0, S_1)}$ and slots them into $\boldsymbol{D_0, D_2, D_4}$ and $\boldsymbol{D_6}$ respectively. Since eight elements have been loaded, the value in $\boldsymbol{R_0}$ now points to the address of $\boldsymbol{P_2}$. Then the second instruction pairs $\boldsymbol{(P_2, P_3)}$, $\boldsymbol{(Q_2, Q_3)}$, $\boldsymbol{(R_2, R_3)}$ and $\boldsymbol{(S_2, S_3)}$ and slots it into the doubleword registers $\boldsymbol{D_1, D_3, D_5}$ and $\boldsymbol{D_7}$ respectively, thus having the same result as shown in Fig. 2.11.

2.3.4 The VFP Unit

VFP stands for **Vector Floating Point** and the ARM Cortex-A15 implements the VFPv4 specification[22]. As the name suggests, it is used to perform floating point operations on single and double precision floating point numbers[22]. Little documentation exists about the hardware architecture of the VFP unit, but it in both [26] and the ARM Cortex-A8 technical manual, it is mentioned that it is **non-pipelined**. However Qualcomm, a company who possesses an architectural license from ARM, implemented a pipelined version of VFPv3 and VFPv4[27]. VFP units are much slower than their NEON counterparts and are generally used to perform scalar operations.

In the Cortex-A8 pipeline, as shown in Fig. 2.9, it sits as one of the parallel pipelines within the Advanced SIMD extension unit. In the ARM Cortex-A15, it is fully integrated into the ARM pipeline and so its instructions can be scheduled out of order. It shares the register file with the NEON unit. However, as shown in Fig. 2.15, VFP unit can only view the registers as thirty two 32 bit single word registers identifiable as $(\boldsymbol{S_0 - S_{16}})$ or sixteen doubleword registers identifiable as $(\boldsymbol{D_0 - D_{15}})$[17]. The VFP unit has the same two system registers as the Advanced SIMD unit: the FPSCR and FPEXC. To enable the VFP unit, the FPEXC register must be allowed non-secure access and the FPEXC.EN bit must be set to 1. Just

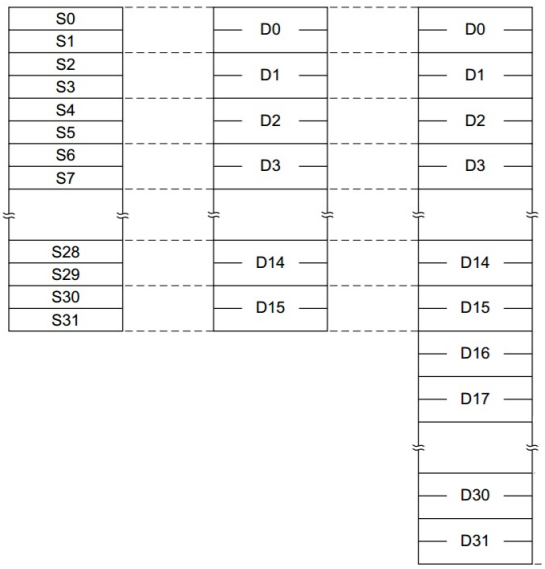

Figure 2.15: The registers as viewed by VFP unit and their equivalent mapping as illustrated in [17].

as is the case for the Advanced SIMD unit, the FPSCR controls various aspects of the VFP unit as well as trap exceptions. The VFP is fully compliant to IEEE floating point standard. Though the word "Vector" is present in the name, ARM

deprecates the use of VFP for vector operations. In the ARM Cortex-A15, only scalar operations are supported and attempts to perform vector calculations with the VFP unit will result in an undefined instruction exception[22]. According to forum posts in [28], VFP is **not a true SIMD** capable unit and executes the operation sequentially in the hardware. They are mainly used to reduce code size. Though documentation on this matter is sparse, this fact is loosely mentioned in page 13-6 in Cortex-A8 technical reference manual. It works by executing multiple iterations and using the mentioned registers as a cyclic queue.

To summarize, VFP deals majorly with scalar operations. Its instructions are similar to NEON instructions, with the exception that it is possible to work on single lanes $\boldsymbol{S0 - S32}$. By using the flag $\boldsymbol{mfpu = VFPv4}$ in GCC, the compiler will automatically generate VFP instructions for scalar floating point calculations.

2.3.5 ARM Performance Management Unit

The ARM PMU is part of the co-processor extension CP15. It helps to collect useful statistics about the operations of the CPU, cache, TLB and memory[22]. It is made up of dedicated hardware present each core in the machine. The PMU sports **6 event counters** and a **cycle counter** per core[17]. Figure 2.16 shows a graphical representation of the cycle counter and event counters. The cycle counter can count once every cycle or once in 64 cycles. In the ARM Cortex-A15, PMUv2 is implemented which offers a little bit more of features. Each counter can choose to count from 126 available events.

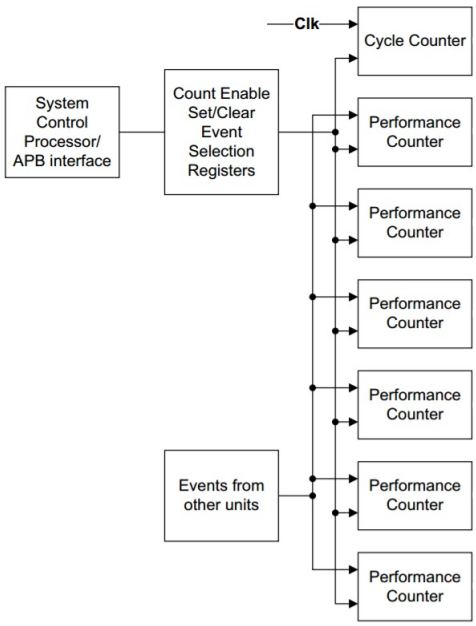

Figure 2.16: The PMU cycle counter and 6 event counters as illustrated in [17].

To control the PMU and the event counters, there following co-processor registers are used[17]:

- **Performance Monitors Cycle Count Register(PMCCNTR):** It holds the value of the cycle counter.
- **Performance Monitors Count Enable Set register(PMCNTENSET):** It is used to enable the cycle counter and any of the event counters.
- **Performance Monitors Control Register(PMCR):**. This is responsible for the main control of the PMU. For example, **bit 5** denotes whether the cycle counter counts every cycle or every 64 cycles. Setting **bit 6** will clear the cycle count register PMCCNTR .
- **Performance Monitors Overflow Flag Status Register(PMOVSR):** Denotes if any of the 6 event counters have overflown.
- **Performance Monitors Event Counter Selection Register(PMSELR):** This register is used to select a particular counter by filling the bits 4 - 0. Once a particular counter has been selected, then it can be set to count a desired event.
- **Performance Monitors User Enable Register(PMUSERENR):** The PMU registers mentioned are not accessible in user space by default. Hence setting bit 0 to one will make them accessible in user space.
- **Performance Monitors Event Count Register(PMXEVCNTR):** It is used to read the count value in the event counter. The counter has to be selected using the PMSELR register.
- **Performance Monitors Event Type Select Register(PMXEVTYPER):** After selecting the desired counter using PMSEL, this register can then be used to denote which type of event it has to count.

Some examples of events are cache read, miss and writeback events, ARM instruction executed event, Advanced SIMD or VFP instruction executed event etc.[22]. However ARM does not guarantee complete accuracy of the PMU and mentions that they are approximately accurate under normal conditions. This can be attributed to the fact that there is no definite point in the pipeline that an event occurrence is recognized and the counters are incremented[17]. Due to pipelining effects, the data given by PMU is approximately accurate. However the cycle counter can be used to count the execution cycles of a program.

2.3.6 Odroid XU4

For this thesis, the available platforms of implementation was the **Odroid XU4**. It contains a Samsung Exynos5422 SoC having a quadcore ARM Cortex-A15 and

a quadcore ARM Cortex-A7 processors[29]. Figure 2.17 shows a block diagram description of the internal components. All cores sport an Advanced SIMD and VFP unit. Some of the Odroid XU4's specifications are:

- **Processor**: Samsung Exynos5422 having ARM Cortex-A15 Quadcore 2.0GHz and Cortex-A7 Quadcore 1.4GHz.
- **Cache**: 32KB L1 Data and Instruction cache, 2MB L2 cache
- **Memory**: 2GB LPDD3
- 10/100/1000Mbps Ethernet and USB3 availability.
- Can use eMMC storage or SD Card storage, up to 64GB.

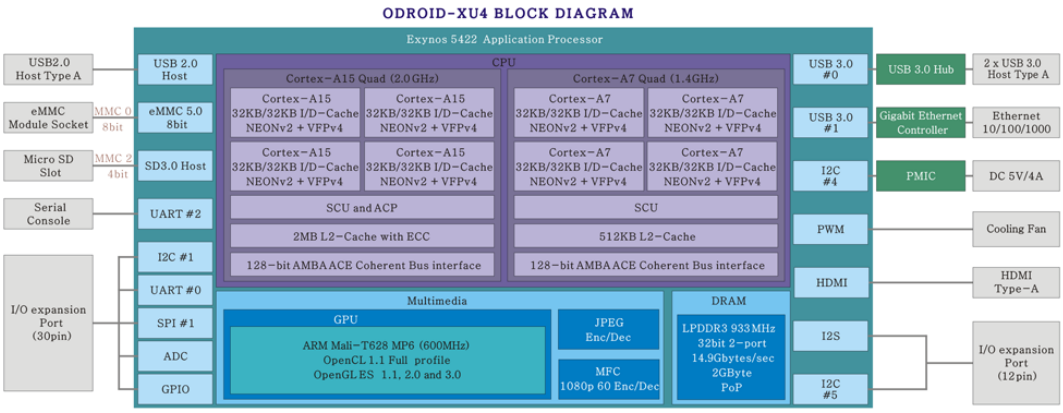

Figure 2.17: A block diagram description of Odroid XU4 as illustrated in [29].

2.4 Execution Time Measurement

Measuring the execution time of real time tasks is needed to validate whether they meet the requirements of the system, i.e, whether they meet the required deadlines. Ideally, the execution time should be measured as accurately as possible for both, the worst case and best case scenarios, and should have zero overhead. According to Wilhelm et al. (2008)[30], worst case execution time(WCET) can be primarily calculated in two broad ways, **Timing Analysis** and **Measurement Based Techniques**[30].

Figure 2.18 shows the probability distribution of a typical application's execution time when it is tested repeatedly. It is usually obtained by measuring how long a task takes from start to end. Such a measurement approach is called **End to End** measurement, and is a common practice used in industry. Wilhelm et al. (2008) warns that whilst this approach gives a comprehensive idea of the execution time,

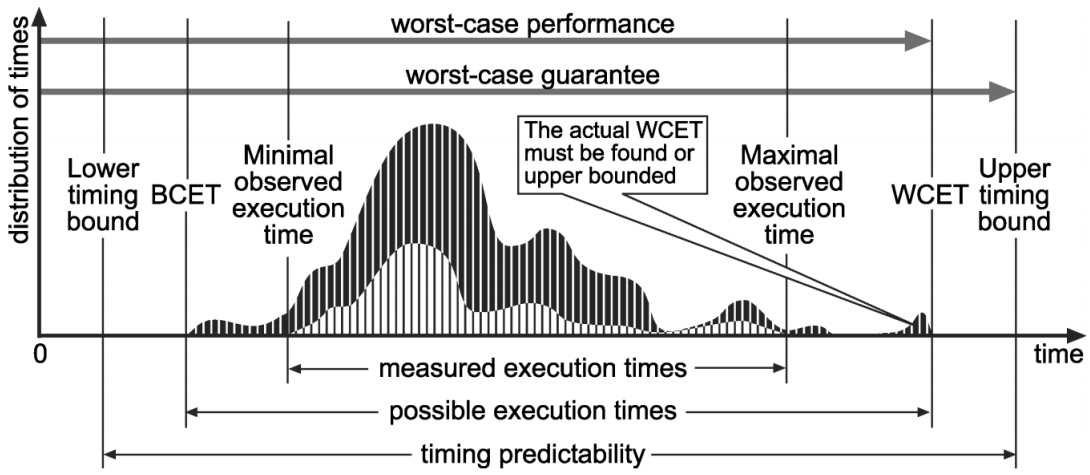

Figure 2.18: Typical probability distribution of execution times. It shows the maximum and minimum observed execution times, upper and lower bounds and WCET, as illustrated in [30].

it is unsafe because it does not show the upper and lower bounds. These bounds are needed to ensure timing predictability.

2.4.1 Static Timing Analysis

Timing analysis techniques involves the process of statically analyzing the code and estimating its bounds and WCET, often with some overestimation to ensure safety. Tools which accomplish this usually carry out the following procedures:

1. **Control Flow Analysis(CFA)**: The source code can have multiple paths which the code can take, depending on branches and conditionals. The analyzing tool must take into account all possible paths as well as all possible inputs which can be given to achieve full path coverage, and determine what the critical paths are[30]. When it comes to loops, this phase also determines what the limits of the loop iterations are. The number of paths should be finite and the program should finish in finite amount of time. This process is achieved by using control flow graphs to represent the source code. It is usually advised to perform this operation on machine code.

2. **Processor behavior Analysis**: This phase takes into account all the necessary characteristics of the processor and architecture platform the code is meant to run on, such as memory, cache, pipeline, number of cores, operating system and so on. Different processors will result in different execution times and this stage relies heavily on accurate information available from manufacturers such as abstract models of the processor along and timing models of the instruction set. It does not take into account the initial state of the

processor. Timing models are more complex for modern processors such as ARM due to its complex pipelines. Also cache behavior is difficult to model in a deterministic manner.

3. **Bounds estimation and visualization**: This phase combines the information obtained from the previous stages and estimates the upper and lower execution time and bounds.

One of the main problems associated with static analysis is the accuracy of the processor behavior analysis phase when dealing with complex processors. Since most calculation is done using theoretical information and processor models, they usually differ to an extent with real world trials. To circumvent this they usually assume the worst case scenario. Some factors which affect this are[30]

- Cache performance : The analysis usually does not take the cache performance into consideration i.e, is the code snippet cache friendly.
- Pipeline performance : Usually pipeline hazards contribute to a significant chunk of the execution time. Scheduling instructions differently to reduce hazards will yield lower execution times. Abstract models of processors such as ARM do not take this into account. Moreover, some processors follow **Out of Order** execution, which further increases the complexities of analyzing the code.
- Abstract Model Accuracy: Abstract models are basically simplified models of the processor, capable of running code and simulation of registers and hardware units. Timing information of such models do not take into account many factors like those mentioned above.

2.4.2 Dynamic Timing Analysis

Dynamic Timing Analysis or Measurement Based Analysis is the process of executing the code on hardware and measuring the elapsed time taken for execution. This can be done usually with hardware timers or cycle counters present in the processor, software timers, logic analyzers or even non intrusive trace mechanisms (ARM Embedded Trace Macrocell). Ideally, the end to end execution time for all possible paths and possible input ranges should be exhaustively tested, whilst keeping the initial conditions of the cache and pipeline as the worst case. However, it can still result in underestimation of WCET. To circumvent this, certain Static Analysis tools replace the processor behavior analysis phase with actual timing measurements of sub blocks obtained from CFA. It does not take into account the initial state of the hardware before execution of the sub block and the results can vary depending on that. The code can also be entered into an Register Transfer Level(RTL) simulation model of the processor, to obtain very accurate information of pipeline stalls, cache misses and processor states. However, manufacturers rarely disclose the RTL models to customers.

2.5 DSP Algorithms

This section presents a comprehensive background study of the six DSP algorithms and relevant work in their vectorization. Each subsection describes the algorithm, its mathematical representation, approaches taken to vectorization and what are the difficulties when it comes to vectorization.

2.5.1 NE10 Library

NE10 is an open source library created by ARM, which provides vectorized implementations for DSP, math and physics algorithms which can run on the Advanced SIMD unit. In the DSP module, basic functions such as FFT and FIR filters are implemented. However there are no implementations available for the Complex Number Multiplication, Cascaded IIR filter and Peak Program Meter algorithm. The algorithm Gain, Mix and Gain and Mix can be implemented using NE10 vectorized multiplication and vectorized addition functions.

2.5.2 Gain

Gain is a simple functionality used to either increase or decrease the magnitude of the input signal. Consider an input signal $\boldsymbol{X}$ of $\boldsymbol{N}$ samples in length. Given that a constant gain of $\boldsymbol{k}$ is to be applied on $\boldsymbol{X}$, any $\boldsymbol{n^{th}}$ sample of the output signal $\boldsymbol{Y}$ is obtained by:

$$\boldsymbol{y_n = k * x_n} \tag{2.4}$$

where $\boldsymbol{0 \leq n < N}$. Considering a SIMD machine which can work on vectors of vector length $\boldsymbol{P}$, the algorithm can be vectorized in the following manner:

$$\boldsymbol{[y_n, y_{(n+1)}, ..., y_{(n+P-1)}] = k \odot [x_n, x_{(n+1)}, ..., x_{(n+P-1)}]} \tag{2.5}$$

So this algorithm can be executed with $\frac{\boldsymbol{N}}{\boldsymbol{P}}$ instructions. **Hence it is favorable to have the buffer size $\boldsymbol{N}$ as a multiple of $\boldsymbol{P}$.** Another way of representation would be two nested loops. The inner loop calculates for the vector size $\boldsymbol{0 : P - 1}$ and the outer loop calculates for the entire sample size as shown in the pseudo code below. The inner loop represents a single SIMD instruction's operation.

Algorithm 1: Sequential Implementation of Gain function:

```
for((p=0; p<(N/P − 1); p++))
{
    for(n=0; n<(P − 1); n++)
    {
        y[p,n] = k*x[p,n]
    }
}
```

2.5.3 Mix

Mixing is another simple functionality exhaustively used in audio processing applications. Consider two input signals $\boldsymbol{A}$ and $\boldsymbol{B}$ of length $\boldsymbol{N}$ samples. To obtain any $\boldsymbol{n^{th}}$ sample of the output signal $\boldsymbol{Y}$, the corresponding $\boldsymbol{n^{th}}$ sample of two input signals $\boldsymbol{A}$ and $\boldsymbol{B}$ are added as shown in Eq. 2.6.

$$y_n = a_n + b_n \tag{2.6}$$

where $0 \leq n < N$. Considering a SIMD machine which can work on vectors of vector length $\boldsymbol{P}$, the algorithm can be vectorized in the following manner:

$$\begin{aligned}[y_n, y_{(n+1)}, ..., y_{(n+P-1)}] = {} & [a_n, a_{(n+1)}, ..., a_{(n+P-1)}] \\ & \oplus [b_n, b_{(n+1)}, ..., b_{(n+P-1)}]\end{aligned} \tag{2.7}$$

So this algorithm can be executed with $\frac{N}{P}$ instructions. A pseudo code of it would be as given below. The inner loop represents a single SIMD instruction's operation.

Algorithm 2: Sequential Implementation of Mix function:

```
for((p=0; p<(N/P − 1); p++))
{
    for(i=0; n<(P − 1); n++)
    {
        y[p,n] = a[p,n] + b[p,n]
    }
}
```

2.5.4 Gain and Mix

In this algorithm, gain is first applied to two input signals and then they are mixed together. Consider two input signals $\boldsymbol{A}$ and $\boldsymbol{B}$ of length $\boldsymbol{N}$ samples. Let $\boldsymbol{k_a}$ and $\boldsymbol{k_b}$ be the gain constants applied to the signals $\boldsymbol{A}$ and $\boldsymbol{B}$ respectively. Hence every $\boldsymbol{n^{th}}$ sample in the signal $\boldsymbol{Y}$ sample is computed in the following manner:

$$y_n = (k_a * a_n) + (k_b * b_n) \tag{2.8}$$

where $0 \leq n < N$. This can be vectorized in a similar manner as the previous algorithms. Moreover, it has more instruction level parallelism as the multiplication unit and the multiply accumulator unit can work in parallel to yield in better performance and fewer pipeline stalls. Considering a SIMD machine which can work on vectors of vector length $\boldsymbol{P}$, the algorithm can be parallelized in the following manner:

$$\begin{aligned}[y_n, y_{(n+1)}, ..., y_{(n+P-1)}] = {} & (k_a \odot [a_n, a_{(n+1)}, ..., a_{(n+P-1)}]) \\ & \oplus (k_b \odot [b_n, b_{(n+1)}, ..., b_{(n+P-1)}])\end{aligned} \tag{2.9}$$

A suitable pseudo code to represent the operation would be as follows. The operations in the inner loop can further be optimized because of the inherent instruction level parallelism.

Algorithm 3: Sequential Implementation of Gain and Mix function:

```
for((p=0; p<(N/P − 1); p++))
{
    for(n=0; n<(P − 1); n++)
    {
        y[p,n] = k_a*a[p,n] + k_b*b[p,n]
    }
}
```

2.5.5 Complex Number Multiplication

This DSP functionality is relatively more tasking to the hardware compared to the previous ones discussed above and is used quite frequently in signal processing applications, especially as a subtask when applying more complex algorithms to the input data. A common example is Fourier analysis on audio data. Consider two input buffers, each of size $\mathbf{2N}$. These buffers are loaded with $\boldsymbol{N}$ complex numbers of signals $\boldsymbol{X}$ and $\boldsymbol{Y}$. The complex numbers of $\boldsymbol{X}$ and $\boldsymbol{Y}$ are represented as $\Re(x) + i\Im(x)$ and $\Re(y) + i\Im(y)$ respectively. To multiply the elements of the two buffers and obtain the output complex number signal $\boldsymbol{Z}$; $\Re(z) + i\Im(z)$ of size $\boldsymbol{N}$, the following formula is used:

$$\begin{aligned}
\Re(z_0) &= \Re(x_0) * \Re(y_0) - \Im(x_0) * \Im(y_0) \\
\Im(z_0) &= \Re(x_0) * \Im(y_0) + \Im(x_0) * \Re(y_0) \\
\\
\Re(z_1) &= \Re(x_1) * \Re(y_1) - \Im(x_1) * \Im(y_1) \\
\Im(z_1) &= \Re(x_1) * \Im(y_1) + \Im(x_1) * \Re(y_1) \\
&\vdots
\end{aligned} \tag{2.10}$$

where $\mathbf{0} \leq n < \boldsymbol{N}$. It can be noted from Eq. 2.10 that it is quite taxing on the hardware when working with large buffer sizes. Belloch et al. (2015)[31] mentions that this process takes up 90% or more of the execution time in their implementation of binaural virtualization of sound. They use SIMD intrinsics to improve its performance, which is used as a background study for this task. Due to complexity in which the data is stored as pairs of real and imaginary parts in succession as shown in Fig. 2.19, vectorization is limited unless the data can be moved around. The pseudo code for the Eq. 2.10 is shown below. When implemented without vectorization intrinsics, each complex number in the output requires 4 floating point multiplications and 2 floating point additions.

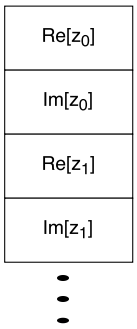

Figure 2.19: Typical arrangement of complex numbers in memory as was also in the case of Belloch et al. (2015)[31].

$$
\begin{aligned}
z_0 &= x_0 * y_0 - x_1 * y_1 && = \Re(Z_0) \\
z_1 &= x_0 * y_1 + x_1 * y_0 && = \Im(Z_0) \\
z_2 &= x_2 * y_2 - x_3 * y_3 && = \Re(Z_1) \\
z_3 &= x_2 * y_3 + x_3 * y_2 && = \Im(Z_1) \\
z_4 &= x_4 * y_4 - x_5 * y_5 && = \Re(Z_2) \\
z_5 &= x_4 * y_5 + x_5 * y_4 && = \Im(Z_2) \\
z_6 &= x_6 * y_6 - x_7 * y_7 && = \Re(Z_3) \\
z_7 &= x_6 * y_7 + x_7 * y_6 && = \Im(Z_3)
\end{aligned}
$$

Belloch et al. (2015)[31] use architecturally dependent techniques to vectorize. In their case buffers $\boldsymbol{X}$ are constants (filter coefficients) and so they are not modified in the calulation. So the real and imaginary parts are separated and is stored in the memory in the same manner as required for the calculation as shown in Fig. 2.20. Then it is loaded into the SIMD vector registers `REAL_X` and `IMAG_X` for calculation. This is a specific case and cannot be applied to general complex number multiplication. Buffer $\boldsymbol{Y}$ is loaded sequentially into another SIMD register `A`. Then the contents of register `A` is transformed to register `B` using the ARM SIMD instruction `vgetq_lane_f32` and `vsetq_lane_f32`. The former is used to get a value from a lane in a vector and the latter is used to insert it into the destination vector. This implies that for every pair of complex number results in the output the implementation described above requires

1. Four data transfer operations involving `vsetq_lane_f32`.

2. Four data transfer operations involving `vgetq_lane_f32`. This results in vector B.

3. One SIMD multiply instruction : $\boldsymbol{Z} = \texttt{REAL_X} \odot \boldsymbol{A}$

4. One SIMD multiply-accumulate instruction : $\boldsymbol{Z} = \boldsymbol{Z} \oplus (\texttt{IMAG_X} \odot \boldsymbol{B})$

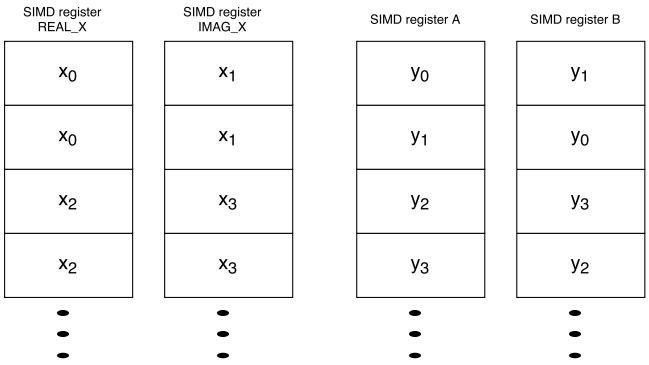

Figure 2.20: Arrangement of buffer X in the case of Belloch et al. (2015)[31].

5. So in total it requires 10 instructions for 2 pair of complex numbers compared to 12 instructions in the non vectorized implementation.

While the difference in the number of instructions may not differ a lot, data transfer between lanes are relatively faster compared to that of multiplication and multiply-accumulate operations. Moreover, the difference in execution time scales up as the buffer size increases. However, significant execution time is spent by arranging the data in the vectors with the `vsetq_lane_f32` and `vgetq_lane_f32` instruction. This can definitely be improved using different data transfer instructions and arrangements. A better solution using lesser instructions is described in the chapter 5.

2.5.6 Cascaded Infinite Impulse Response Filter

The Impulse infinite response filter(IIR) is a recursive digital filter used commonly in DSP applications. The transfer function of a filter $\boldsymbol{H}$ in time domain, as shown in Eq. 2.11, is the relationship between the input and output signals[32]. In the continuous time domain, the Laplace transform is used to denote the transfer function as shown in Eq. 2.12. For discrete signals, the transfer function can be denoted by using the $\boldsymbol{z}$ transform as shown in Eq. 2.13.

$$\boldsymbol{H} = \left[\frac{\boldsymbol{OutputSignal}}{\boldsymbol{InputSignal}}\right] \tag{2.11}$$

$$\boldsymbol{H(s)} = \frac{\boldsymbol{Y(s)}}{\boldsymbol{X(s)}} \tag{2.12}$$

$$\boldsymbol{H(z)} = \frac{\boldsymbol{Y(z)}}{\boldsymbol{X(z)}} \tag{2.13}$$

A linear filter whose response is defined by weighted constants $\boldsymbol{a_0, a_1, ...a_n}$ and $\boldsymbol{b_0, b_1, ...b_n}$[32] is shown in the difference equation Eq. 2.14. A difference equation is that which calculates the current output sample with past input and output samples and current input sample[33]. These weighted constants are referred to as

the filter's coefficients or filter taps[34]. So the transfer function of an IIR filter can be represented as shown in Eqs. 2.15 and 2.16, and is known as the Direct Form I realization. It consists of feedforward and feedback parts[32], which is an important characteristic of IIR filters since FIR filters do not have a feedback part. In other words, the current output sample depends on the previous output samples. N is the number of filter coefficients on the input signal and M is the number of coefficients on the output.

$$\begin{aligned} Y(z) &= b_0X(z) + b_1z^{-1}X(z) + ...b_Mz^{-M}X(z) \\ &= a_1z^{-1}Y(z) + a_2z^{-2}Y(z)...a_Nz^{-N}Y(z) \end{aligned} \tag{2.14}$$

$$\frac{Y(z)}{X(z)} = k * \frac{b_0 + b_1z^{-1} + ... + b_Mz^{-M}}{1 + a_1z^{-1} + ... + a_nz^{-N}} \tag{2.15}$$

$$= \frac{\sum_{i=0}^{N-1} b_iz^{-i}}{1 - \sum_{i=1}^{M-1} a_iz^{-i}} \tag{2.16}$$

$$y_n = \sum_{i=0}^{N-1} b_ix_{n-i} + \sum_{i=1}^{M-1} a_iy_{n-i} \tag{2.17}$$

According to the delay or shift theorem in z transform, z^{-n} represents a delay of n samples[32]. For example, the inverse z transform of $z^{-n}X(z)$ represents the sample $x_{(n-1)}$. Hence the nth output sample of the IIR filter can be represented as shown in Eq. 2.17[33].

The **zeroes** of the filter are obtained by finding the roots of the numerator polynomial in Eq. 2.15. Similarly, the **poles** of the filter are obtained by finding the roots of the denominator in the same equation[32]. The order of the filter is the order of the numerator and denominator polynomials. Usually, due to the recursive nature of IIR filters, higher order filters can sometimes be unstable[34] and is more prone to quantization noise[32]. Hence they are usually split into second order sections in series resulting in what is known as a cascaded realization[33].

Second order filters with **two poles** and **two zeroes** are referred to as a biquad[33] because of they form a quadratic equation in the numerator and denominator. A biquad IIR filter is defined as shown in 2.19:

$$H(z) = \frac{b_0 + b_1z^{-1} + b_2z^{-2}}{1 + a_1z^{-1} + a_2z^{-2}} \tag{2.18}$$

$$y_n = b_0x_n + b_1x_{(n-1)} + b_2x_{(n-2)} + a_1y_{(n-1)} + a_2y_{(n-2)} \tag{2.19}$$

where b_0, b_1, b_2 and a_1, a_2 are the FIR and IIR coefficients respectively. Biquads connected in series are referred to as a **cascaded realization**[32] and is shown in Fig. 2.21. The net response is obtained by multiplying the individual responses of

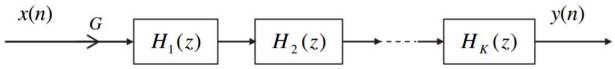

Figure 2.21: A cascaded realization of a higher order filters using biquads as illustrated in [35]

the filters as shown in Eq. 2.20.

$$\boldsymbol{H_{cascade}(z) = \prod_{k=1}^{K} H_k(z)} \tag{2.20}$$

For the purpose of this thesis, a cascade realization whose application is a four band parametric equalizer was to be vectorized. The recursive nature of a biquad creates dependencies due to which vectorization is more complex. R. Kutil et al. (2008)[36] approaches this problem by solving the dependencies using fused filter taps, which is similar to the approach attempted by Intel[37].
For the purpose of describing the approach used by R. Kutil et al. (2008), consider an IIR filter where the number of taps are $\boldsymbol{N = M = 4}$, a SIMD machine of vector size $\boldsymbol{p = 4}$. Any sample in the output $\boldsymbol{y_n}$ is calculated as shown in Eq. 2.17 and the goal is to calculate an output vector $[\boldsymbol{y_n, y_{(n+1)}, y_{(n+2)}, y_{(n+3)}}]$.

$$\begin{aligned}
\boldsymbol{y_n} &= \boldsymbol{b_0 x_n + b_1 x_{(n-1)} + b_2 x_{(n-2)} + b_3 x_{(n-3)}} \\
&\quad \boldsymbol{+ a_1 y_{(n-1)} + a_2 y_{(n-2)} + a_3 y_{(n-3)}} \\
\boldsymbol{y_{(n+1)}} &= \boldsymbol{b_0 x_{(n+1)} + b_1 x_n + b_2 x_{(n-1)} + b_3 x_{(n-2)}} \\
&\quad \boldsymbol{+ a_1 y_n + a_2 y_{(n-1)} + a_3 y_{(n-2)}} \\
\boldsymbol{y_{n+2}} &= \boldsymbol{b_0 x_{(n+2)} + b_1 x_{(n+1)} + b_2 x_n + b_3 x_{(n-1)}} \\
&\quad \boldsymbol{+ a_1 y_{(n+1)} + a_2 y_n + a_3 y_{(n-1)}} \\
\boldsymbol{y_{(n+3)}} &= \boldsymbol{b_0 x_{(n+3)} + b_1 x_{(n+2)} + b_2 x_{(n+1)} + b_3 x_n} \\
&\quad \boldsymbol{+ a_1 y_{(n+2)} + a_2 y_{(n+1)} + a_3 y_n}
\end{aligned} \tag{2.21}$$

The FIR part can be vectorized in a straightforward way since there are no dependencies as shown in Eq. 2.22.

$$\begin{aligned}
\boldsymbol{[u_0, u_1, u_2, u_3]} &= \boldsymbol{[x_n, x_{(n+1)}, x_{(n+2)}, x_{(n+3)}] \odot b_0} \\
\boldsymbol{[u_0, u_1, u_2, u_3]} &= \boldsymbol{[u_0, u_1, u_2, u_3] \oplus \Big([x_{(n-1)}, x_n, x_{(n+1)}, x_{(n+2)}] \odot b_1 \Big)} \\
\boldsymbol{[u_0, u_1, u_2, u_3]} &= \boldsymbol{[u_0, u_1, u_2, u_3] \oplus \Big([x_{(n-2)}, x_{(n-1)}, x_n, x_{(n+1)}] \odot b_2 \Big)} \\
\boldsymbol{[u_0, u_1, u_2, u_3]} &= \boldsymbol{[u_0, u_1, u_2, u_3] \oplus \Big([x_{(n-3)}, x_{(n-2)}, x_{n-1}, x_{(n)}] \odot b_3 \Big)}
\end{aligned} \tag{2.22}$$

To obtain the FIR filter response $[u_0, u_1, u_2, u_3]$ for input vector $[x_n, x_{(n+1)}, x_{(n+2)}, x_{(n+3)}]$, it requires one SIMD multiplication and three SIMD multiply accumulate operations. Now Eq. 2.21 can be represented as

$$\begin{aligned} y_n &= u_0 + a_1 y_{(n-1)} + a_2 y_{(n-2)} + a_3 y_{(n-3)} \\ y_{(n+1)} &= u_1 + a_1 y_n + a_2 y_{(n-1)} + a_3 y_{(n-2)} \\ y_{(n+2)} &= u_2 + a_1 y_{(n+1)} + a_2 y_n + a_3 y_{(n-1)} \\ y_{(n+3)} &= u_3 + a_1 y_{(n+2)} + a_2 y_{(n+1)} + a_3 y_n \end{aligned} \tag{2.23}$$

In Eq. 2.23, the already available terms are $y_{(n-1)}, y_{(n-2)}$ and $y_{(n-3)}$. Also y_n can be obtained because it can be calculated from the available terms. Hence Eq. 2.23 can be rewritten in terms of y_n for $y_{(n+2)}$ and $y_{(n+3)}$. For $y_{(n+2)}$, the dependency $y_{(n+1)}$ can be expressed in terms of y_n as follows.

$$\begin{aligned} a_1 y_{(n+1)} &= a_1(u_1 + a_1 y_n + a_2 y_{(n-1)} + a_3 y_{(n-2)}) \\ &= a_1(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) + {a_1}^2 y_n \end{aligned} \tag{2.24}$$

Using 2.24, $y_{(n+2)}$ can be expressed by

$$\begin{aligned} y_{(n+2)} &= u_2 + a_1 y_{(n+1)} + a_2 y_n + a_3 y_{(n-1)} \\ &= u_2 + a_1(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) + ({a_1}^2 + a_2) y_n + a_3 y_{(n-1)} \end{aligned} \tag{2.25}$$

Similarly the procedure can be done for $y_{(n+3)}$.

$$\begin{aligned} a_1 y_{(n+2)} &= a_1(u_2 + a_1 y_{(n+1)} + a_2 y_n + a_3 y_{(n-1)}) \\ &= a_1(u_2 + a_3 y_{(n-1)}) + {a_1}^2 y_{(n+1)} + a_1 a_2 y_n \end{aligned} \tag{2.26}$$

$$\begin{aligned} {a_1}^2 y_{(n+1)} &= {a_1}^2(u_1 + a_1 y_n + a_2 y_{(n-1)} + a_3 y_{(n-2)}) \\ &= {a_1}^2(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) + {a_1}^3 y_n \end{aligned} \tag{2.27}$$

Substituting 2.27 in 2.26:

$$\begin{aligned} a_1 y_{(n+2)} &= a_1(u_2 + a_3 y_{(n-1)}) + {a_1}^2(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) \\ &\quad + ({a_1}^3 + a_1 a_2) y_n \end{aligned} \tag{2.28}$$

Solving for $a_2 y_{(n+1)}$

$$\begin{aligned} a_2 y_{(n+1)} &= a_2(u_1 + a_1 y_n + a_2 y_{(n-1)} + a_3 y_{(n-2)}) \\ &= a_2(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) + a_1 a_2 y_n \end{aligned} \tag{2.29}$$

So $y_{(n+3)}$ can be expressed as:

$$\begin{aligned} y_{(n+3)} &= u_3 + a_1(u_2 + a_3 y_{(n-1)}) + ({a_1}^2 + a_2)(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)}) \\ &\quad + ({a_1}^3 + 2a_1 a_2 + a_3) y_n \end{aligned} \tag{2.30}$$

The terms $(u_1 + a_2 y_{(n-1)} + a_3 y_{(n-2)})$ and $(u_2 + a_3 y_{(n-1)})$ in 2.25 and 2.29 is obtained while iterating to get $y_{(n)}$. In each iteration, the available terms can be added to $u_{(0,1,2,3)}$ until $y_{(n)}$ is calculated. **Thus $[y_n, y_{(n+1)}, y_{(n+2)}, y_{(n+3)}]$ can be vectorized with the following SIMD iterations**[36].

$$
\begin{aligned}
[u_0, u_1, u_2, u_3] &= [u_0, u_1, u_2, u_3] \\
&\oplus \Big([y_{(n-3)}, y_{(n-2)}, y_{(n-1)}, 0) \odot [a_3, a_3, a_3, 0] \Big) \\
[u_0, u_1, u_2, u_3] &= [u_0, u_1, u_2, u_3] \\
&\oplus \Big([y_{(n-2)}, y_{(n-1)}, 0, u_2] \odot [a_2, a_2, 0, a_1] \Big) \\
[u_0, u_1, u_2, u_3] &= [u_0, u_1, u_2, u_3] \\
&\oplus \Big([y_{(n-1)}, 0, u_1, u_1] \odot [a_1, 0, a_1, {a_1}^2 + a_2] \Big) \\
[u_0, u_1, u_2, u_3] &= [u_0, u_1, u_2, u_3] \\
&\oplus \Big([0, u_0, u_0, u_0] \odot [0, a_1, {a_1}^2 + a_2, {a_1}^3 + 2a_1 a_2 + a_3] \Big)
\end{aligned}
\tag{2.31}
$$

So in total this approach requires **seven SIMD operations** for each biquad. Note that this approach gives more vectorization potential as the filter order increases and is independent of the vector length P. In fact, there is no difference in performance for filter order $\{N, M\} \leq P - 1$[36]. Furthermore, in the case of cascaded biquads, the performance increase is not satisfying. Whilst the execution time per biquad is still reduced, there is still room for more parallelization. Another problem which arises during the implementation phase is that it requires many lane shift operations, especially in the IIR part. This results in low availability of instruction level parallelism. However introducing more input signals to be processed at once such as multiple audio channels can help increase parallelism and in turn, speedup.

2.5.7 Peak Program Meter

A peak program meter(PPM) is an analogue loudness measurement device, used primarily to measure loudness levels in audio[38]. They are designed and calibrated to give a loudness measurement similar to that of the human ear. It is often used as a visual tool to make sure audio levels are not too high that clipping occurs, or not too low that the signal to noise ratio is low[38]. They are also commonly used in radio broadcasting, where the signals have to meet certain standards. The PPM meters are characterized by attack time and release time. The attack time is how much time it takes for the meter to respond to an input peak and the latter is how much time it takes to settle down. PPM meters usually have fast response times and pick up small peaks[38]. The PPM meter under consideration are IEC 60268-10 Type I and IEC 60268-10 Type II and were digitally modelled and implemented by Fons Adriaensen[39] in the Jmeters source code. It was based on a peak envelope

detector.

A peak envelope detector is similar to a one pole recursive low pass filter whose coefficient changes depending on the amplitude of the signal[40]. Consider a peak envelope detector with attack coefficient $\boldsymbol{AC}$, release coefficient $\boldsymbol{RC}$ and input signal $\boldsymbol{X}$. If amplitude of input sample $|\boldsymbol{x_n}|$ is greater than the peak of the previous sample $\boldsymbol{z_{(n-1)}}$ then the new peak $|\boldsymbol{z_n}|$ is given by[40]:

$$\text{If} \boldsymbol{z_n} < |\boldsymbol{x_n}|$$
$$\boldsymbol{z_n = (1 - AC) * z_{(n-1)} + AC * |x_n|} \tag{2.32}$$

If amplitude of input sample $|\boldsymbol{x_n}|$ is lesser than or equal to the peak of the previous sample $\boldsymbol{z_{(n-1)}}$ then the new peak $|\boldsymbol{z_n}|$ is given by:

$$\text{If} \boldsymbol{z_n} \geq |\boldsymbol{x_n}|$$
$$\boldsymbol{z_n = (1 - RC) * z_{n-1}} \tag{2.33}$$

$$\boldsymbol{AC = 1 - e^{-2.2/(t_a.F_s)}} \tag{2.34}$$
$$\boldsymbol{RC = 1 - e^{-2.2/(t_r.F_s)}} \tag{2.35}$$

Equation 2.33 shows that when the input amplitude is lesser than the position of the needle in the peak meter, the needle settles down at the rate defined by the release coefficient. The attack and release coefficients are calculated as shown in Eq. 2.34 and 2.35[40], where $\boldsymbol{t_a}$ and $\boldsymbol{t_r}$ are the attack time and release times respectively (in seconds). The attack and release times for IEC 60268-10 Type I and Type II used in the implementation of this model are defined by their standards. The algorithm to be vectorized is derived from the work of Fons Adriaensen[39] who uses two attack coefficients $\boldsymbol{AC_1}$ and $\boldsymbol{AC_2}$ and one release coefficient $\boldsymbol{RC}$. Two peaks are calculated for both the attack coefficients.

$$\text{If} \boldsymbol{z_{(1,n)}} < |\boldsymbol{x_n}|$$
$$\boldsymbol{z_{(1,n)} = (1 - AC_1) * z_{(1,n-1)} + AC_1 * |x_n|} \tag{2.36}$$
$$\text{If} \boldsymbol{z_{(2,n)}} < |\boldsymbol{x_n}|$$
$$\boldsymbol{z_{(2,n)} = (1 - AC_2) * z_{(2,n-1)} + AC_2 * |x_n|} \tag{2.37}$$

$$\text{If} \boldsymbol{z_{(1,n)}} \geq |\boldsymbol{x_n}|$$
$$\boldsymbol{z_{(1,n)} = (1 - RC) * z_{(1,n-1)}} \tag{2.38}$$
$$\text{If} \boldsymbol{z_{(2,n)}} \geq |\boldsymbol{x_n}|$$
$$\boldsymbol{z_{(2,n)} = (1 - RC) * z_{(2,n-1)}} \tag{2.39}$$

For a buffer size of $\boldsymbol{N}$, the algorithm is as shown below. Note that the largest peak information $\boldsymbol{Z_{largest}}$ in the buffer is also stored which is necessity for other DSP

Algorithm 4: Peak Program Meter algorithm for buffer size $\boldsymbol{N}$:

```
for(n=0; n<N; n++)
{
    z1 = z1 * RC
    z2 = z2 * RC
    if(|xn| > z1)
    {
        z1 = z1 + AC1*(|xn|-z1)
    }
    if(|xn| > z2)
    {
        z2 = z2 + AC2*(|xn|-z2)
    }
    if(Zlargest < z1+z2)
    {
        Zlargest = z1+z2
    }
}
```

functions. Due to this unique implementation requirement, there is little literature which vectorizes this algorithm up to the authors knowledge. The requirement also specifies that this should be applied to $\boldsymbol{C}$ number of channels. Furthermore it has to be noted that, just like the IIR filter, the output $\boldsymbol{z_1}$ and $\boldsymbol{z_2}$ has dependencies and also relies on the outcome of the branching as well. Hence it is not possible to vectorize for $\boldsymbol{P}$ subsequent input samples where $\boldsymbol{P}$ is the vector size of the SIMD machine. Different approaches were explored as shown in the chapter 5.

Chapter 3

Development and Testing Methodology

In this chapter, the subsequent sections deals with:

- What methodology of research was used.
- How the project is developed.
- How it was tested.
- How it was benchmarked.

Accompanying them are the reasons for choosing the same. The goal is to have well documented and optimized functions which can be easily be integrated into other functions.

3.1 Methodology of Research

For this thesis, the goal is to improve performance of DSP functionality using Vectorization with SIMD instructions. It should answer how much the speed up will be, and what is the optimal way to vectorize. It needs experimental testing with comparisons with compiler auto-vectorized implementations, along with trial and error of different approaches to vectorization. This involves collecting large samples of data and analyzing them using statistical methods. Hence, **quantitative research methodology** is chosen to conduct research for this thesis.

3.2 Development Methodology

To achieve the goals and requirements of the thesis, a defined structure and procedure is needed to aide in efficient development and collection of results. For that, the requirements that need to be answered are:

1. Define a development cycle.
2. Define a project folder structure.

3. Decide on a build system

4. Decide on which programming language to use and how the final code be packaged.

5. Decide on a naming scheme for functions which is self explanatory of the functionality.

3.2.1 Programming Language and Packaging of Functions

Since this thesis deals with SIMD instructions, it is implied that the functions have to be written in assembly language comprised of NEON instructions. To interface with the assembly code, it is best to use C++. Since it is very close to C and has easy interfacing options such as **extern "C"**, both the sequential implementations and vectorized implementations can be written in the same code together. Also C++ allows intrinsics, which are basically lines of assembly code within the C++ code. After compilation, the C++ code is converted to assembly instructions and the vectorized implementations can be directly linked. A detailed description of the build system used is described in Appendix A.

The main deliverable of this project is vectorized code which can be used by other functions. For that reason it is decided to package the code as a **static library** along with a header file, which defines data types, structures, the function definition and comments explaining how to use them with examples. This static library can then be linked to any other code base and the functions can be used.

3.2.2 Development cycle

To facilitate quantitative research, a suitable cycle which includes validating the correctness of the implementation and running benchmarks is needed. The flowchart in Fig.3.1 explains the approach taken. To make it easier for Step 3 and Step 6, it was decided that two static libraries be created, one for the auto-vectorized code and one for the manually vectorized implementation. After step 7 the code was uploaded to the repository for version control. Also the benchmark code was packed into a library, so that a command line interface can use this library and run the desired benchmark.

3.2.3 Folder Structure

To keep things modular and easy to understand when implementing the procedural flowchart in Fig. 3.1, the following folder structure was adopted.

- A folder for source files of vectorized implementations as well as the header file

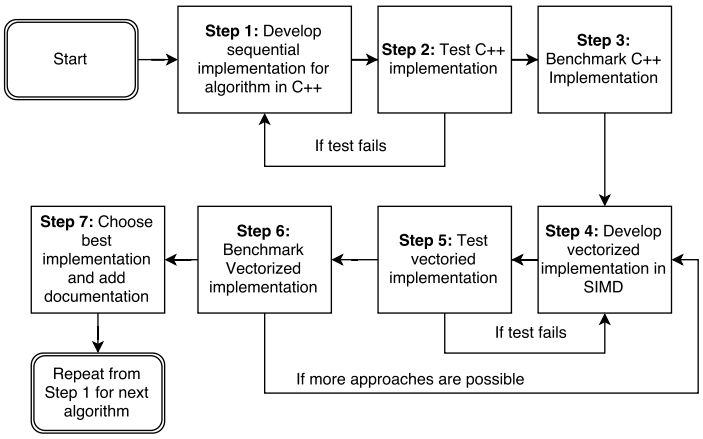

Figure 3.1: Flow chart outlining the approach towards development.

- A folder for source files of sequential implementations as well as the header file .
- A folder for source files which deals with testing and example programs.
- A common include folder where header files used by all source files are placed.
- A folder for source files dealing with benchmarking and reporting of benchmarks.
- A miscellaneous folder which contains other source files needed for development.
- A folder which contains the initialization code of the Performance management unit, more details are explained in the benchmarking section.
- Finally, a build folder where all the compiled binaries are placed.

3.2.4 List of Functions Developed

To implement the required algorithms, below is the list of the functions that were developed.

1. `gain_NEON`: Vectorized Gain algorithm implementation for a buffer of $\boldsymbol{N}$ samples in size.
2. `mix_NEON`: Vectorized Mix algorithm implementation for a buffer of $\boldsymbol{N}$ samples in size.
3. `mix_with_gain_NEON`: Vectorized Gain and Mix algorithm implementation for a buffer of $\boldsymbol{N}$ samples in size.

4. `filter4b_1CH_NEON`: Vectorized implementation of the Cascaded IIR Filter algorithm for a buffer of $\boldsymbol{N}$ samples in size.

5. `complex_multiply_NEON`: Vectorized Complex Multiplication algorithm implementation for a buffer of $\boldsymbol{N}$ samples in size.

6. `peak_meter_NEON`: Vectorized PPM algorithm implementation for a buffer of $\boldsymbol{N}$ samples in size.

3.2.5 General Code Structure

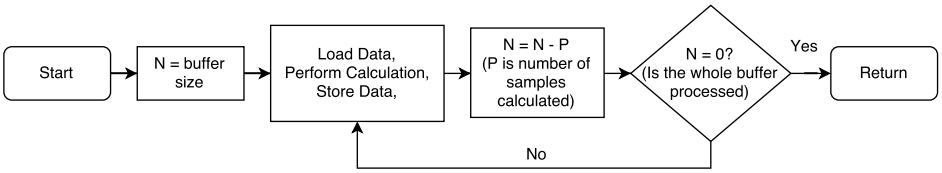

Figure 3.2: Flow chart representing the general structure or template of how each function is implemented.

Figure 3.2 shows the flowchart representing the general code structure of the implemented functions. These functions calculate over a dynamic buffer size $\boldsymbol{N}$, which is iterated over $\boldsymbol{P}$ times, the number of samples calculated per loop. This $\boldsymbol{P}$ can be either lesser or more than the vector size and is implementation specific. To summarize, Fig. 3.2 represents also the control flow graph, which is sequential code without any conditional branches other than iterating the whole buffer.

3.3 Testing Methodology

Testing is necessary to ensure the correctness of the implementation as well as to aide in debugging. The test program should check the correctness of every sample of the output for all possible inputs. Also, it should be easy to set up, produce repeatable test results and report errors if any. For this reason, it was decided to perform **unit testing** with the **Google Test framework**. It provides unit testing functionality for C++ such as assertions along with command line use. Google Test is available as source code, which is compiled locally on the ARM Cortex-A15 to give the respective libraries. A detailed description of the features of Google Test and how it was used to test the functions developed in this thesis is given in Appendix B.

Chapter 4

Timing measurement and Benchmarking Methodology

In this chapter the reasons motivating the chosen method of benchmarking and the methodology of implementation is explained. It describes an analyses of the requirements, feasibility study of possible approaches and evaluation of the chosen method's performance.

4.1 Calculation of WCET

For this thesis, it was decided to use **END to END measurement methodology** for the following reasons.

1. Static measurement techniques need accurate timing information of the underlying processor, which is difficult to obtain from the manufacturer. An Out of Order CPU like the ARM Cortex-A15 is complex to analyze in the processor behavior analyses phase. Also, pipeline and cache states have to be taken into account.

2. Static measurements techniques analyze the CFG and evaluate all possible branches. The general CFG of implementation of the code is as shown in the flowchart in Fig. 3.2, which is sequential in nature and has only one branch which iterates over the entire buffer. It is akin to straight line code and so CFA is not needed.

3. The goal of the thesis is to vectorize the algorithms and compare the speedup with that of the auto-vectorized implementations. These implementations are usually used by higher level functions in the hierarchy. End to End measurement makes more sense here because it compares observable WCETs of the function than estimates. Estimations can be applied to the functions in the upper hierarchy, which use these implementations.

4. Since the timing measurement system requires fine timing granularity, it needs to be accurate as possible. For example, such granular timing measurements can help in instruction scheduling and prevent pipeline stalls. This is explained in detail in the following subsections below.

4.2 Instruction Scheduling Methodology

To explain why critical timing measurement is needed to improve instruction level parallelism and to aide in instruction scheduling, consider the example code snippet shown below which vectorizes the FIR part of the biquad IIR filter. The mathematical operations of the code snippet is as shown in Eqs. 4.1 to 4.8. Five SIMD registers `coeff_B`,`vecXNA`,`vecXNA`,`vecXN1` and `vecXN2` are used. `coeff_B` holds the FIR coefficients (Eq. 4.1), `vecXNB` holds the previous samples (Eq. 4.2) and `vecXNA` holds the current samples (Eq. 4.3). Using `VEXT` instruction, `vecXN1` and `vecXN2` is obtained as shown in Eqs. 4.4 and 4.5.

$$\texttt{coeff_B} = (\boldsymbol{b_0}, \boldsymbol{b_1}, \boldsymbol{b_2}, \mathbf{0}) \tag{4.1}$$

$$\texttt{vecXNB} = \boldsymbol{x}_{(\boldsymbol{n-4}, \boldsymbol{n-3}, \boldsymbol{n-2}, \boldsymbol{n-1})} \tag{4.2}$$

$$\texttt{vecXNA} = \boldsymbol{x}_{(\boldsymbol{n}, \boldsymbol{n+1}, \boldsymbol{n+2}, \boldsymbol{n+3})} \tag{4.3}$$

$$\texttt{vecXN1} = \boldsymbol{x}_{(\boldsymbol{n-1}, \boldsymbol{n}, \boldsymbol{n+1}, \boldsymbol{n+2})} \tag{4.4}$$

$$\texttt{vecXN2} = \boldsymbol{x}_{(\boldsymbol{n-2}, \boldsymbol{n-1}, \boldsymbol{n}, \boldsymbol{n+1})} \tag{4.5}$$

$$\texttt{vfir} = \texttt{vfir} \odot \texttt{coeff_B[0]} \tag{4.6}$$

$$\texttt{vfir} = \texttt{vfir} \oplus \texttt{vecXN1} \odot \texttt{coeff_B[1]} \tag{4.7}$$

$$\texttt{vfir} = \texttt{vfir} \oplus \texttt{vecXN2} \odot \texttt{coeff_B[2]} \tag{4.8}$$

```
// Start Timer
VLD1.32      {vecXNA},[input_address:128]!
VEXT.F32     vecXN1,vecXNB,vecXNA,#3
VEXT.F32     vecXN2,vecXNB,vecXNA,#2
VMUL         vfir,vecXNA,coeff_B0
VMLA         vfir,vecXN1,coeff_B1
VMLA         vfir,vecXN2,coeff_B2
// Stop Timer
```

It can be seen from the code snippet that it causes pipeline stalls since there are data dependencies. Let the total time taken for the above code snippet be $\boldsymbol{t_{block}}$. If the `VMUL` instruction takes $\boldsymbol{t_{vmul}}$ time to execute, then the `VMLA` instruction can be executed only after $\boldsymbol{t_{vmul}}$ time. In order to improve instruction level parallelism, nondependent instructions whose total time is $\boldsymbol{t_{nondep}}$ should be inserted between the dependent instructions, where $\boldsymbol{t_{nondep}} \leq \boldsymbol{t_{vmul}}$. In this manner it is possible to execute more instructions in the same time $\boldsymbol{t_{vmul}}$, thereby increasing

the throughput of the code. When $t_{nondep} > t_{vmul}$, the extra time implies that the VMUL operation is over and the nondependent instructions add to the execution time. Hence accurate timing in terms of clocks cycles for each instruction or a block of instructions is needed here. This is too small of a case for static measurement techniques.

It can be inferred that scheduling is a NP hard problem. Using trial and error, it is possible to improve the scheduling and is easier to approach it block by block. First a timer can be started before VMUL and stopped immediately after to get t_{block}. Then a nondependent instruction such as VEXT can be added inside the timing measurement. If the resulting t_{block} is the same **this means that there is more room for nondependent instructions** as shown below in the pseudo code. So in this way, instructions can be added below the VMUL instruction until t_{block} increases.

```
// First Trial, Start Timer
VMUL        vfir,vecXNA,coeff_B0
// Stop Timer. Now we get t_block

// Second Trial, Start Timer
VMUL        vfir,vecXNA,coeff_B0
VEXT.F32    vecXN1,vecXNB,vecXNA,#3
// Stop Timer. Now we get t_block.

// If t_block remains unchanged then add more
   instructions.

// Third Trial, Start Timer
VMUL        vfir,vecXNA,coeff_B0
VEXT.F32    vecXN1,vecXNB,vecXNA,#3
VEXT.F32    vecXN2,vecXNB,vecXNA,#2
// Stop Timer. If t_block remains unchanged then add
   more instructions.
```

4.2.1 Guidelines for Scheduling SIMD Instructions With Timing Information

A more general rule of thumb to approach scheduling while development was conceptualized for this thesis and is given in the flowchart below. This can be used in any hand optimized SIMD implementations. Schedulability depends on the mathematical nature of the algorithm and can be improved with rearrangement of instructions, accurate timing information, use of temporary registers and **most importantly, the programmers intuition**. It is for this reason that hand optimized code usually outperforms auto-vectorized code.

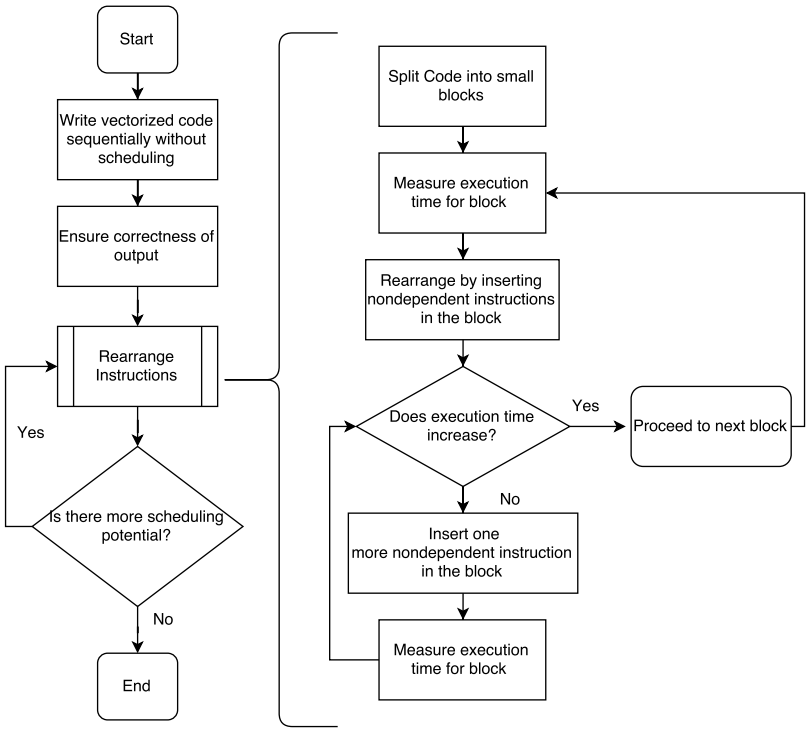

Figure 4.1: Flow chart representing a general procedure while implementing hand tuned SIMD code.

1. First, write the vectorized implementation in a straight forward way, for example, as the code snippet above. It can be a direct implementation of the algorithm and be sequential in an algorithmic sense.

2. Test the correctness of the output produced by the vectorized implementation. It is required to ensure that the code works before rearranging and serves as a reference point.

3. Now the rearrangement of instructions can be performed. Identify which instructions are dependent and nondependent. Also, consider that the hardware units can cause pipeline stalls and ideally they should be available when an instructions requires their use.

4. Approach by identifying small basic blocks of dependent code. A basic block can even have just one instruction and is simple straight line code. Measure execution time of this block.

5. Insert a nondependent instruction after the dependent one. Note that the instruction being placed should also be nondependent in the new sequence. Instructions which were dependent earlier can also be placed here, if it can be ensured that the dependencies are solved by the time the execution of the processor reaches this line.

6. Memory load and store operations and register transfer operations are perfect candidates for nondependent instructions. Use temporary registers to aide in calculation. The main instructions which cause pipeline stalls are computationally heavy operations like `VMUL, VMLA ,VADD` etc. Interleave dependent instructions of one block with dependent instructions of another, as long these two blocks are nondependent.

7. Measure the execution time again. If it remains the same, repeat step 5. If not, this means that the dependent instruction's operation is over and the result is available. The extra time is accounted by the nondependent instructions.

8. Check correctness of the output again to ensure that the algorithm is still the same and has not been altered. After this repeat step 4 for more blocks.

As an end note, the author acknowledges that it depends on intuition and is an iterative process, but suggests a optimal solution usually can be reached within three to four trials.The author also recommends that effort should preferably be applied to complex algorithms to get a considerable improvement in speedup.

4.3 Timing Measurement

To help implement the techniques mentioned above, high precision timing measurement is needed. It was in the interest of the company that available hardware or open source software be used, as opposed to using third party tracing mechanisms. The goal here is to find a timing measurement solution which can not only benchmark but also help in scheduling SIMD instructions. Four options are explored here.

4.3.1 Puslar.Webshaker Cycle Counter for Cortex A8

It is an online visual tool[41] in which ARM as well as NEON assembly instructions can be written and simulated. It gives the timing information in clock cycles and highlights pipeline status and stalls. This cycle counter cannot be used for benchmarking the entire program[41], but can serve as a useful tool for scheduling. However, there are considerable differences between ARM Cortex-A8 and the ARM Cortex-A15 cores. Many new SIMD instructions such as `VEXT` are not supported. It has numerous bugs as it was intended to be a hobbyist project and the author withdrew support in 2015. Though it is not reliable, it was still occasionally used to "quick schedule" certain basic blocks, to ensure that the dependencies are solved.

4.3.2 GEM5 Simulator

GEM5 is an open source, cycle accurate, event based CPU simulator which supports the simulation of ARM Cortex-A9 core with hardware level granularity. Some features are:

- It can simulate an OS image or a Linux/bare-metal binary.

- It is usually run with an accompanying python simulation script which holds all the necessary configuration information of the CPU such as CPU speed in ticks, memory topology etc. A tick in GEM5 represents 1ps and the clock of the processor is mentioned in terms of ticks. The python script links with the GEM5 executable to simulate.

- The simulation output a lot of valuable information, most importantly, the number of ticks the program took to finish.

- An important feature of GEM5 is the ability to simulate four different types of CPU, from abstract models to very detailed pipelined OoO CPU(**O3CPU**). In the configuration script, the timing information for each instruction, cache and memory topology, pipeline stages and depths, etc can be specified. In this way, even custom CPU's can also be simulated.

- It provides a visual pipeline viewer which analyzes the output of the O3CPU and creates a view of how the instructions pass through the pipeline.

Using the O3CPU with the flag `-debug-flags=O3PipeView` gives the pipeline status after which the pipeline viewer can be used. This pipeline information can be very helpful in scheduling because it will show a topological view of the pipeline and the total ticks can be used as an indicator for execution time However, only the ARM Cortex-A9 core was implemented, where the NEON unit is decoupled from the main pipeline. A. Butko et al. (2012)[42] conducted various benchmarks for the ARM A9 core in both a real CPU and GEM5. They report an error ranging between 1.39% to 17.94% compared to real world benchmarks. Such an error rate is not suitable for instruction scheduling.

GEM5 was also pursued during the implementation phase of the thesis. The main problem arises during its usage. Firstly, the program has to be compiled on the platform and then the binary should be transferred to the host development computer. Doing this in a repetitive manner, especially when trying to schedule instructions is a tedious task. GEM5 reports the execution time of the whole binary. It is not possible to analyze an individual function. So the output of GEM5 has manually analyzed to find the start and stop tick of the function under consideration. This involves sieving through more than thousand lines of assembly code which is impractical. The same case applies with the pipeline output viewer. A single instruction added can drastically change the layout and considerable amount of time is spent searching for the function. For these reasons it was decided not to use cycle accurate simulators and rely on hardware timing information.

4.3.3 C++ Chronos Library

Chronos is a timing measurement library provided by GCC and is a part of the C++ Standard Library. It is designed to aide in execution time calculation as well as date and time keeping[43][44]. It works by performing system calls using the POSIX Clocks and Timers API[45] and provides a standard interface to it from user space C++ programs. There are 3 types of clocks which Chronos can use. **Steady Clocks** are monotonic in nature and its value can be guaranteed to not change. **System Clock** represents the real time clock of the system and **High Precision Clock** has the smallest possible tick that can be provided by the system. The clock class also has a member function to determine whether it is steady or not. A detailed description of how the Chronos library was used to measure the execution time is given in Appendix C

High precision clock can be used to get as precise as possible timing measurements. The Chronos library appears to be a suitable tool for execution time measurement but the high precision timer can be used only if it is steady. This essentially means that the timing difference between ticks should be constant. It is important to note that the high precision timer is still experimental in embedded Linux[45] because it is implementation defined. Also, Chronos depends on system calls to the kernel to get the timing information. This implies that there are lot of factors at play such as whether the resource is locked and being used by another process, OS call overhead etc. So the results might not be completely accurate and can vary. An evaluation is shown in the later subsections.

4.3.4 Performance Management Unit

The PMU's cycle counter can be used as a timer to measure how many cycles the code takes to execute. It can be accessed through the ARM Coprocessor CP15 interface. To enable the cycle counter, the coprocessor register `PMCNTENSET` has to be enabled. The value of the counter can be accessed through `PMCCNTR` and the counter is set to count every clock cycle. However to access all these registers in user space, `PMUSERENR` has to be set to 1. Since `PMUSERENR` itself is not accessible from user space, a kernel module is written which executes the instructions to enable access and setup the counter. User space access can also be configured during kernel compile time. Description of the kernel module is given in Appendix D.1.

However, initial implementations of this method was inaccurate. When it was utilized for instruction scheduling, the cycle counter showed no difference whatsoever as more instructions were added to the basic block. In fact, sometimes the execution time of the whole SIMD code remained constant though more instructions were added. After more investigation, it was found that this was caused by the fact that the ARM core and the Advanced SIMD Unit execute in parallel and Out of Order.

The ARM Integer pipeline first executes the timer reset instructions. Then the SIMD instructions are issued which the ARM core does not execute and forwards it to the Advanced unit. It issues the SIMD instructions faster than the speed at which they can be executed. Once all the instructions are issued, the ARM core **executes the timer read instruction before the SIMD instructions have finished executing**. This causes discrepancy in the measurement. To overcome this, a mechanism is introduced which prevents the timer read instructions from executing in the ARM Integer pipeline until the SIMD instructions have finished executing.

Instruction Synchronization Barriers (`ISB`) halt execution of new instructions until the pipeline is flushed. But this only applies to the ARM core and can be used only in the sequential implementations. There are no barriers to ensure that the Advanced SIMD Unit's pipeline has been flushed. A workaround is to implement **Data Synchronization Barriers** (`DSB`). This will halt execution until all previously issued data load/store instructions have been executed. Therefore, the timer read instruction can be placed after an `ISB` and `DSB` instruction. This should be preceded by a store on the result of the last SIMD operation. The DSB ensures that the timer is reset only after the entire code executes.

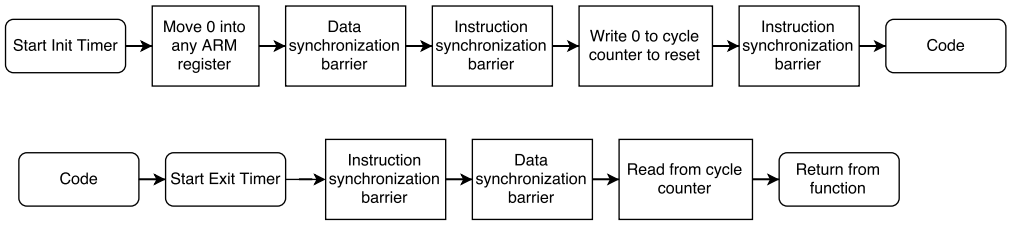

Figure 4.2: Flow chart representing the `timer_init` and `timer_exit` macros to initialize PMU cycle counter and read from it respectively.

The flowchart in Fig. 4.2 describes the `Init Timer` macro which resets the timer and `Exit Timer` macro which reads the timer value from the cycle counter register. These macros are placed as a part of the NEON and C code, to prevent overhead from function calls. The goal here is similar to emulation of an RTL model of the core without any instructions in the pipeline other than the SIMD instructions. The code for the macros are shown in Appendix D.2.

1. First, 0 is moved into any ARM register. This is then later moved into the cycle counter to reset it.

2. An `ISB` placed here to ensure that all previous instructions have been executed before the execution time of the code is measured.

3. Following this, a `DSB` is placed here to ensure that all previous load/store operations have been executed before the execution time of the code is measured.

4. Then the cycle counter is reset to 0.

5. Now the NEON code is executed. Note that the last NEON instruction in the code stores the result of the last NEON operation.

6. An `ISB` is placed here to prevent NEON instructions from executing until the cycle counter is reset. The ARM core issues multiple instructions in a cycle and so there is a possibility that the succeeding SIMD instructions will be issued in the pipeline before the cycle counter starts.

7. After the NEON code has been issued (but maybe not completely executed), the ARM core will execute the `Exit Timer` macro. An `ISB` is issued to flush any instructions in the ARM Integer pipeline.

8. Then a `DSB` is inserted which halts the execution of the ARM Integer pipeline until the last store operation in the NEON code is executed.

9. After the store operation is completed, the cycle counter value is read.

4.3.5 Development Platform Details

The platform to be used is the Odroid XU4 running a custom Linux kernel version : `Linux odroid 3.10.97-rt106+`. Its various specifications are as follows:

1. It is comprised of the Linux kernel with various patches such as the `RT PREEMPT` patch to ensure user space programs scheduled as a top priority non pre-emptable task. All benchmarks must be run as such.

2. The Linux kernel in the Odroid is configured to not use cores 4,5,6 and 7 in order to perform uninterrupted user space program execution. These are the four ARM Cortex-A15 cores in the Odroid. This is done by setting the kernel boot parameter `ISOLCPUS` to 4,5,6,7. However this does not guarantee the isolation to be 100% but is more akin to less affinity to the specified cores.

3. It was decided to run the benchmarks on core number 7. Frequency scaling was disabled by setting the scaling governor to "performance" mode. This ensured a constant 2GHz frequency on core 7.

4. The `GCC` version used was 4.9.2. To auto vectorize the sequential implementations, the flags "`-O3 -funroll-loops -ftree-vectorize -ffast-math` and "`-mfpu=neon`" was used.

5. The number of cycles taken for execution is used as a measurement of execution speed. This is because it serves a simple unit which directly relates to the performance of the architecture. It also helps while scheduling instructions.

4.4 Accuracy Evaluation of PMU cycle counter and Chronos

In this section, the accuracy of execution time measurement with the PMU cycle counter and the Chronos library is compared against each other. To use as a test program, the `gain` algorithm is implemented both sequentially and with SIMD vectorization. The benchmark program was run with the highest possible priority for 1 million iterations. While measuring with the Chronos Timing Library, the resulting execution time obtained, which is in seconds, is converted to number of cycles by multiplying it with the CPU frequency, i.e 2GHz.

From Fig. 4.3, it is observed that the execution time, when measured with the PMU cycle counter, varies extremely less when compared to that measured with the Chronos method, which is shown in Fig. 4.4. In fact, **all values in Fig. 4.3 are above a minimum value**, (86 clock cycles) which is the BCET. Though there are spikes, they are not that recurring compared to that as shown in Fig. 4.4. The Chronos method results in execution times much higher implying that there is a lot of overhead and exhibits a lot of variance. It does not have a stable minimum value, which is essential for instruction scheduling. **A further investigation revealed that the `high_precision_clock` was not steady**. The only clock which was steady in the Odroid XU4 was the `steady_clock`. It does not have high time granularity and so it cannot be used for execution time measurements.

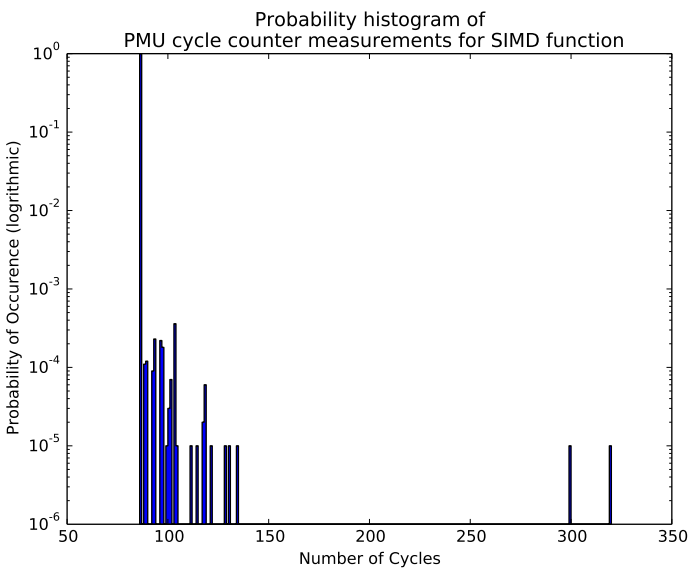

Figure 4.3: Probability histogram of execution times measured with PMU cycle timer for the SIMD vectorized gain algorithm. Execution times were measured over 1 million iterations.

It can be clearly seen that, in the case of the PMU cycle counter method, that the probability of outliers other than minimum is $\mathbf{10^{-3}}$ and almost 99% of the measurements taken fall at the minimum value. The fact that there is a high probability of the minimum is a quintessential factor which favors reproducibility of the optimized performance in real world scenarios. It also implies the mean, median and the minimum are very close to each other; around 86. In the case of the Chronos library on the other hand, the histogram shows the large variance from the mean as well as the large range of execution cycle. Such variance in a measurement technique is not reliable.

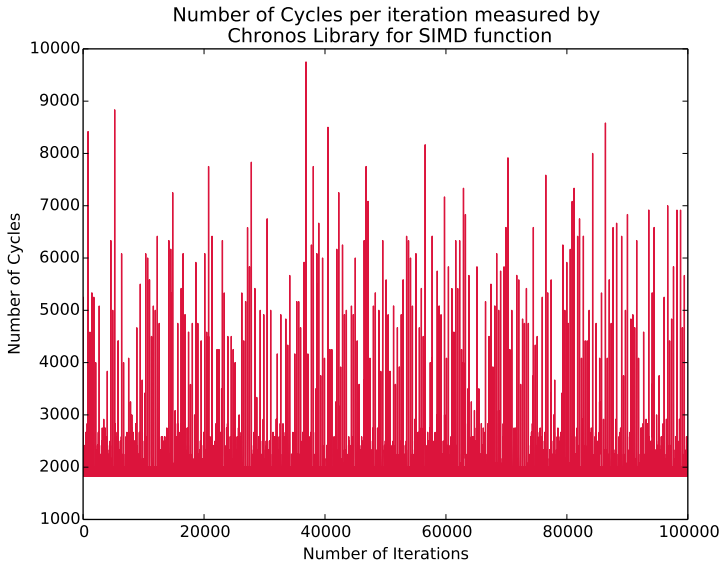

Figure 4.4: Probability histogram of execution times measured with Chronos library for the SIMD vectorized gain algorithm. Execution times were measured over 1 million iterations.

4.4.1 Measuring Cycles per Instruction

As a quintessential parameter of reference for scheduling SIMD instructions, the measurement method should have high timing precision and granularity. It should be stable enough to be capable of measuring the contributions of each instruction to the execution time. This is known as Cycles Per Instruction(CPI). With this information, it helps in being intuitive while hand optimizing SIMD code. To test whether the PMU cycle counter or the Chronos timer can support this, a block of the SIMD code which vectorizes the FIR part in a biquad filter for 4 samples, as shown in the previous chapter, is used.

```
VLD1.32      {vecXNA},[input_address:128]!      1
VEXT.F32     vecXN1,vecXNB,vecXNA,#3            2
VEXT.F32     vecXN2,vecXNB,vecXNA,#2            3
VMUL         vfir,vecXNA,coeff_B0               4
VMLA         vfir,vecXN1,coeff_B1               5
VMLA         vfir,vecXN2,coeff_B2               6
```

The execution time contributed by each SIMD instruction is measured by the process as shown below. The total execution time of a block is $\boldsymbol{T}$. The contribution of each instruction is denoted by $\boldsymbol{t_{(ins,i)}}$ where $\boldsymbol{ins}$ denotes the instruction name and $\boldsymbol{i}$ denotes the line number in the code. Every block is measured for 100 iterations.

1. First, the execution time of instruction 1 (`VLD1.32`) is measured.
   $\boldsymbol{T_{(1)} = t_{(VLD,1)}}$

2. Then, the second instruction (`VEXT`) is included in the measurement block and the execution time is measured again.

   $$\boldsymbol{T_{(1,2)} = t_{(VLD,1)} + t_{(VEXT,2)}}$$

   Now `VEXT` execution time is $\boldsymbol{T_{(1,2)} - T_{(1)}}$. The timer should definitely satisfy the fact that $\boldsymbol{T_{(1,2)} \geq T_{(1)}}$ and it should have stable values with less variance.

3. Repeat the process by including more instructions in the measurement block.

Figure 4.5 shows the execution time as measured by the PMU cycle timer as each instruction is included in the measurement block. It is extremely stable with no peaks and has constant measurement values for every iteration. This proves that the **PMU cycle timer methodology provides stable execution time results**. The measurements in Fig. 4.5 are described as follows:

- The contribution of the first instruction alone is $\boldsymbol{t_{(VLD,1)} = 36}$.
- The execution time of the first two instructions is $\boldsymbol{T_{(1,2)} = 37}$. This means that the second instruction was issued much earlier and was waiting for the result of instruction 1.

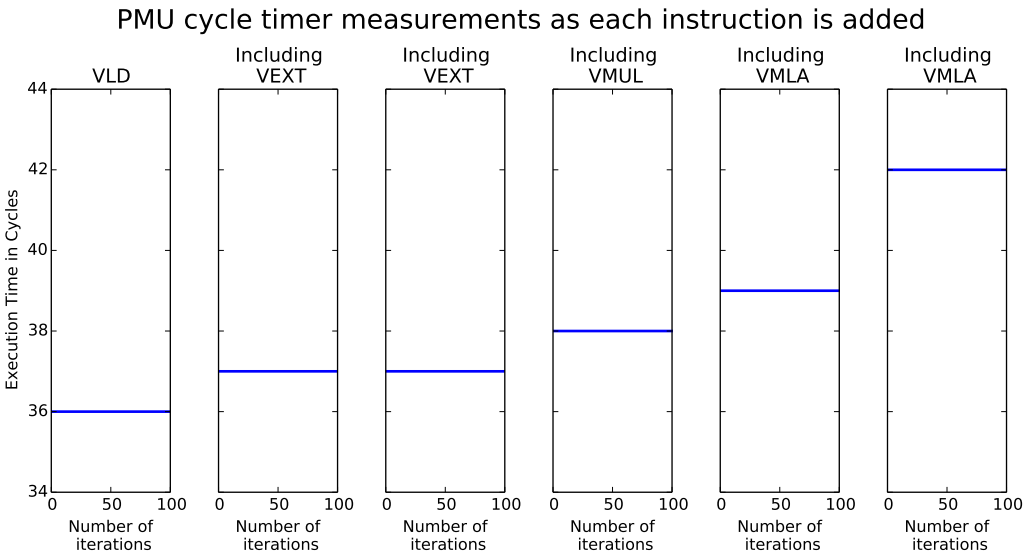

Figure 4.5: Execution time as each instruction is included the measurement block based on the PMU cycle timer.

- The execution time of the first three instructions is also $\boldsymbol{T_{(1,2,3)} = 37}$. This third instruction is also issued earlier like the second instruction, and they produce the result on the same clock cycle as they are nondependent to each other.

- The execution time of the first four instructions is $\boldsymbol{T_{(1,2,3,4)} = 38}$. The fourth instruction also waits for the result of the first one.

- Then $\boldsymbol{T_{(1,2,3,4,5)} = 39}$ after the first `VMLA` instruction is included. This depends on the result of the the `VMUL` instruction in line 4. The unavailability of the source register delayed the execution time by one clock cycle.

- Finally the last `VMLA` instruction is included and the total execution time for all instructions $\boldsymbol{T_{(1,2,3,4,5,6)} = 42}$. The last instruction is stalled until all the dependencies are resolved.

Figure 4.6 shows the measurements taken with the Chronos timer. It stays around the 2200 cycles mark no matter how many instructions are included in the measurement block. A reason could be that it takes more time for the operating system calls than the code itself. It is not precise and fast enough.

To summarize this section, **it was decided that the PMU timer was to be used to measure the execution times**. This was because of its fine time granularity and reliability of the measurements.

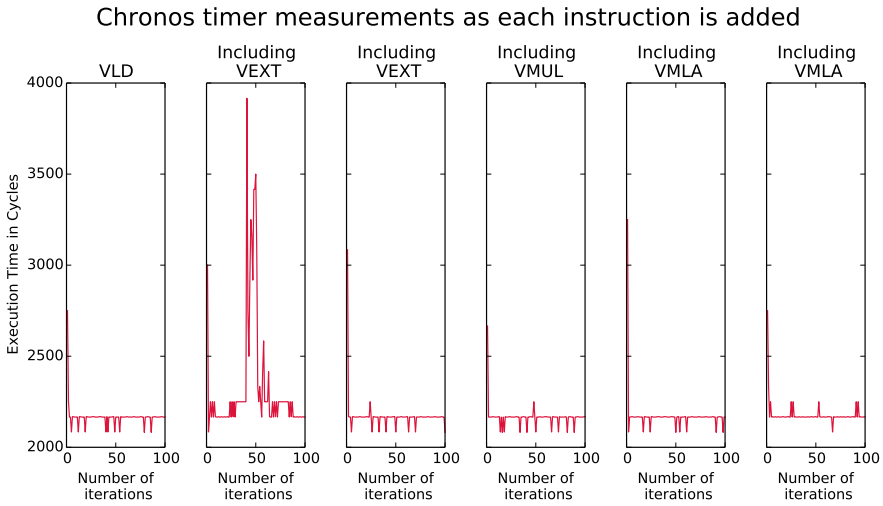

Figure 4.6: Execution time as each instruction is included the measurement block based on the Chronos cycle timer.

4.4.2 Cost of using PMU Cycle Timer with Barriers

To determine how much latency is introduced by the barriers and the store operation, an empty code section is measured with the PMU cycle counter following the same procedure as shown in Fig. 4.2 for 100,000 times. The corresponding code snippet is shown in Appendix D.2.

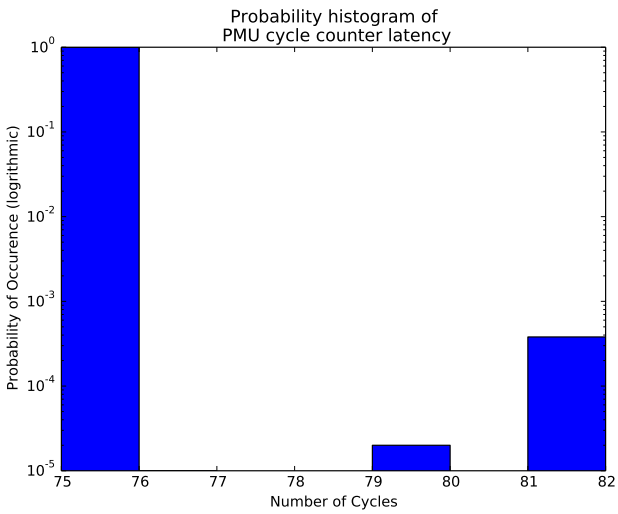

Figure 4.7: Probability histogram of PMU cycle counter latency.

The normalized probability histogram is shown in Fig. 4.7. It can be seen that

the latency introduced by the barriers and the store operation is mostly 75 clock cycles. It reaches 82 clock cycles in some iterations but that occurs due to cache issues or OS context switching. The probability of the latency being more than 75 clock cycles is less than $\mathbf{10^{-3}}$ and hence it is safe to assume that it is 75 clock cycles. The execution time measured can be compensated by subtracting 75 clock cycles from the cycle counter readings.

4.5 Performance Metrics

With the information obtained from the previous section, it can be concluded that the PMU timer offers good accuracy and stability when it comes to execution time measurements. This section describes what parameters are used to describe how well the vectorized SIMD implementations performs compared to that of the auto-vectorized C implementations.

4.5.1 Speed Up

The Speed Up is a ratio of the execution times of both the implementations, and denotes how well one performs with respect to the other as shown in Eq. 4.9[46].

$$\boldsymbol{Speedup = \frac{T_C}{T_{SIMD}}} \tag{4.9}$$

when $\boldsymbol{T_C \geq T_{SIMD}}$ and $\boldsymbol{T_{SIMD} > 0}$. Equation 4.9 assumes that the SIMD implementations will always be faster than the C. The above formula works well for single threaded programs without branches or conditions[46]. However execution time measurements usually exhibit variance and follow a distribution of values, even when tested for the same input values. This is because there are many factors at play such as OS Scheduling Policy, Instruction and Data Cache States, pipeline stalls etc. Hence the measurements has to be repeated for a large sample size.

A suitable statistical representation is needed that accurately answers:

- How well the SIMD vectorized implementation runs compared to the compiler's auto-vectorized code?
- What is the probability of seeing the improved performance in any random run by the user? In other words, the speedup should be reproducible in any random run.

The Median is a good representative and is commonly used by benchmarks such as SPEC[47]. It is not sensitive to outliers and makes more sense in the case of this thesis. However, S-A-A Touati et. al (2013)[47] explains that when performing benchmarks with the same data, it is important to justify using statistical tests.

Consider two benchmarking runs of iterations for both the implementations, $\boldsymbol{B_{SIMD}}$ and $\boldsymbol{B_C}$. $\boldsymbol{b_{SIMD,i}}$ and $\boldsymbol{b_{C,i}}$ represent the execution time obtained from an individual run $\boldsymbol{i}$. These runs do not correspond to each other and are unpaired. They only work on the same set of inputs. S-A-A Touati et. al (2013)[47] defines an $\boldsymbol{\alpha}$, which is a measure of the probability that there is an error and $\boldsymbol{B_{SIMD}}$ has an execution time equal to $\boldsymbol{B_C}$. Ideally, this should be 0 but in real world scenarios, it is usually set to 0.05[47] as a requirement.

To test with probability of $\mathbf{(1-\alpha)}$ that the $\texttt{median}(\boldsymbol{B_{SIMD}}) < \texttt{median}(\boldsymbol{B_C})$, S-A-A Touati et al. (2013) suggests that the **Wilcoxon-Mann-Whitney** test be used. It finds the probability $\boldsymbol{P_w}$ of a random execution time sample picked from $\boldsymbol{B_{SIMD}}$ is equal to or greater than any sample in $\boldsymbol{B_C}$[47]. In other words, it tests the hypothesis that the population of $\boldsymbol{B_{SIMD}}$ and $\boldsymbol{B_C}$ are equal and $\boldsymbol{P_w}$ is the probability of such an occurrence. If $\boldsymbol{P_w} \leq \boldsymbol{\alpha}$ then it meets the requirement that there is $\mathbf{1-\alpha}$ probability that any random sample of $\boldsymbol{B_{SIMD}}$ is faster than $\boldsymbol{B_C}$.

To summarize, the speedup of the program can be calculated using the median execution times, provided the probability $\boldsymbol{P_w}$ is sufficiently low. This is a more reasonable way to describe the performance gains. To increase the accuracy of the data, it was decided to measure execution times over large iterations such as 10 Million.

Chapter 5

SIMD Vectorization of DSP Functions

In this chapter, the methodology of algorithmic and architectural optimizations applied to each of the algorithm is explained. Accompanying it are the results of the optimizations and comparisons with the performance of other approaches. This chapter is divided into sections for each DSP function, with sub sections for describing the optimization methodology. All instructions are described by their mathematical equivalents through equations.

5.1 Input and Output Audio Buffers

Simple input and output audio buffers are used for testing and validating the algorithm. To get optimal cache locality, They are stored in a contiguous manner in the memory, and aligned to 32 bytes. This results in lesser clock cycles for memory transfer operations and prevents cache thrashing, which is the situation when the input and output buffers are mapped to the same cache line. The application which uses these functions work on buffer sizes of 64 and so the results for that buffer size are shown.

5.2 Gain

The gain function is quite simple to vectorize. The performance can be only improved by trying to schedule it in a different way than sequential. Its too simple to optimize in an algorithmic level and only architectural optimizations can be applied. Described below is the optimization of applying gain to a signal $\boldsymbol{X}$ of size $\boldsymbol{N} = \mathbf{64}$. The performance is then compared to that of the NE10 Library's implementation and the auto-vectorized code.

5.2.1 Architectural Optimization and Implementation

The only instruction level parallelism which can be exploited is to parallelize the load/store operations along the vectorized multiplication. When the Advanced

SIMD unit is working on one set of 16 samples, the next set of 16 samples can be loaded into the SIMD registers. This requires the signal size to be a minimum of 16 or multiples of 16. The algorithm can be represented with four parts, `INIT`, `LOOP1`, `LOOP2` and finally `ENDLOOP`. Four sets of quadword input registers for the input channel and four sets for the output registers are used as shown below.

Signal Denotations

- The input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{X}$ and its samples are denoted as $\boldsymbol{x_n}$, where $\boldsymbol{0 \leq n < N}$.
- The output signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{Y}$ and its samples are denoted as $\boldsymbol{y_n}$, where $\boldsymbol{0 \leq n < N}$.

SIMD Register Denotations

- $\boldsymbol{I_1}$, $\boldsymbol{I_2}$, $\boldsymbol{I_3}$, $\boldsymbol{I_4}$: Quadword registers to contain input signal samples $\boldsymbol{x_n}$.
- $\boldsymbol{O_1}$, $\boldsymbol{O_2}$, $\boldsymbol{O_3}$, $\boldsymbol{O_4}$: Quadword registers to contain output signal samples $\boldsymbol{y_n}$.
- $\boldsymbol{G}$: Upper half of a doubleword register which contains the scalar gain constant.

INIT

This section deals with the procedure when the function is first called. The initial operations before the main loop is shown in the equations below in Eqs. 5.1 and 5.2. Eight samples from the input are loaded into $\boldsymbol{I_1}$ and $\boldsymbol{I_2}$.

$$\boldsymbol{I_1 = [x_0, x_1, x_2, x_3]} \tag{5.1}$$
$$\boldsymbol{I_2 = [x_4, x_5, x_6, x_7]} \tag{5.2}$$

LOOP1

The approach taken here is to parallelize the `VLD` and `VMUL` operation on the input registers. The first part of the loop begins by loading input samples $\boldsymbol{(x_0, x_1, ..., x_{15})}$ for the next calculation as shown in Eqs. 5.3 and 5.4. This load operation can run in parallel whilst the calculation can be performed on samples $\boldsymbol{(x_n, x_{(n+1)}, ..., x_{(n+7)})}$ as shown in Eqs. 5.5 and 5.6.

$$\boldsymbol{I_3 = [x_{(n+8)}, x_{(n+9)}, x_{(n+10)}, x_{(n+11)}]} \tag{5.3}$$
$$\boldsymbol{I_4 = [x_{(n+12)}, x_{(n+13)}, x_{(n+14)}, x_{(n+15)}]} \tag{5.4}$$

$$\boldsymbol{O_1 = G \odot I_1 = [y_n, y_{(n+1)}, y_{(n+2)}, y_{(n+3)}]} \tag{5.5}$$
$$\boldsymbol{O_2 = G \odot I_2 = [y_{(n+4)}, y_{(n+5)}, y_{(n+6)}, y_{(n+7)}]} \tag{5.6}$$

LOOP2

This follows the same structure as `LOOP1` but the registers are reversed. **The idea is to alternate between `LOOP1` and `LOOP2` to keep instructions nondependent and reduce pipeline stalls.**

$$\boldsymbol{I_1} = [\boldsymbol{x_{(n+8)}}, \boldsymbol{x_{(n+9)}}, \boldsymbol{x_{(n+10)}}, \boldsymbol{x_{(n+11)}}] \tag{5.7}$$

$$\boldsymbol{I_2} = [\boldsymbol{x_{(n+12)}}, \boldsymbol{x_{(n+13)}}, \boldsymbol{x_{(n+14)}}, \boldsymbol{x_{(n+15)}}] \tag{5.8}$$

$$\boldsymbol{O_3} = \boldsymbol{G} \odot \boldsymbol{I_3} = [\boldsymbol{y_n}, \boldsymbol{y_{(n+1)}}, \boldsymbol{y_{(n+2)}}, \boldsymbol{y_{(n+3)}}] \tag{5.9}$$

$$\boldsymbol{O_4} = \boldsymbol{G} \odot \boldsymbol{I_4} = [\boldsymbol{y_{(n+4)}}, \boldsymbol{y_{(n+5)}}, \boldsymbol{y_{(n+6)}}, \boldsymbol{y_{(n+7)}}] \tag{5.10}$$

ENDLOOP

The final section of code is the `ENDLOOP` which deals with the procedure when the function exits. It calculates the remaining samples left in the buffer. It follows the same approach as `LOOP2` but excludes the load operation. It operations are the same as represented in Eqs. 5.9 and 5.10.

Algorithm of the Gain SIMD function

The functions works as shown in algorithm 5 below.

Algorithm 5: Gain SIMD function for buffer size $\boldsymbol{N} \geq 16$

```
INIT
buffer_size = buffer_size - 16
LOOP1
If (buffer_size == 0)
    GOTO ENDLOOP

StartLoop:
    LOOP2
    Store O1 and O2
    buffer_size = buffer_size - 16
    LOOP1
    Store O3 and O4
    If (buffer_size == 0)
        GOTO ENDLOOP
    GOTO StartLoop

ENDLOOP
Store O1 and O2
Store O3 and O4
```

The program flow is as described below:

1. `INIT` loads the samples $(x_0, x_1, ..., x_7)$.
2. `buffer_size` is negated by 16 as 16 output samples will be produced.
3. In `LOOP1`, input samples $(x_8, x_9, ..., x_{15})$ are loaded while the output samples $(y_0, y_1, ..., y_7)$. are computed.
4. Program enters `StartLoop`. Every iteration, `buffer_size` is negated by 16. Let I be the iteration count, which represents the number of times `StartLoop` has iterated.
5. In each iteration, `StartLoop` calculates outputs $(y_n, y_{(n+1)}, ..., y_{(n+15)})$. The loop iterates until `buffer_size` $= 0$.
6. Note that after `LOOP2` calculates the $(y_n, y_{(n+1)}, ..., y_{(n+7)})$, a store of the outputs obtained from `LOOP1` in the previous iteration is performed. This is because, at this point in the code, the results of the operation of `LOOP1` are available. This principle also applies to the store operation after `LOOP1`.
7. The `ENDLOOP` section calculates $(y_{(N-8)}, ..., y_{(N-1)})$. and stores $(y_{(N-16)}, ..., y_{(N-1)})$.

5.2.2 Results

To test the performance of the three implementations, the execution time measurements were taken over one million iterations for a buffers size of 64. Figure 5.1 shows the probability histogram of the execution times taken over 1 million iterations and the results are tabulated in Table 5.1. The approach described above has an execution time of 52 clock cycles (median) while the NE10 library takes almost twice the amount of time, around 90 clock cycles (median). The NE10 vectorized multiplication code loads four samples, multiplies then and stores it back. This process is sequential with pipeline stalls and dependencies, which impair the performance. The auto-vectorized code performs poorly in comparison to the other two, taking around 134 clock cycles.

Table 5.1: Gain algorithm execution time results.

Performance Metric	SIMD Vectorized Gain Implementation	NE10 Library Gain Implementation	Auto-Vectorized Gain Implementation
Minimum execution time (clock cycles)	51	90	134
Maximum execution time (clock cycles)	109	683	638
Median execution time (clock cycles)	52	90	137
Speedup against NE10's code	**1.73**		
Speedup against Auto-Vectorized Code	**2.635**		
Confidence Rate (1-α)	99%		

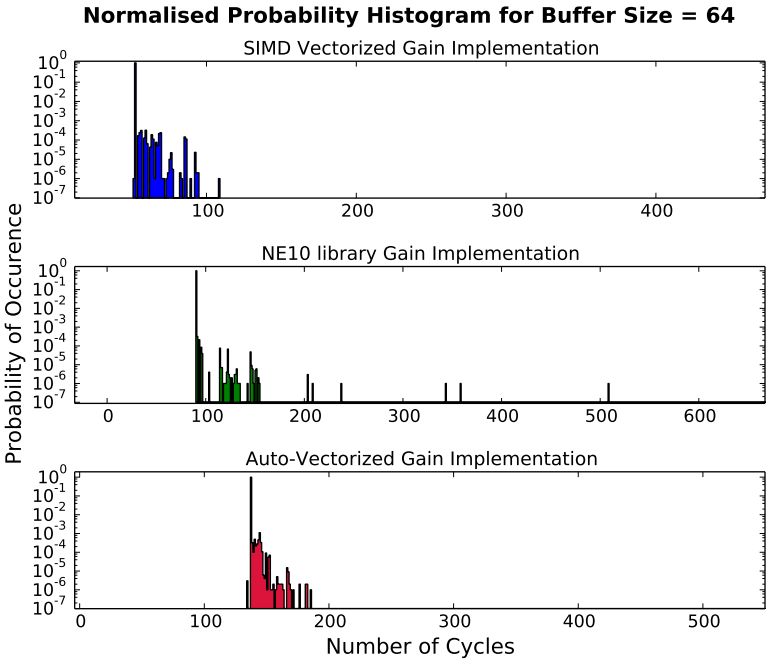

Figure 5.1: Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Gain algorithm.

Table 5.1 shows the speedup calculated using the medians. The SIMD Vectorized gain implementation is **2.635** times faster than the auto-vectorized implementation and **1.7** times faster than the NE10 implementation. Also the confidence rate $\mathbf{1-\alpha}$ is 99% because the distributions intersect very less with each other. The maximum execution cycle reading for the SIMD gain function is 109 clock cycles, which is lesser than the minimum execution cycles taken by the C function. This shows that, even for a simple gain functionality, hand optimized SIMD vectorized code works much more efficient compared to auto-vectorized produced by the compiler. Also, it can be inferred that pipeline stalls can cause significant performance

loss.

5.3 Mix

The Mix algorithm is also quite simple and only architectural optimizations along with proper scheduling of instructions can help increase performance. It works exactly similar to the Gain function and has the same four algorithm sections.

5.3.1 Architectural Optimization and Implementation

The approach described here is the same as the one for the `Gain` algorithm, with the exception that it has two input signals. The code loads the next set of samples while calculating the output for the current set. It calculates the output in chunks of 16 samples and so the buffer size should be a minimum of 16 or multiples of 16. The algorithm can be represented with four parts, `INIT`, `LOOP1`, `LOOP2` and finally `ENDLOOP`.

Signal Denotations

- The first input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{A}$ and its samples are denoted as $\boldsymbol{a_n}$, where $\mathbf{0} \leq \boldsymbol{n} < \boldsymbol{N}$.
- The second input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{B}$ and its samples are denoted as $\boldsymbol{b_n}$, where $\mathbf{0} \leq \boldsymbol{n} < \boldsymbol{N}$.
- The output signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{Y}$ and its samples are denoted as $\boldsymbol{y_n}$, where $\mathbf{0} \leq \boldsymbol{n} < \boldsymbol{N}$.

SIMD Register Denotations

- $\boldsymbol{I_{(A,1)}}$, $\boldsymbol{I_{(A,2)}}$, $\boldsymbol{I_{(A,3)}}$ and $\boldsymbol{I_{(A,4)}}$: Quadword registers to contain samples $\boldsymbol{a_n}$ from signal $\boldsymbol{A}$.
- $\boldsymbol{I_{(B,1)}}$, $\boldsymbol{I_{(B,2)}}$, $\boldsymbol{I_{(B,3)}}$ and $\boldsymbol{I_{(B,4)}}$: Quadword registers to contain samples $\boldsymbol{b_n}$ from signal $\boldsymbol{B}$.
- $\boldsymbol{O_1}$, $\boldsymbol{O_2}$, $\boldsymbol{O_3}$, $\boldsymbol{O_4}$: Quadword registers to contain output signal samples $\boldsymbol{y_n}$.

INIT

The initial operations before the main loop is shown in the equations below in Eqs. 5.11, 5.12, 5.13 and 5.14. Eight samples from signal $\boldsymbol{A}$ is loaded into $\boldsymbol{I_{(A,1)}}$, $\boldsymbol{I_{(A,2)}}$

and eight from signal $\boldsymbol{B}$ is loaded into $\boldsymbol{I}_{(B,1)}$, $\boldsymbol{I}_{(B,2)}$.

$$\boldsymbol{I}_{(A,1)} = [\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3] \tag{5.11}$$
$$\boldsymbol{I}_{(A,2)} = [\boldsymbol{a}_4, \boldsymbol{a}_5, \boldsymbol{a}_6, \boldsymbol{a}_7] \tag{5.12}$$
$$\boldsymbol{I}_{(B,1)} = [\boldsymbol{b}_0, \boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3] \tag{5.13}$$
$$\boldsymbol{I}_{(B,2)} = [\boldsymbol{b}_4, \boldsymbol{b}_5, \boldsymbol{b}_6, \boldsymbol{b}_7] \tag{5.14}$$

LOOP1

The approach taken here is to parallelize the `VLD` and `VADD` operation on the two input registers for both the signals. The first part of the loop begins by loading input samples $(\boldsymbol{a}_8, \boldsymbol{a}_9, ..., \boldsymbol{a}_{15})$ and $(\boldsymbol{b}_8, \boldsymbol{b}_9, ..., \boldsymbol{b}_{15})$ for the next calculation as shown in Eqs. 5.15, 5.16, 5.17 and 5.18. This load operation can run in parallel whilst the calculation can be performed on samples $(\boldsymbol{a}_n, \boldsymbol{a}_{(n+1)}, ..., \boldsymbol{a}_{(n+7)})$ and $(\boldsymbol{b}_n, \boldsymbol{b}_{(n+1)}, ..., \boldsymbol{b}_{(n+7)})$ as shown in Eqs. 5.19 and 5.20.

$$\boldsymbol{I}_{(A,3)} = [\boldsymbol{a}_{(n+8)}, \boldsymbol{a}_{(n+9)}, \boldsymbol{a}_{(n+10)}, \boldsymbol{a}_{(n+11)}] \tag{5.15}$$
$$\boldsymbol{I}_{(A,4)} = [\boldsymbol{a}_{(n+12)}, \boldsymbol{a}_{(n+13)}, \boldsymbol{a}_{(n+14)}, \boldsymbol{a}_{(n+15)}] \tag{5.16}$$
$$\boldsymbol{I}_{(B,3)} = [\boldsymbol{b}_{(n+8)}, \boldsymbol{b}_{(n+9)}, \boldsymbol{b}_{(n+10)}, \boldsymbol{b}_{(n+11)}] \tag{5.17}$$
$$\boldsymbol{I}_{(B,4)} = [\boldsymbol{b}_{(n+12)}, \boldsymbol{b}_{(n+13)}, \boldsymbol{b}_{(n+14)}, \boldsymbol{b}_{(n+15)}] \tag{5.18}$$

$$\boldsymbol{O}_1 = \boldsymbol{I}_{(A,1)} \oplus \boldsymbol{I}_{(B,1)} = [\boldsymbol{y}_n, \boldsymbol{y}_{(n+1)}, \boldsymbol{y}_{(n+2)}, \boldsymbol{y}_{(n+3)}] \tag{5.19}$$
$$\boldsymbol{O}_2 = \boldsymbol{I}_{(A,2)} \oplus \boldsymbol{I}_{(B,2)} = [\boldsymbol{y}_{(n+4)}, \boldsymbol{y}_{(n+5)}, \boldsymbol{y}_{(n+6)}, \boldsymbol{y}_{(n+7)}] \tag{5.20}$$

LOOP2

This code section follows the same structure as `LOOP1` with the exception that the registers are reversed as shown below.

$$\boldsymbol{I}_{(A,1)} = [\boldsymbol{a}_{(n+8)}, \boldsymbol{a}_{(n+9)}, \boldsymbol{a}_{(n+10)}, \boldsymbol{a}_{(n+11)}] \tag{5.21}$$
$$\boldsymbol{I}_{(A,2)} = [\boldsymbol{a}_{(n+12)}, \boldsymbol{a}_{(n+13)}, \boldsymbol{a}_{(n+14)}, \boldsymbol{a}_{(n+15)}] \tag{5.22}$$
$$\boldsymbol{I}_{(B,1)} = [\boldsymbol{b}_{(n+8)}, \boldsymbol{b}_{(n+9)}, \boldsymbol{b}_{(n+10)}, \boldsymbol{b}_{(n+11)}] \tag{5.23}$$
$$\boldsymbol{I}_{(B,2)} = [\boldsymbol{b}_{(n+12)}, \boldsymbol{b}_{(n+13)}, \boldsymbol{b}_{(n+14)}, \boldsymbol{b}_{(n+15)}] \tag{5.24}$$

$$\boldsymbol{O}_3 = \boldsymbol{I}_{(A,3)} \oplus \boldsymbol{I}_{(B,3)} = [\boldsymbol{y}_n, \boldsymbol{y}_{(n+1)}, \boldsymbol{y}_{(n+2)}, \boldsymbol{y}_{(n+3)}] \tag{5.25}$$
$$\boldsymbol{O}_4 = \boldsymbol{I}_{(A,4)} \oplus \boldsymbol{I}_{(B,4)} = [\boldsymbol{y}_{(n+4)}, \boldsymbol{y}_{(n+5)}, \boldsymbol{y}_{(n+6)}, \boldsymbol{y}_{(n+7)}] \tag{5.26}$$

ENDLOOP

The final section of code is the `ENDLOOP` section which deals with the procedure when the function exits. It calculates the remaining samples left in the buffer. It

follows the same approach as `LOOP2` but excludes the load operation. Its operations are the same as represented in Eqs. 5.25 and 5.26.

Algorithm of Mix SIMD function

The functions works as shown in algorithm 6 below.

Algorithm 6: Mix SIMD function for buffer size $N \geq 16$

```
INIT
buffer_size = buffer_size - 16
LOOP1
If (buffer_size == 0)
    GOTO ENDLOOP

LOOP2
Store O1 and O2

StartLoop:
    buffer_size = buffer_size - 16
    LOOP1
    Store O3 and O4
    If (buffer_size == 0)
        GOTO ENDLOOP
    LOOP2
    Store O1 and O2
    GOTO StartLoop

ENDLOOP
Store O1 and O2
Store O3 and O4
```

The program flow is as described below:

1. `INIT` loads the samples $(a_0, a_1, ..., a_7)$ and $(b_0, b_1, ..., b_7)$.
2. `buffer_size` is negated by 16.
3. In `LOOP1`, input samples $(a_8, a_9, ..., a_{15})$ and $(b_8, b_9, ..., b_{15})$ are loaded while output samples $(y_0, y_1, ..., y_7)$ are calculated.
4. In `LOOP2`, input samples $(a_{16}, a_{17}, ..., a_{23})$ and $(b_{16}, b_{17}, ..., b_{23})$ are loaded while output samples $(y_8, y_9, ..., y_{15})$ are calculated.
5. Output samples $(y_0, y_1, ..., y_7)$ are stored.
6. Program enters `StartLoop`. Every iteration `buffer_size` is negated by 16. Let I be the iteration count, which represents the number of times `StartLoop` has iterated.

7. In each iteration, `StartLoop` calculates outputs $(y_n, y_{(n+1)}, \ldots, y_{(n+15)})$. The loop iterates until `buffer_size` $= \mathbf{0}$ and has calculated outputs $(y_{(n-16)}, y_{(n-15)}, \ldots, y_{(n-9)})$.

8. The `ENDLOOP` section calculates $(y_{(n-8)}, y_{(n-7)}, \ldots, y_{(n-1)})$ and stores $(y_{(n-16)}, y_{(n-15)}, \ldots, y_{(n-1)})$.

5.3.2 Results

Again, the performance is tested for all the three implementations by measuring the execution time for 1 million iterations and the results are shown below. Figure 5.2 shows the normalized probability distribution of the execution times. It can that the approach described above takes 54 clock cycles 99% of the time, with four peaks above 400 clock cycles. Execution clock cycles of more than 54 occur less than 0.001% of the time. The auto-vectorized function takes 190 clock cycles 99% of the time with some peaks occurring rarely. It takes more time as there is more cost associated with loading and storing of memory compared to the hand optimized version. The NE10 library's Mix function performs significantly slower as well. It takes 96 clock cycles, almost twice as the hand optimized SIMD version. As was the case with the NE10's Gain implementation, pipeline stalls impair the performance of the code.

Table 5.2 shows the speedups(median) obtained and medians of the three imple-

Table 5.2: Mix algorithm execution time results.

Performance Metric	SIMD Vectorized Mix Implementation	NE10 Library Mix Implementation	Auto-Vectorized Mix Implementation
Minimum execution time (clock cycles)	53	96	183
Maximum execution time (clock cycles)	520	292	654
Median execution time (clock cycles)	54	96	190
Speedup as compared with NE10's code	**1.77**		
Speedup as compared with auto-vectorized code	**3.518**		
Confidence Rate (1-α)	99%		

mentations. The speedup as compared with the auto-vectorized code is **3.51** and compared to NE10's code, the speedup is **1.77**. The median values are very close to minimum and they occur more 99% as shown in Fig. 5.2 for the three implementations. The confidence rate is nearly 100% but for the sake of safety it is rounded to 99%. To summarize, the approach described above delivers much better performance compared to even the NE10 library, mainly due to the fact that it vectorizes in a sequential way.

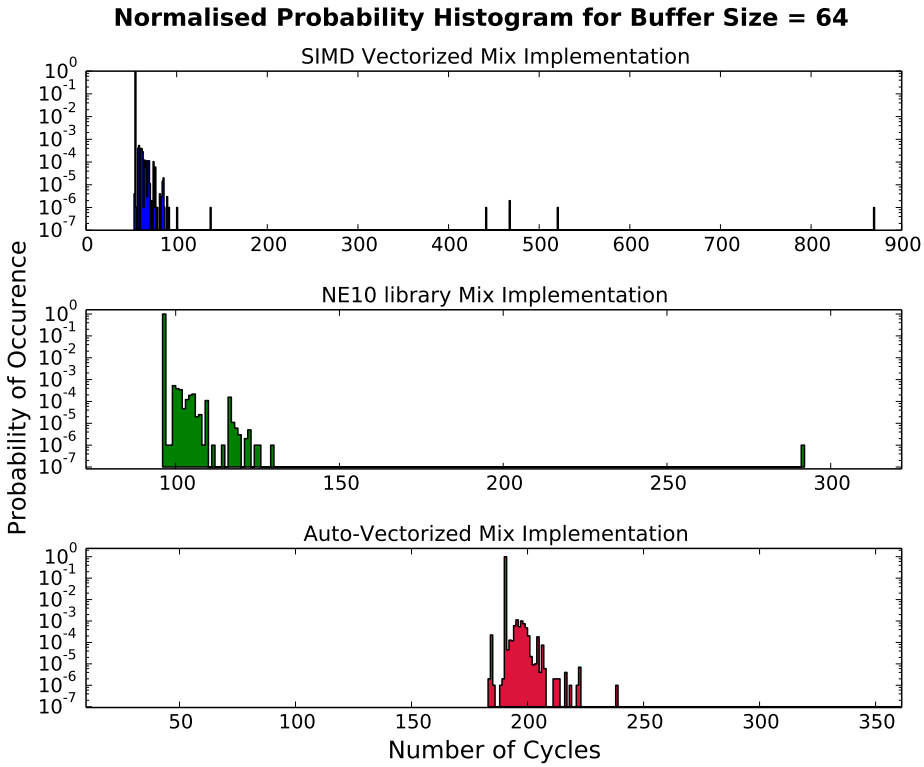

Figure 5.2: Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Mix algorithm.

5.4 Gain and Mix

This function applies gain to two signals and adds them to produce the output. While there is not much possibility to perform algorithmic optimizations, there is a lot of room to optimize with better instruction scheduling. The algorithm and instruction scheduling methodology is described below and the performance is compared to that of the NE10 library and the auto-vectorized code.

5.4.1 Architectural Optimization and Implementation

There are more independent arithmetic operations in this function compared to the previous two. This means that while a set of outputs for input signal $\boldsymbol{A}$ is being calculated, the inputs from signal B can be loaded and worked upon simultaneously, thereby increasing the throughput. The following set of equations explains the general instruction scheduling methodology. It calculates the output in chunks of 16 samples and so the signal size should be a minimum of 16 or multiples of 16. The algorithm is comprised of four sections `INIT`, `LOOP1`, `LOOP2` and finally `ENDLOOP`.

Signal Denotations

- The first input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{A}$ and its samples are denoted as $\boldsymbol{a_n}$, where $\boldsymbol{0 \leq n < N}$.
- The second input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{B}$ and its samples are denoted as $\boldsymbol{b_n}$, where $\boldsymbol{0 \leq n < N}$.
- The output signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{Y}$ and its samples are denoted as $\boldsymbol{y_n}$, where $\boldsymbol{0 \leq n < N}$.
- The gain constant for signal $\boldsymbol{A}$ is $\boldsymbol{k_a}$.
- The gain constant for signal $\boldsymbol{B}$ is $\boldsymbol{k_b}$.

SIMD Register Denotations

- $\boldsymbol{I_{(A,1)}}$, $\boldsymbol{I_{(A,2)}}$, $\boldsymbol{I_{(A,3)}}$ and $\boldsymbol{I_{(A,4)}}$: Quadword registers to contain samples $\boldsymbol{a_n}$ from signal $\boldsymbol{A}$.
- $\boldsymbol{I_{(B,1)}}$, $\boldsymbol{I_{(B,2)}}$, $\boldsymbol{I_{(B,3)}}$ and $\boldsymbol{I_{(B,4)}}$: Quadword registers to contain samples $\boldsymbol{b_n}$ from signal $\boldsymbol{B}$.
- $\boldsymbol{O_1}$, $\boldsymbol{O_2}$, $\boldsymbol{O_3}$, $\boldsymbol{O_4}$: Quadword registers to contain output signal samples $\boldsymbol{y_n}$.
- $\boldsymbol{G_A}$: Upper half of doubleword register $\boldsymbol{G}$ to contain $\boldsymbol{k_a}$.
- $\boldsymbol{G_B}$: Lower half of doubleword register $\boldsymbol{G}$ to contain $\boldsymbol{k_b}$.

INIT

The initial operations before the main loop is shown in the equations below in Eqs. 5.27, 5.28, 5.29 and 5.30. Eight samples from each signal $\boldsymbol{A}$ is loaded into $\boldsymbol{I_{(A,1)}}$, $\boldsymbol{I_{(A,2)}}$ and eight from signal $\boldsymbol{B}$ is loaded into $\boldsymbol{I_{(B,1)}}$, $\boldsymbol{I_{(B,2)}}$.

$$\boldsymbol{I_{(A,1)} = [a_0, a_1, a_2, a_3]} \tag{5.27}$$

$$\boldsymbol{I_{(A,2)} = [a_4, a_5, a_6, a_7]} \tag{5.28}$$

$$\boldsymbol{I_{(B,1)} = [b_0, b_1, b_2, b_3]} \tag{5.29}$$

$$\boldsymbol{I_{(B,2)} = [b_4, b_5, b_6, b_7]} \tag{5.30}$$

LOOP1

The approach taken here is to parallelize the `VMUL` and `VMLA` operation on the two registers for both the input signals. The first part of the loop begins by loading input samples $\boldsymbol{(a_8, a_9, ..., a_{15})}$ and $\boldsymbol{(b_8, b_9, ..., b_{15})}$ for the next calculation as shown in Eqs. 5.31, 5.32, 5.33 and 5.34. This load operation can run in parallel

whilst the calculation can be performed on samples $(a_n, a_{(n+1)}, ..., a_{(n+7)})$ and $(b_n, b_{(n+1)}, ..., b_{(n+7)})$ as shown in Eqs. 5.35, 5.36, 5.37 and 5.38.

$$I_{(A,3)} = [a_{(n+8)}, a_{(n+9)}, a_{(n+10)}, a_{(n+11)}] \tag{5.31}$$
$$I_{(A,4)} = [a_{(n+12)}, a_{(n+13)}, a_{(n+14)}, a_{(n+15)}] \tag{5.32}$$
$$I_{(B,3)} = [b_{(n+8)}, b_{(n+9)}, b_{(n+10)}, b_{(n+11)}] \tag{5.33}$$
$$I_{(B,4)} = [b_{(n+12)}, b_{(n+13)}, b_{(n+14)}, b_{(n+15)}] \tag{5.34}$$

$$O_1 = G_A \odot I_{(A,1)} \tag{5.35}$$
$$O_2 = G_A \odot I_{(A,2)} \tag{5.36}$$
$$O_1 = O_1 \oplus (G_B \odot I_{(B,1)}) = [y_n, y_{(n+1)}, y_{(n+2)}, y_{(n+3)}] \tag{5.37}$$
$$O_2 = O_2 \oplus (G_B \odot I_{(B,2)}) = [y_{(n+4)}, y_{(n+5)}, y_{(n+6)}, y_{(n+7)}] \tag{5.38}$$

Equations 5.35 and 5.37 are dependent on each other. There will be a pipeline halt if they are placed in a succeeding manner. So a nondependent instruction as shown in Eq. 5.36 can be placed in between. This is synergistic because Eq. 5.38 is dependent on Eq. 5.36. Thus, by interleaving operations on two separate signals, the dependencies can be solved. This is the reason eight samples are loaded into the memory instead of four.

LOOP2

The second part of the loop follows the same structure as the first part, with the only exception that the input and output registers are reversed, as shown below.

$$I_{(A,1)} = [a_{(n+8)}, a_{(n+9)}, a_{(n+10)}, a_{(n+11)}] \tag{5.39}$$
$$I_{(A,2)} = [a_{(n+12)}, a_{(n+13)}, a_{(n+14)}, a_{(n+15)}] \tag{5.40}$$
$$I_{(B,1)} = [b_{(n+8)}, b_{(n+9)}, b_{(n+10)}, b_{(n+11)}] \tag{5.41}$$
$$I_{(B,2)} = [b_{(n+12)}, b_{(n+13)}, b_{(n+14)}, b_{(n+15)}] \tag{5.42}$$

$$O_3 = G_A \odot I_{(A,3)} \tag{5.43}$$
$$O_4 = G_A \odot I_{(A,4)} \tag{5.44}$$
$$O_3 = O_3 \oplus G_B \odot I_{(B,3)} = [y_n, y_{(n+1)}, y_{(n+2)}, y_{(n+3)}] \tag{5.45}$$
$$O_4 = O_4 \oplus G_B \odot I_{(B,4)} = [y_{(n+4)}, y_{(n+5)}, y_{(n+6)}, y_{(n+7)}] \tag{5.46}$$

ENDLOOP

This section of code deals with the procedure when the loop exits. It follows the same approach as `LOOP2`, but excludes the load operation. Its operations are the same as shown in Eqs. 5.43, 5.44, 5.45 and 5.46.

Algorithm of Gain and Mix SIMD function

The algorithm works exactly as shown in algorithm 6 for the Mix function, with the difference being the `LOOP1`, `LOOP2` and `ENDLOOP` work as described for the Gain and Mix function.

5.4.2 Results

For a buffer size $\boldsymbol{N = 64}$, the execution time for both the three implementations are measured and their respective probability histograms are shown in Fig. 5.3. The approach described above takes 62 clock cycles more than 99% of one million iterations whereas the auto-vectorized implementation takes 240 clock cycles. It is interesting to note that the Gain and Mix Function implemented with NE10 library performs much slower. This is again because of the pipeline stalls and dependencies. Table 5.3 shows the performance numbers for this function. The speedup of median execution times compared with the NE10 implementation is **2.87** and compared with the auto-vectorized code, it is **3.87**. with a confidence rate $\boldsymbol{1 - \alpha > 0.99}$. The confidence rate is nearly 100% but due to the two outliers measured in the SIMD implementation, it is rounded off to 99% for a more conservative rate.

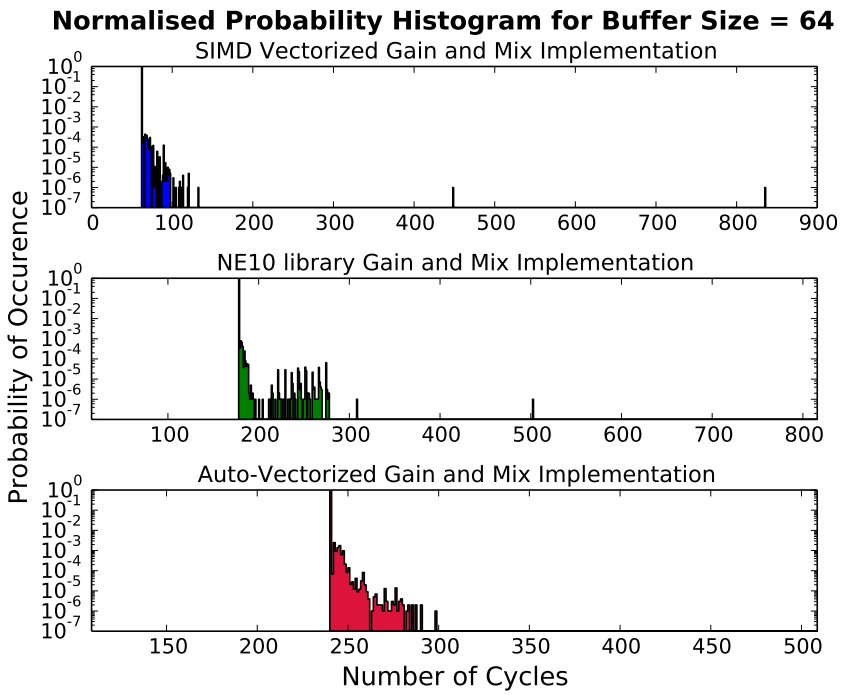

Figure 5.3: Normalized probability histograms for the SIMD Vectorized implementation, NE10 implementation and auto-vectorized implementation of the Gain and Mix algorithm.

Table 5.3: Gain and Mix algorithm execution time results.

Performance Metric	SIMD Vectorized Gain and Mix Implementation	NE10 Library Gain and Mix Implementation	Auto-Vectorized Gain and Mix Implementation
Minimum execution time (clock cycles)	62	178	240
Maximum execution time (clock cycles)	836	833	555
Median execution time (clock cycles)	62	178	240
Speedup as compared with NE10's code	**2.87**		
Speedup as compared with auto-vectorized code	**3.87**		
Confidence Rate (1-α)	99%		

5.5 Complex Number Multiplication

The complex number multiplication is a basic algorithm with dependencies as described in section 2.5.4. Due to the way the data is arranged in the memory, conventional vectorization as approached for the functions above cannot be applied here because of the dependencies. So a technique of vectorization with deinterleaving the input data is implemented. The function accepts four arguments as described below:

5.5.1 Architectural Optimization and Implementation

The complex samples from the input buffers are arranged in the memory as shown in Fig. 2.19. They can be deinterleaved using the `VLD2.32` instruction. When called to load 8 samples, it places the even addresses in one register and the odd ones in another, essentially splitting the complex numbers in the buffer into real and imaginary parts. Consider 8 samples from complex number signal $\boldsymbol{A}$ containing complex numbers as shown in Eq. 5.47.

$$\boldsymbol{A} = ((\Re \boldsymbol{A_0}, \Im \boldsymbol{A_0}), (\Re \boldsymbol{A_1}, \Im \boldsymbol{A_1}), (\Re \boldsymbol{A_2}, \Im \boldsymbol{A_2}), (\Re \boldsymbol{A_3}, \Im \boldsymbol{A_3})) \tag{5.47}$$

When `VLD2.32` instruction is used to load data from $\boldsymbol{A}$ into two SIMD registers $\boldsymbol{R_A}$ and $\boldsymbol{I_A}$, it loads the even indexes into the first register and the odd ones into the second, essentially splitting the real and imaginary parts as shown in the Eq. 5.49. When storing it back into the memory, the `VST2.32` interleaves the data in the registers and stores it in the memory in the same fashion as expressed in 5.47.

$$\boldsymbol{R_A} = (\Re \boldsymbol{A_0}, \Re \boldsymbol{A_1}, \Re \boldsymbol{A_2}, \Re \boldsymbol{A_3}) \tag{5.48}$$

$$\boldsymbol{I_A} = (\Re \boldsymbol{A_0}, \Re \boldsymbol{A_1}, \Re \boldsymbol{A_2}, \Re \boldsymbol{A_3}) \tag{5.49}$$

Now that the real and imaginary parts are split, there are no more dependency issues. The same technique of using multiple input buffers with nondependent data is used to increase instruction level parallelism and ultimately, the throughput. The algorithm with four sections `INIT`, `LOOP1`, `LOOP2` and finally `ENDLOOP`.

Signal Denotations

- The first complex input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{A}$ and its samples are denoted as $\Re \boldsymbol{a_n} + \Im \boldsymbol{a_n}$, where $\boldsymbol{0 \leq n < N}$
- The second complex input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{B}$ and its samples are denoted as $\Re \boldsymbol{b_n} + \Im \boldsymbol{b_n}$, where $\boldsymbol{0 \leq n < N}$
- The output complex input signal of size $\boldsymbol{N}$ is denoted as $\boldsymbol{Y}$ and its samples are denoted as $\Re \boldsymbol{y_n} + \Im \boldsymbol{y_n}$, where $\boldsymbol{0 \leq n < N}$

SIMD Register Denotations

- $\boldsymbol{R_{(A,1)}}$, $\boldsymbol{R_{(A,2)}}$: Two quadword registers for real parts of input signal $\boldsymbol{A}$.
- $\boldsymbol{I_{(A,1)}}$, $\boldsymbol{I_{(A,2)}}$: Two quadword registers for imaginary parts of input signal $\boldsymbol{A}$.
- $\boldsymbol{R_{(B,1)}}$, $\boldsymbol{R_{(B,2)}}$: Two quadword registers for real parts of input signal $\boldsymbol{B}$.
- $\boldsymbol{I_{(B,1)}}$, $\boldsymbol{I_{(B,2)}}$: Two quadword registers for imaginary parts of input signal $\boldsymbol{B}$.
- $\boldsymbol{O_{(R,1)}}$, $\boldsymbol{O_{(R,2)}}$: Two quadword registers for the real parts of output signal $\boldsymbol{Y}$.
- $\boldsymbol{O_{(I,1)}}$, and $\boldsymbol{O_{(I,2)}}$: Two quadword registers for the imaginary parts of output signal $\boldsymbol{Y}$.

INIT

This code deals with the initial procedure when the function is called. First, the code loads eight samples of complex numbers from both the signals into their respective registers using the `VLD2.32` as shown below in Eqs. 5.50, 5.51, 5.52 and 5.53 .

$$\boldsymbol{R_{(A,1)}} = [\Re \boldsymbol{a_0}, \Re \boldsymbol{a_1}, \Re \boldsymbol{a_2}, \Re \boldsymbol{a_3}] \tag{5.50}$$

$$\boldsymbol{I_{(A,1)}} = [\Im \boldsymbol{a_0}, \Im \boldsymbol{a_1}, \Im \boldsymbol{a_2}, \Im \boldsymbol{a_3}] \tag{5.51}$$

$$\boldsymbol{R_{(B,1)}} = [\Re \boldsymbol{b_0}, \Re \boldsymbol{b_1}, \Re \boldsymbol{b_2}, \Re \boldsymbol{b_3}] \tag{5.52}$$

$$\boldsymbol{I_{(B,2)}} = [\Im \boldsymbol{b_0}, \Im \boldsymbol{b_1}, \Im \boldsymbol{b_2}, \Im \boldsymbol{b_3}] \tag{5.53}$$

LOOP1

The approach taken here is similar to the previous algorithms, where the succeeding set of complex numbers samples are loaded into the memory while calculating the results of the current working set.

$$\boldsymbol{R}_{(A,2)} = [\Re \boldsymbol{a}_{(n+4)}, \Re \boldsymbol{a}_{(n+5)}, \Re \boldsymbol{a}_{(n+6)}, \Re \boldsymbol{a}_{(n+7)}] \tag{5.54}$$
$$\boldsymbol{I}_{(A,2)} = [\Im \boldsymbol{a}_{(n+4)}, \Im \boldsymbol{a}_{(n+5)}, \Im \boldsymbol{a}_{(n+6)}, \Im \boldsymbol{a}_{(n+7)}] \tag{5.55}$$
$$\boldsymbol{R}_{(B,2)} = [\Re \boldsymbol{a}_{(b+4)}, \Re \boldsymbol{a}_{(b+5)}, \Re \boldsymbol{b}_{(n+6)}, \Re \boldsymbol{b}_{(n+7)}] \tag{5.56}$$
$$\boldsymbol{I}_{(B,2)} = [\Im \boldsymbol{a}_{(b+4)}, \Im \boldsymbol{a}_{(b+5)}, \Im \boldsymbol{b}_{(n+6)}, \Im \boldsymbol{b}_{(n+7)}] \tag{5.57}$$

$$\boldsymbol{O}_{(R,1)} = \boldsymbol{R}_{(A,1)} \odot \boldsymbol{R}_{(B,1)} \tag{5.58}$$
$$\boldsymbol{O}_{(I,1)} = \boldsymbol{R}_{(A,1)} \odot \boldsymbol{I}_{(B,1)} \tag{5.59}$$
$$\boldsymbol{O}_{(R,1)} = \boldsymbol{O}_{(R,1)} \ominus (\boldsymbol{I}_{(A,1)} \odot \boldsymbol{I}_{(B,1)}) \tag{5.60}$$
$$= [\Re \boldsymbol{y}_n, \Re \boldsymbol{y}_{(n+1)}, \Re \boldsymbol{y}_{(n+2)}, \Re \boldsymbol{y}_{(n+3)}] \tag{5.61}$$
$$\boldsymbol{O}_{(I,1)} = \boldsymbol{O}_{(I,1)} \oplus (\boldsymbol{I}_{(A,1)} \odot \boldsymbol{R}_{(B,1)}) \tag{5.62}$$
$$= [\Im \boldsymbol{y}_n, \Im \boldsymbol{y}_{(n+1)}, \Im \boldsymbol{y}_{(n+2)}, \Im \boldsymbol{y}_{(n+3)}] \tag{5.63}$$

LOOP2

This section is similar to `LOOP1` but uses alternate registers.

$$\boldsymbol{R}_{(A,1)} = [\Re \boldsymbol{a}_{(n+4)}, \Re \boldsymbol{a}_{(n+5)}, \Re \boldsymbol{a}_{(n+6)}, \Re \boldsymbol{a}_{(n+7)}] \tag{5.64}$$
$$\boldsymbol{I}_{(A,1)} = [\Im \boldsymbol{a}_{(n+4)}, \Im \boldsymbol{a}_{(n+5)}, \Im \boldsymbol{a}_{(n+6)}, \Im \boldsymbol{a}_{(n+7)}] \tag{5.65}$$
$$\boldsymbol{R}_{(B,1)} = [\Re \boldsymbol{a}_{(b+4)}, \Re \boldsymbol{a}_{(b+5)}, \Re \boldsymbol{b}_{(n+6)}, \Re \boldsymbol{b}_{(n+7)}] \tag{5.66}$$
$$\boldsymbol{I}_{(B,1)} = [\Im \boldsymbol{a}_{(b+4)}, \Im \boldsymbol{a}_{(b+5)}, \Im \boldsymbol{b}_{(n+6)}, \Im \boldsymbol{b}_{(n+7)}] \tag{5.67}$$

$$\boldsymbol{O}_{(R,2)} = \boldsymbol{R}_{(A,2)} \odot \boldsymbol{R}_{(B,2)} \tag{5.68}$$
$$\boldsymbol{O}_{(I,2)} = \boldsymbol{R}_{(A,2)} \odot \boldsymbol{I}_{(B,2)} \tag{5.69}$$
$$\boldsymbol{O}_{(R,2)} = \boldsymbol{O}_{(R,2)} \ominus (\boldsymbol{I}_{(A,2)} \odot \boldsymbol{I}_{(B,2)}) \tag{5.70}$$
$$= [\Re \boldsymbol{y}_n, \Re \boldsymbol{y}_{(n+1)}, \Re \boldsymbol{y}_{(n+2)}, \Re \boldsymbol{y}_{(n+3)}] \tag{5.71}$$
$$\boldsymbol{O}_{(I,2)} = \boldsymbol{O}_{(I,2)} \oplus (\boldsymbol{I}_{(A,2)} \odot \boldsymbol{R}_{(B,2)}) \tag{5.72}$$
$$= [\Im \boldsymbol{y}_n, \Im \boldsymbol{y}_{(n+1)}, \Im \boldsymbol{y}_{(n+2)}, \Im \boldsymbol{y}_{(n+3)}] \tag{5.73}$$

ENDLOOP

As is the case with the previous algorithms, the `ENDLOOP` section deals with the last four complex numbers in the buffer. It is similar to `LOOP1` and is as exactly as that represented in Eqs. 5.68, 5.69, 5.70 and 5.72.

Algorithm of Complex Number Multiply SIMD function

The algorithm follows a very similar structure as that for the Mix SIMD function with the major difference being the number of samples calculated per iteration is 8 instead of 16. Every load operations perform deinterleaving of the data and every store operation interleaves it back and stores in into memory.

5.5.2 Results

For a buffer size of $\boldsymbol{N = 64}$, the execution time taken for approach described above and for the auto-vectorization implementation is measured for 1 million iterations. The probability histogram of the execution times are shown in Fig. 5.4. The histogram shows that the execution time with a probability of 99% for the SIMD implementation is 219 clock cycles and for the auto-vectorized implementation, it is about 378 clock cycles. The execution time for the auto-vectorized code exhibits a lot of variance and ranges from 377 to 564 clock cycles. This can be attributed to poor auto-vectorization of the algorithm by the compiler along with a lot of memory fetches.

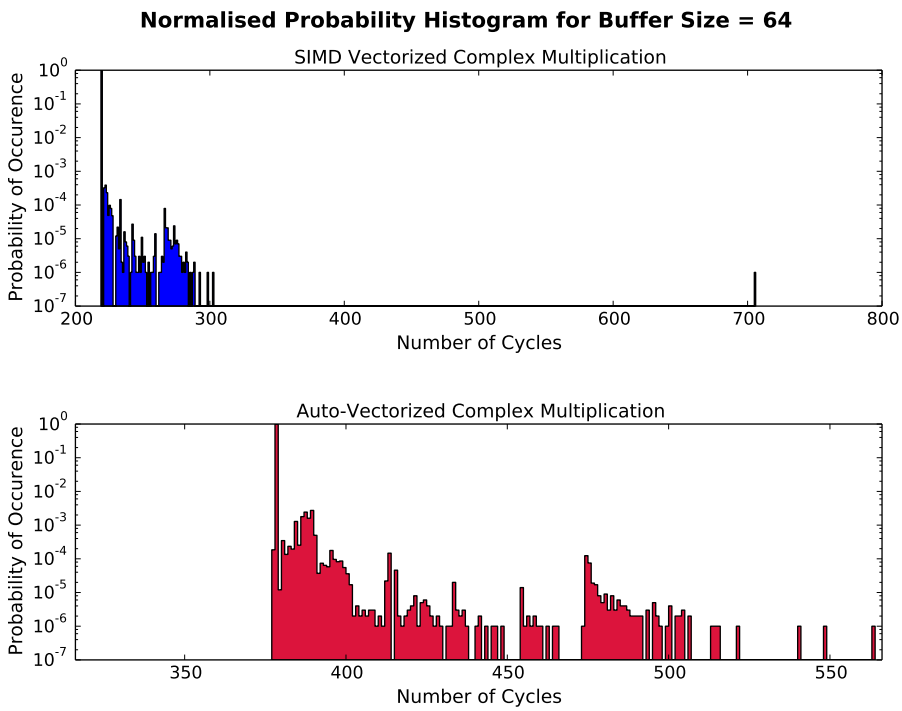

Figure 5.4: Normalized probability histograms for the SIMD Vectorized implementation and auto-vectorized implementation of the Complex Number Multiplication algorithm.

Table 5.4: Complex number multiplication algorithm execution time results

Performance Metric	SIMD Vectorized Complex Number Multiplication Implementation	Auto-Vectorized Complex Number Multiplication Implementation
Minimum execution time (clock cycles)	219	96
Maximum execution time (clock cycles)	564	292
Median execution time (clock cycles)	219	96
Speedup as compared with auto-vectorized code	**1.72**	
Confidence Rate (1-α)	99%	

Table 5.4 shows the performance results. The speed up obtained is only **1.72**. This is attributed to the fact that it can only work on 8 samples per iteration, each having 6 SIMD floating point operations, compared to 2 floating point operations in the Gain and Mix function. Pipeline stalls are caused by the multiply and MAC hardware unit, responsible for computing the Eqs. 5.58, 5.59, 5.60 and 5.62 being unavailable as multiple floating point operations are queued one after the other. This is also the case for Eqs. 5.68, 5.69, 5.70 and 5.72. Other operations can be interleaved with these instructions, but in the case of this algorithm, only load/store operations can be interleaved. If the buffer size is fixed, the loops can be manually unrolled to improve performance.

5.6 Peak Program Meter

One of the biggest problems with the Peak Program Meter algorithm when it comes to vectorization is the dependency caused by the one pole filter. It is made worse by the branching condition as well. As shown in Eqs. 2.32 and 2.33, different filter coefficients are used depending on the amplitude of the input signal. It is not possible to vectorize directly on the inputs due to this reason. However, after removing branches from the algorithm, a technique of vectorizing on multiple **channels** is described below.

5.6.1 Algorithmic and Architectural Optimization

The peak program meter algorithm has to applied to $\boldsymbol{C}$ number of channels, each which contain $\boldsymbol{N}$ number of samples. Vectorization is difficult because it has a one pole recursive filter. Moreover, the coefficient is chosen depending on the amplitude, thereby requiring a branching condition. The NEON SIMD instruction set does not support any branching based on logical comparison operations. This is because such a feature requires the result of the logical comparison to be the same for all the lanes in the vector. However, it provides vector masking instructions such as the `VCGT` instruction. If two SIMD registers are passed as inputs to `VCGT`, it will

perform a logical comparison on each lane of the two SIMD registers to find which is greater. Then it will fill the respective lane in the result register with ones if the logical result of the comparison is true or zeros if it is false. Equation 5.74 shows the mathematical representation of it operation, where two SIMD registers $\boldsymbol{A}$ and $\boldsymbol{B}$ are compared and the boolean result is stored in $\boldsymbol{Y}$. The `VMAX` instruction also performs a similar function but instead returns the greater value of the two being compared.

$$\boldsymbol{Y_p} = \begin{cases} \texttt{0xFFFFFFFF} & \text{if } \boldsymbol{A_p} > \boldsymbol{B_p} \\ \boldsymbol{0} & \text{if } \boldsymbol{A_p} \leq \boldsymbol{B_p} \end{cases} \tag{5.74}$$

Hence by utilizing the `VCGT` instruction, the branch can be effectively removed by using it in conjunction with the logical bit wise `AND` operation. The results for both the paths are calculated and is finally AND-ed with the result of the comparison to obtain the peak. The largest peak can be calculated by using the `VMAX` instruction. Let the input samples be represented as $\boldsymbol{x_{(c,n)}}$ where $\boldsymbol{c}$ is the channel number and $\boldsymbol{n}$ is the sample number in the buffer. The peak information $\boldsymbol{z1}$ and $\boldsymbol{z2}$ for each channel is represented as $\boldsymbol{z1_c}$ and $\boldsymbol{z2_c}$ respectively. A branch less version of the peak program meter which computes the peaks for each channel is shown in Algorithm 7.

Algorithm 7: Branch less Peak Program Meter Algorithm:

for($\boldsymbol{n}$=0; $\boldsymbol{n}$<$\boldsymbol{N}$; $\boldsymbol{n}$++)
{
$\boldsymbol{z1_c} = \boldsymbol{z1_c} * \boldsymbol{RC}$
$\boldsymbol{z2_c} = \boldsymbol{z2_c} * \boldsymbol{RC}$
$\boldsymbol{z1_{comp}} = |\boldsymbol{x_{(c,n)}}| > \boldsymbol{z1_c}$
$\boldsymbol{z2_{comp}} = |\boldsymbol{x_{(c,n)}}| > \boldsymbol{z2_c}$
$\boldsymbol{z1_{temp}} = \boldsymbol{z1_c} + \boldsymbol{AC_1}*(|\boldsymbol{x_{(c,n)}}|\text{-}\boldsymbol{z1_c})$
$\boldsymbol{z2_{temp}} = \boldsymbol{z2_c} + \boldsymbol{AC_2}*(|\boldsymbol{x_{(c,n)}}|\text{-}\boldsymbol{z2_c})$
$\boldsymbol{z1_c} = \boldsymbol{z1_c} + (\boldsymbol{z1_{comp}} \ \& \ \boldsymbol{z1_{temp}})$
$\boldsymbol{z2_c} = \boldsymbol{z2_c} + (\boldsymbol{z2_{comp}} \ \& \ \boldsymbol{z2_{temp}})$
$\boldsymbol{Z_{largest}} = \texttt{MAX}(\boldsymbol{Z_{largest}}, \boldsymbol{z1_c} + \boldsymbol{z2_c})$
}

If $|\boldsymbol{x_n}| > \boldsymbol{z_1}$, $\boldsymbol{z1_{comp}}$ will be filled with ones and $\boldsymbol{z_1}$ will be added with $\boldsymbol{z1_{temp}}$. If $|\boldsymbol{x_n}| < \boldsymbol{z_1}$, $\boldsymbol{z1_{comp}}$ will be filled with zeroes and consequently, so will $\boldsymbol{z1_{temp}}$. So $\boldsymbol{z_1}$ retains its value in case the amplitude is lesser than it. The same case applies to $\boldsymbol{z_2}$. Removing branches now introduces 10 floating point calculations to every iteration as compared to that of the sequential implementation (for cases where the branch comparison failed). This can be detrimental to speedup and is evaluated in

the next subsection.

However, since the branches have been removed, all samples go through the same number of operations until the final output is calculated. For example, if sample $|\boldsymbol{x_{1,n}}|$ in channel $\boldsymbol{C_1}$ has an amplitude greater than $\boldsymbol{z1_1}$ whereas $|\boldsymbol{x_{(2,n)}}|$ of channel $\boldsymbol{C_2}$ does not, both will go through the same number operations. This allows the possibility to vectorize multiple channels, which would not have been possible if data from one channel goes through lesser number of operations than the others in the vector. Four output samples for four respective channels can be computed per iteration. Such a representation of a sample number $\boldsymbol{n}$ across multiple channels is referred to as an **audio frame**. The function is divided into four parts, `INIT` `CHANNELSTART`, `COMPUTE_F`, AND `CHANNELEND`.

SIMD Register Denotations

- Four input quadword registers $\boldsymbol{F_1}$, $\boldsymbol{F_2}$, $\boldsymbol{F_3}$ and $\boldsymbol{F_4}$, representing input audio frame 1,2,3 and 4 respectively.

- Quadword registers $\boldsymbol{Z_1}$ and $\boldsymbol{Z_2}$, which holds the $\boldsymbol{z1}$ and $\boldsymbol{z2}$ values for each channel.

- Quadword register $\boldsymbol{Z_L}$ for holding the largest peak information.

- Four temporary registers $\boldsymbol{T_1}$, $\boldsymbol{T_2}$, $\boldsymbol{T_3}$ and $\boldsymbol{T_4}$to move around data and increase instruction level parallelism.

INIT

In this section, the respective registers containing the pointers to the data are set up. Also the constants $\boldsymbol{AC_1}$, $\boldsymbol{AC_2}$ and $\boldsymbol{RC}$ are loaded into the SIMD register here.

CHANNELSTART

This section deals with the loading of the data from the channels. Input samples from four channels are loaded into their respective registers as shown in Eqs. 5.77, 5.78, 5.79 and 5.80. This arrangement is achieved by using the `VLD4.32` instruction which loads four samples and places them in intervals of 4 lanes in the registers, effectively placing each sample from a channel in a separate quadword register. The previous peaks of the four channels are loaded into $\boldsymbol{Z_1}$ and $\boldsymbol{Z_2}$ as shown in Eqs.

5.75 and 5.76.

$$\boldsymbol{Z_1} = [\boldsymbol{z1_c}, \boldsymbol{z1_{(c+1)}}, \boldsymbol{z1_{(c+2)}}, \boldsymbol{z1_{(c+3)}}] \tag{5.75}$$
$$\boldsymbol{Z_2} = [\boldsymbol{z2_c}, \boldsymbol{z2_{(c+1)}}, \boldsymbol{z2_{(c+2)}}, \boldsymbol{z2_{(c+3)}}] \tag{5.76}$$

$$\boldsymbol{F_1} = [\boldsymbol{x_{(c,0)}}, \boldsymbol{x_{(c+1,0)}}, \boldsymbol{x_{(c+2,0)}}, \boldsymbol{x_{(c+3,0)}}] \tag{5.77}$$
$$\boldsymbol{F_2} = [\boldsymbol{x_{(c,1)}}, \boldsymbol{x_{(c+1,1)}}, \boldsymbol{x_{(c+2,1)}}, \boldsymbol{x_{(c+3,1)}}] \tag{5.78}$$
$$\boldsymbol{F_3} = [\boldsymbol{x_{(c,2)}}, \boldsymbol{x_{(c+1,2)}}, \boldsymbol{x_{(c+2,2)}}, \boldsymbol{x_{(c+3,2)}}] \tag{5.79}$$
$$\boldsymbol{F_4} = [\boldsymbol{x_{(c,3)}}, \boldsymbol{x_{(c+1,3)}}, \boldsymbol{x_{(c+2,3)}}, \boldsymbol{x_{(c+3,3)}}] \tag{5.80}$$

COMPUTE_F

The goal is to compute the output of four audio frames of 4 channels. In this section, the peaks for audio frame $\boldsymbol{F_f}$ as well as the largest peak is calculated, where $\boldsymbol{f}$ represents the audio frame. This section iterates $\boldsymbol{N}$ times, which represents its buffer size. The algorithm for the audio frame $\boldsymbol{F_1}$ is shown below and has identical structure to the other audio frames $\boldsymbol{F_2}$, $\boldsymbol{F_3}$ and $\boldsymbol{F_4}$, the only difference being the input audio frame register. To minimize both hardware and data dependencies, certain instructions from the consecutive audio frame computation are placed in the current computation. Equations 5.81 to 5.92 represent the computation of the output of the audio frame. The only instructions which are independent between the computation of the frames are the **absolute** (`ABS`) instruction and the load and store and they can be scheduled to favor more parallelism. $\boldsymbol{F_4}$ loads the next frames into memory while it is performing its calculation.

$$\boldsymbol{Z_1} = \boldsymbol{Z_1} \odot \boldsymbol{RC} \tag{5.81}$$
$$\boldsymbol{Z_2} = \boldsymbol{Z_2} \odot \boldsymbol{RC} \tag{5.82}$$
$$\boldsymbol{T_1} = |\boldsymbol{f}| \ominus \boldsymbol{Z_1} \tag{5.83}$$
$$\boldsymbol{T_2} = |\boldsymbol{f}| \ominus \boldsymbol{Z_2} \tag{5.84}$$
$$\boldsymbol{T_3} = \texttt{VCGT.F32}(|\boldsymbol{f}|, \boldsymbol{Z_1}) \tag{5.85}$$
$$\boldsymbol{T_4} = \texttt{VCGT.F32}(|\boldsymbol{f}|, \boldsymbol{Z_2}) \tag{5.86}$$
$$\boldsymbol{T_1} = \boldsymbol{T_1} \odot \boldsymbol{AC_1} \tag{5.87}$$
$$\boldsymbol{T_2} = \boldsymbol{T_2} \odot \boldsymbol{AC_2} \tag{5.88}$$
$$\boldsymbol{T_3} = \boldsymbol{T_1} \& \boldsymbol{T_3} \tag{5.89}$$
$$\boldsymbol{T_4} = \boldsymbol{T_2} \& \boldsymbol{T_4} \tag{5.90}$$
$$\boldsymbol{Z_1} = \boldsymbol{Z_1} \oplus \boldsymbol{T_3} \tag{5.91}$$
$$\boldsymbol{Z_2} = \boldsymbol{Z_2} \oplus \boldsymbol{T_4} \tag{5.92}$$
$$\boldsymbol{Z_L} = \texttt{VMAX.F32}(\boldsymbol{Z_L}, \boldsymbol{Z_1} \oplus \boldsymbol{Z_2}) \tag{5.93}$$

CHANNELEND

In this section, after the computation of the audio frames, the results are stored here.

Algorithm of SIMD Peak Program Meter Function

The algorithm for the SIMD implementation is as shown below. There are two main loops, one which iterates over the buffer size and one which iterates over the number of channels. It was decided that four audio frames be computed per iteration because of lack of extra registers.

Algorithm 8: Algorithm of SIMD Peak Program Meter Function for number of channels ≥ 4

```
INIT
CHANNELSTART
    channel_size = channel_size - 4
    buffer_size = N //reset buffer size
    LOOP
        buffer_size = buffer_size - 4
        COMPUTE_F1
        COMPUTE_F2
        COMPUTE_F3
        COMPUTE_F4
        If (buffer_size > 0)
            GOTO LOOP
    CHANNELEND
    STORE Z_L, Z_1 and Z_2
    If (channel_size > 0)
        GOTO CHANNELSTART
    END
```

5.6.2 Results

To test both the auto-vectorized and the SIMD versions, 4 channel audio data with 48000 samples in each channel was fed in to the Peak Program Meter. It was configured to process buffer sizes in chunks of 64 and the timing measurements were measured over how long it would take to process data of 64 samples. This process was repeated 1333 times to roughly get one million iterations. Fig. 5.5 show the results of the execution time measurements. It can be seen that for the SIMD implementation, the execution time ranges from 1299 to 2100 clock cycles for 99% of the iterations. But there is a significant distribution (at least few hundred iterations) of clock cycles at the 2050 to 2250 clock cycles. This variation can be attributed to cache misses. Due to the fact that the vectorized implementation accesses data from multiple channels at once and the size of the test data is big, there may be some L1 cache misses. The auto-vectorized implementation has a lot of variance and ranges from 3540 to 9150 clock cycles. This variance is mostly caused by the branching; audio buffers which do not have many peaks gets processed faster than those which have frequent peaks. The speed up achieved is 5.11 as shown in table 5.5 where the other results are also tabulated.

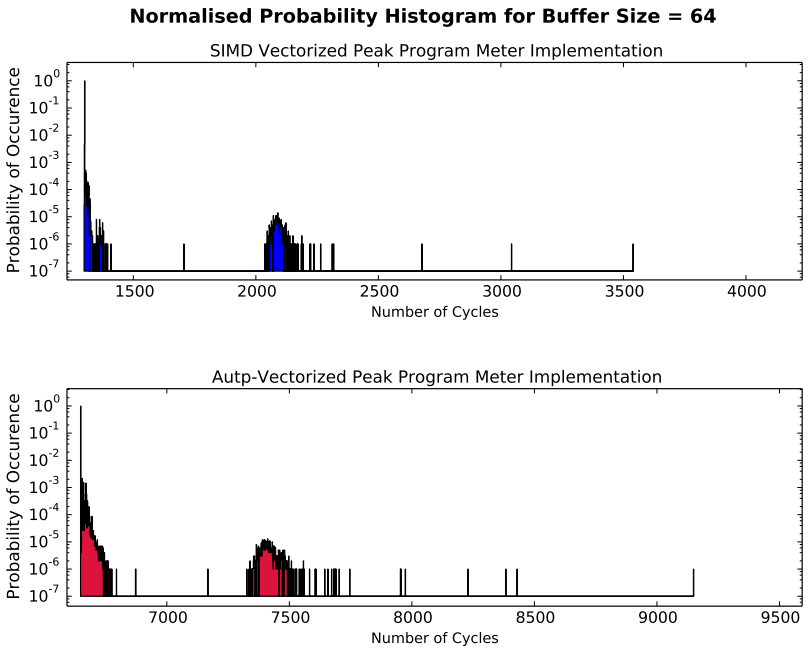

Figure 5.5: Normalized probability histograms for the SIMD Vectorized implementation and auto-vectorized implementation of the Peak Meter Algorithm algorithm.

Table 5.5: Peak program meter algorithm execution time results

Performance Metric	SIMD Vectorized Peak Program Meter Implementation	Auto-Vectorized Peak Program Meter Implementation
Minimum execution time (clock cycles)	1299	6648
Maximum execution time (clock cycles)	3540	9150
Median execution time (clock cycles)	1301	6648
Speedup as compared with auto-vectorized code	**5.11**	
Confidence Rate (1-α)	99%	

5.7 Four Band Equalizer with Cascaded Biquad Filters

The sequential implementation of this algorithm has quite heavier computational intensity compared to the other functions explored in this thesis, due to its recursive nature. The requirement was four biquad filters in a cascaded form as a part of a four band equalizer. As vectorization of biquads is difficult, a technique of converting the cascaded biquads to a parallel form to favor vectorization is described below. This technique will allow the possibility of vectorizing multiple filters at once rather than multiple input samples or multiple coefficients. Also described in this section is a comparison of how it performs against the vectorization method proposed by R. Kutil et al. (2008)[36].

5.7.1 Initial State Coefficients

Consider a biquad filter which is applied to a buffer of size N where each sample is denoted as x_n where $0 \leq n \leq N-1$. The general Direct Form I equation of a biquad filter is as shown in Eq. 5.94 below.

$$y_{(n)} = b_0.x_{(n)} + b_1.x_{(n-1)} + b_2.x_{(n-2)} + a_1.y_{(n-1)} + a_2.y_{(n-2)} \tag{5.94}$$

$$y_{(0)} = b_0.x_{(0)} + b_1.x_{(-1)} + b_2.x_{(-2)} + a_1.y_{(-1)} + a_2.y_{(-2)} \tag{5.95}$$

$$y_{(1)} = b_0.(x_1) + b_1.(x_0) + b_2.x_{(-1)} + a_1.y_{(0)} + a_2.y_{(-1)} \tag{5.96}$$

For correct functionality, it requires the z^{-1} and z^{-2} delay elements of X and Y to be stored in the memory in order to compute the first and second output of the next buffer as shown in Eqs. 5.95 and 5.96. This is usually referred to as **Initial State Coefficients** of the filter. In other words, after the outputs of the filter have been calculated for the whole buffer, the values $x_{(N-1)}$, $x_{(N-2)}$, $y_{(N-1)}$ and $y_{(N-2)}$ have to be stored in memory. Hence, each biquad filter requires four values to be stored in the memory, which is inefficient when multiple biquad filters are

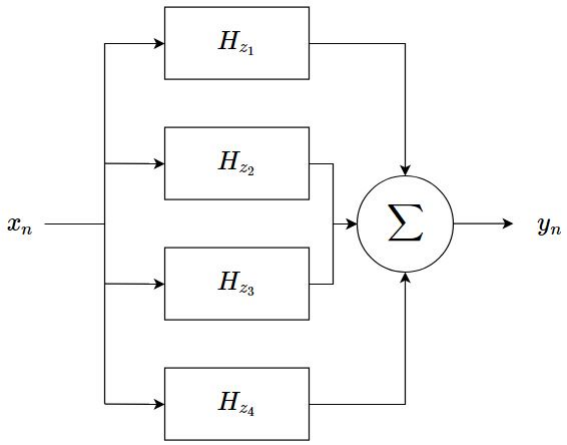

Figure 5.6: Parallel realization comprised of four second order sections. The final response is obtained by summing the responses from the individual filters.

cascaded; in this case 16 samples have to be stored in the memory which impairs performance. To mitigate this, a different approach is used as shown in Eqs. 5.97 and 5.98.

$$\boldsymbol{s_1 = b_1.x_{(N-1)} + b_2.x_{(N-2)} + a_1.y_{(N-1)} + a_1.y_{(N-2)}} \tag{5.97}$$

$$\boldsymbol{s_2 = b_2.x_{(N-1)} + a_2.y_{(N-1)}} \tag{5.98}$$

$$\boldsymbol{y_{(N)} = b_0.x_{(N)} + s_1} \tag{5.99}$$

$$\boldsymbol{y_{(N+1)} = b_0.x_{(N+1)} + b_1.x_{(N)} + a_1.y_{(N)} + s_2} \tag{5.100}$$

The computation dealing with the delay elements can be performed at the end of the computation of the previous buffer to obtain the new initial state coefficients $\boldsymbol{s_1}$ and $\boldsymbol{s_2}$. Using these, the first two output samples for the next buffer can be calculated as shown in 5.99 and 5.100. This reduces by half the amount of initial state coefficients that have to be stored.

5.7.2 Cascade To Parallel

A parallel realization is as shown in Fig. 5.6[33]. The final response of a parallel filter is obtained by summing up the responses of the individual filters as shown in Eq. 5.101[33]. The goal is to convert the cascade realization to the parallel one such that they have the same filter response.

$$\boldsymbol{H_{par} = H_{z_1} + H_{z_2} + H_{z_3} + H_{z_4} = H_{casc}} \tag{5.101}$$

The transfer function of the cascade realization represented in z transform is as shown in Eq. 5.103.

$$H_{casc} = H_{z_1} * H_{z_2} * H_{z_3} * H_{z_4} \tag{5.102}$$

$$\begin{aligned}\frac{Y(z)}{X(z)} &= \left(k_{H_{z_1}} * \frac{b_{(0,H_{z_1})} + b_{(1,H_{z_1})}z^{-1} + b_{(2,H_{z_1})}z^{-2}}{1 + a_{(1,H_{z_1})}z^{-1} + a_{(2,H_{z_1})}z^{-2}}\right) \\ &* \left(k_{H_{z_2}} * \frac{b_{(0,H_{z_2})} + b_{(1,H_{z_2})}z^{-1} + b_{(2,H_{z_2})}z^{-2}}{1 + a_{(1,H_{z_2})}z^{-1} + a_{(2,H_{z_2})}z^{-2}}\right) \\ &* \left(k_{H_{z_3}} * \frac{b_{(0,H_{z_3})} + b_{(1,H_{z_3})}z^{-1} + b_{(2,H_{z_3})}z^{-2}}{1 + a_{(1,H_{z_3})}z^{-1} + a_{(2,H_{z_3})}z^{-2}}\right) \\ &* \left(k_{H_{z_4}} * \frac{b_{(0,H_{z_4})} + b_{(1,H_{z_4})}z^{-1} + b_{(2,H_{z_4})}z^{-2}}{1 + a_{(1,H_{z_4})}z^{-1} + a_{(2,H_{z_4})}z^{-2}}\right)\end{aligned} \tag{5.103}$$

These second order sections can be combined to form an eighth order equation as shown in Eq. 5.104.

$$H_{casc} = \hat{k} * \frac{\hat{b_0} + \hat{b_1}z^{-1} + \hat{b_2}z^{-2} + ... + \hat{b_8}z^{-8}}{1 + \hat{a_1}z^{-1} + \hat{a_2}z^{-2} + ... + \hat{a_8}z^{-8}} \tag{5.104}$$

Using **partial fraction expansion**[33], higher order equations which have $M = N$ can be split into a sum of complex single pole filters in parallel with an FIR part as shown in Eq. 5.105[33] where p_i are the complex poles and r_i are the residues of the poles[33].

$$H_{casc} = F(z) + z^{-1}\sum_{i=1}^{8} k_i \frac{r_i}{1 - p_i z^{-1}} \tag{5.105}$$

The parallel FIR part can be obtained through long division. For **linear time invariant** filters with real distinct coefficients, the complex poles always occur in conjugate pairs[33]. These pairs can then be combined to form real second order sections. The result is four new second order sections with different gain and coefficients. Equation 5.104 after partial fraction expansion can be expressed as:

$$H_{casc} = H^p_{z_1} + H^p_{z_2} + H^p_{z_3} + H^p_{z_4} = H_{par} \tag{5.106}$$

5.7.3 Architectural Optimization And Implementation Of The Algorithm

Since the filters' coefficients have been transformed to achieve the same response in a parallel form, each SIMD vector can be dedicated to calculate the response of each filter's coefficient. The computation can be easily pipelined as shown in Fig. 5.7 below. As an input sample x_n in pipeline stage 1 moves to the next stage in the pipeline, it represents $x_{(n-1)}$ for the current sample in stage 1. In other

words, the inherent instruction level parallelism can be exploited as it contains lot of independent operations. This technique will be optimal if the number of biquad filters in parallel are equal to or a multiple of $\boldsymbol{P}$, the vector size of the SIMD unit. In the case of the thesis, as per the requirement, four filters were used but the NEON SIMD unit has enough registers to handle eight.

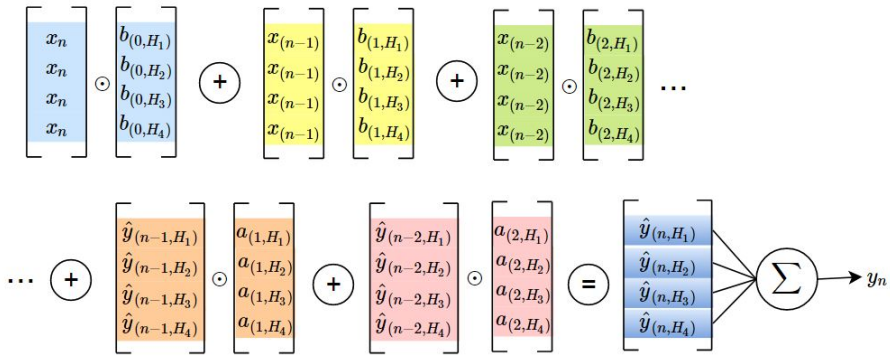

Figure 5.7: Visual representation of computing the response of four parallel biquad filters using vectorization.

Signal and Filter Denotations

To illustrate the algorithm of the SIMD implementation, the following denotations are used:

- The input signal $\boldsymbol{X}$ has a buffer size $\boldsymbol{N}$ and each sample is denoted as $\boldsymbol{x_n}$.
- The four parallel biquad filters are denoted as $\boldsymbol{H_{z_h}}$ where $\boldsymbol{1 \leq h \leq 4}$.
- An output sample from each filter is denoted as $\boldsymbol{\hat{y}_{(n,H_{z_1})}}$, $\boldsymbol{\hat{y}_{(n,H_{z_2})}}$, $\boldsymbol{\hat{y}_{(n,H_{z_3})}}$ and $\boldsymbol{\hat{y}_{(n,H_{z_4})}}$. They are summed to get the final output sample $\boldsymbol{y_n}$ which makes up the output signal $\boldsymbol{Y}$.
- The FIR coefficients are represented as $\boldsymbol{b_{(i,H_{z_1})}}$, $\boldsymbol{b_{(i,H_{z_2})}}$, $\boldsymbol{b_{(i,H_{z_3})}}$ and $\boldsymbol{b_{(i,H_{z_4})}}$, where $\boldsymbol{0 \leq i \leq 2}$.
- The IIR coefficients as $\boldsymbol{a_{(j,H_{z_1})}}$, $\boldsymbol{a_{(j,H_{z_2})}}$, $\boldsymbol{a_{(j,H_{z_3})}}$ and $\boldsymbol{a_{(j,H_{z_4})}}$, where $\boldsymbol{1 \leq j \leq 2}$.

SIMD Register Denotations

The following SIMD quadword registers are used for the coefficients, input samples, output samples and initial state coefficients:

- $\boldsymbol{B_0}$: for coefficients $\boldsymbol{b_{(0,H_{z_1})}}$, $\boldsymbol{b_{(0,H_{z_2})}}$, $\boldsymbol{b_{(0,H_{z_3})}}$ and $\boldsymbol{b_{(0,H_{z_4})}}$.
- $\boldsymbol{B_1}$: for coefficients $\boldsymbol{b_{(1,H_{z_1})}}$, $\boldsymbol{b_{(1,H_{z_2})}}$, $\boldsymbol{b_{(1,H_{z_3})}}$ and $\boldsymbol{b_{(1,H_{z_4})}}$.

- $\boldsymbol{B_2}$: for coefficients $\boldsymbol{b_{(2,H_{z_1})}}$, $\boldsymbol{b_{(2,H_{z_2})}}$, $\boldsymbol{b_{(2,H_{z_3})}}$ and $\boldsymbol{b_{(2,H_{z_4})}}$.
- $\boldsymbol{A_1}$: for coefficients $\boldsymbol{a_{(1,H_{z_1})}}$, $\boldsymbol{a_{(1,H_{z_2})}}$, $\boldsymbol{a_{(1,H_{z_3})}}$ and $\boldsymbol{a_{(1,H_{z_4})}}$.
- $\boldsymbol{A_2}$: for coefficients $\boldsymbol{a_{(2,H_{z_1})}}$, $\boldsymbol{a_{(2,H_{z_2})}}$, $\boldsymbol{a_{(2,H_{z_3})}}$ and $\boldsymbol{a_{(2,H_{z_4})}}$
- $\boldsymbol{I_1}$ and $\boldsymbol{I_2}$: for the input samples. Each lane is denoted as $\boldsymbol{I_{1_{(p)}}}$ and $\boldsymbol{I_{2_{(p)}}}$ where $\boldsymbol{0 \leq p \leq 3}$.
- $\boldsymbol{S_1}$: quadword register which contains the $\boldsymbol{s_1}$ initial state coefficient for each biquad filter. Each lane is represented as $\boldsymbol{S_{1_{(p)}}}$ where $\boldsymbol{0 \leq p \leq 3}$ and contains the coefficients $\boldsymbol{s_{(1,H_{z_1})}}$, $\boldsymbol{s_{(1,H_{z_2})}}$, $\boldsymbol{s_{(1,H_{z_3})}}$ and $\boldsymbol{s_{(1,H_{z_4})}}$.
- $\boldsymbol{S_2}$: quadword register which contains the $\boldsymbol{s_2}$ initial state coefficient for each biquad filter. Each lane is represented as $\boldsymbol{S_{2_{(p)}}}$ where $\boldsymbol{0 \leq p \leq 3}$ and contains the coefficients $\boldsymbol{s_{(2,H_{z_1})}}$, $\boldsymbol{s_{(2,H_{z_2})}}$, $\boldsymbol{s_{(2,H_{z_3})}}$ and $\boldsymbol{s_{(2,H_{z_4})}}$.
- $\boldsymbol{\hat{Y}}$: quadword register to contain the output samples from each of filter. Each lane in this vector contains $\boldsymbol{\hat{y}_{(n,H_{z_1})}}$, $\boldsymbol{\hat{y}_{(n,H_{z_2})}}$, $\boldsymbol{\hat{y}_{(n,H_{z_3})}}$ and $\boldsymbol{\hat{y}_{(n,H_{z_4})}}$ and is denoted by $\boldsymbol{\hat{Y}_p}$ where $\boldsymbol{0 \leq p \leq 3}$.
- Temporary quadword registers $\boldsymbol{T_1}$, $\boldsymbol{T_2}$, $\boldsymbol{T_3}$, $\boldsymbol{T_4}$ and $\boldsymbol{T_5}$ to help aide in the computation.
- $\boldsymbol{O}$: Registers to hold 8 output samples. Each lane is represented as $\boldsymbol{O_p}$ where $\boldsymbol{0 \leq p \leq 3}$.

Straightforward Implementation

A straightforward vectorized implementation is as shown in Eqs. 5.107 to 5.138. It follows a very similar structure as that of the sequential implementation. The response of each coefficient is calculated and the result is carried forward to the next step. First the registers are loaded with their corresponding values as shown in Eqs. 5.107 to 5.114.

$$\boldsymbol{B_0 = [b_{(0,H_{z_1})}, b_{(0,H_{z_2})}, b_{(0,H_{z_3})}, b_{(0,H_{z_4})}]} \tag{5.107}$$

$$\boldsymbol{B_1 = [b_{(1,H_{z_1})}, b_{(1,H_{z_2})}, b_{(1,H_{z_3})}, b_{(1,H_{z_4})}]} \tag{5.108}$$

$$\boldsymbol{B_2 = [b_{(2,H_{z_1})}, b_{(2,H_{z_2})}, b_{(2,H_{z_3})}, b_{(2,H_{z_4})}]} \tag{5.109}$$

$$\boldsymbol{A_1 = [a_{(1,H_{z_1})}, a_{(1,H_{z_2})}, a_{(1,H_{z_3})}, a_{(1,H_{z_4})}]} \tag{5.110}$$

$$\boldsymbol{A_2 = [a_{(2,H_{z_1})}, a_{(2,H_{z_2})}, a_{(2,H_{z_3})}, a_{(2,H_{z_4})}]} \tag{5.111}$$

$$\boldsymbol{I_1 = [x_n, x_{(n+1)}, x_{(n+2)}, x_{(n+3)}]} \tag{5.112}$$

$$\boldsymbol{S_1 = [s_{(1,H_{z_1})}, s_{(1,H_{z_2})}, s_{(1,H_{z_3})}, s_{(1,H_{z_4})}]} \tag{5.113}$$

$$\boldsymbol{S_2 = [s_{(2,H_{z_1})}, s_{(2,H_{z_2})}, s_{(2,H_{z_3})}, s_{(2,H_{z_4})}]} \tag{5.114}$$

The first two outputs $\boldsymbol{y_0}$ and $\boldsymbol{y_1}$ can be calculated using the $\boldsymbol{s_1}$ and $\boldsymbol{s_2}$ coefficients as shown in Eqs. 5.115 to 5.122. Register $\boldsymbol{T_1}$ is used to temporarily store the outputs obtained in Eq. 5.115 and is used in Eq. 5.120. It now represents the $\boldsymbol{z^{-1}}$ output elements of each filter needed to compute $\boldsymbol{y_1}$. Similarly the outputs $\boldsymbol{\hat{y}_{(1,H_{z_1})}}$, $\boldsymbol{\hat{y}_{(1,H_{z_2})}}$, $\boldsymbol{\hat{y}_{(1,H_{z_3})}}$ and $\boldsymbol{\hat{y}_{(1,H_{z_4})}}$ which is needed for $\boldsymbol{y_2}$ are stored in $\boldsymbol{T_1}$ as shown in Eq. 5.123.

$$\boldsymbol{\hat{Y} = S_1 \oplus (B_0 \odot x_0)} \tag{5.115}$$

$$\begin{aligned}\boldsymbol{y_0} &= \boldsymbol{\hat{y}_{(0,H_{z_1})} + \hat{y}_{(0,H_{z_2})} + \hat{y}_{(0,H_{z_3})} + \hat{y}_{(0,H_{z_4})}} \\ &= \boldsymbol{\hat{Y}_0 + \hat{Y}_1 + \hat{Y}_2 + \hat{Y}_3}\end{aligned} \tag{5.116}$$

$$\boldsymbol{T_1 = \hat{Y}} \tag{5.117}$$

$$\boldsymbol{\hat{Y} = B_0 \odot x_1} \tag{5.118}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \oplus (B_1 \odot x_0)} \tag{5.119}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \ominus (A_1 \odot T_0)} \tag{5.120}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \oplus S_2} \tag{5.121}$$

$$\begin{aligned}\boldsymbol{y_1} &= \boldsymbol{\hat{y}_{(1,H_{z_1})} + \hat{y}_{(1,H_{z_2})} + \hat{y}_{(1,H_{z_3})} + \hat{y}_{(1,H_{z_4})}} \\ &= \boldsymbol{\hat{Y}_0 + \hat{Y}_1 + \hat{Y}_2 + \hat{Y}_3}\end{aligned} \tag{5.122}$$

$$\boldsymbol{T_1 = \hat{Y}} \tag{5.123}$$

Now the remaining output samples can be calculated in the same fashion as Eq. 5.97 and is shown below in Eq. 5.124 to 5.130.

$$\boldsymbol{\hat{Y} = B_0 \odot x_n} \tag{5.124}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \oplus (B_1 \odot x_n)} \tag{5.125}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \oplus (B_2 \odot x_n)} \tag{5.126}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \ominus (A_1 \odot T_1)} \tag{5.127}$$

$$\boldsymbol{\hat{Y} = \hat{Y} \ominus (A_2 \odot T_1)} \tag{5.128}$$

$$\begin{aligned}\boldsymbol{y_n} &= \boldsymbol{\hat{y}_{(n,H_{z_1})} + \hat{y}_{(n,H_{z_2})} + \hat{y}_{(n,H_{z_3})} + \hat{y}_{(n,H_{z_4})}} \\ &= \boldsymbol{\hat{Y}_0 + \hat{Y}_1 + \hat{Y}_2 + \hat{Y}_3}\end{aligned} \tag{5.129}$$

$$\boldsymbol{T_1 = \hat{Y}} \tag{5.130}$$

After the last output sample $\boldsymbol{y_{(N-1)}}$ of the buffer is computed, the initial state coefficients $\boldsymbol{s_1}$ and $\boldsymbol{s_2}$ of each filter are calculated and stored in memory. To achieve this, the $\boldsymbol{z^{-1}}$ and $\boldsymbol{z^{-2}}$ output delay elements are stored in temporary quadword

registers $\boldsymbol{T_1}$ and $\boldsymbol{T_2}$ respectively.

$$\boldsymbol{T_1} = [\hat{\boldsymbol{y}}_{(\boldsymbol{N-1},\boldsymbol{H_{z_1}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-1},\boldsymbol{H_{z_2}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-1},\boldsymbol{H_{z_3}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-1},\boldsymbol{H_{z_4}})}] \tag{5.131}$$

$$\boldsymbol{T_2} = [\hat{\boldsymbol{y}}_{(\boldsymbol{N-2},\boldsymbol{H_{z_1}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-2},\boldsymbol{H_{z_2}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-2},\boldsymbol{H_{z_3}})}, \hat{\boldsymbol{y}}_{(\boldsymbol{N-2},\boldsymbol{H_{z_4}})}] \tag{5.132}$$

$$\boldsymbol{S_1} = \boldsymbol{B_1} \odot \boldsymbol{x}_{(\boldsymbol{N-1})} \tag{5.133}$$

$$\boldsymbol{S_1} = \boldsymbol{S_1} \oplus (\boldsymbol{B_2} \odot \boldsymbol{x}_{(\boldsymbol{N-2})}) \tag{5.134}$$

$$\boldsymbol{S_1} = \boldsymbol{S_1} \ominus (\boldsymbol{A_1} \odot \boldsymbol{T_1}) \tag{5.135}$$

$$\boldsymbol{S_1} = \boldsymbol{S_1} \ominus (\boldsymbol{A_2} \odot \boldsymbol{T_2}) \tag{5.136}$$

$$\boldsymbol{S_2} = \boldsymbol{B_2} \odot \boldsymbol{x}_{(\boldsymbol{N-1})} \tag{5.137}$$

$$\boldsymbol{S_2} = \boldsymbol{S_2} \ominus (\boldsymbol{A_1} \odot \boldsymbol{T_1}) \tag{5.138}$$

However, almost every instruction is dependent on the result of the previous operation. There is also possibility of pipeline stalls due to hardware unit being unavailable. This is detrimental to performance. Furthermore NEON instruction set does not have an instruction to sum all the lanes in a quadword vector which is needed by Eqs. 5.116, 5.122 and 5.129. To improve performance, multiple computations which are independent of each other can be scheduled. The computations for the FIR part are independent in nature. For example, after scheduling the computation to obtain $\boldsymbol{B_1} \odot \boldsymbol{x}_{(\boldsymbol{n-1})}$, the computation $\boldsymbol{B_1} \odot \boldsymbol{x_n}$ can also be scheduled together as the result will be used in the later stage. In this manner, the technique presented below borrows a lot from the techniques used by R.Kutil et al. (2008) to vectorize the FIR part and incorporates them in the instruction scheduling methodology. This is to maximize the throughput of the SIMD unit by scheduling instructions whose result will be used much later in the program's code, which, in this case, are the responses of the FIR part. The parallelism can be increased by loading more samples from the input. Since there are many possibilities of arranging instructions, a trial and error based approach was taken to find a near optimal solution.

SIMD Implementation

The ARM SIMD instruction set does not provide semantics for adding up all the the lanes in a single vector. This operation is needed for adding up the responses of the individual filters in register $\hat{\boldsymbol{Y}}$. `VPADD` is the closest instruction available which can add adjacent pairs of a quadword. Hence, using `VPADD` the operations in Eqs. 5.122 to 5.129 can be described by the following equation.

`VPADD` a quadword:

$$\begin{aligned}[\boldsymbol{O_0}, \boldsymbol{O_1}] &= [(\hat{\boldsymbol{Y}}_0 + \hat{\boldsymbol{Y}}_1), (\hat{\boldsymbol{Y}}_2 + \hat{\boldsymbol{Y}}_3)] \\ &= [(\hat{\boldsymbol{y}}_{(\boldsymbol{n},\boldsymbol{H_{z_1}})} + \hat{\boldsymbol{y}}_{(\boldsymbol{n},\boldsymbol{H_{z_2}})}), (\hat{\boldsymbol{y}}_{(\boldsymbol{n},\boldsymbol{H_{z_3}})} + \hat{\boldsymbol{y}}_{(\boldsymbol{n},\boldsymbol{H_{z_4}})}]\end{aligned} \tag{5.139}$$

The following algorithm works on 8 input samples per iteration. First, The SIMD implementation of the four parallel biquad filters are divided into five parts, `INIT`,

`COMPUTE_A`$(\boldsymbol{x_n})$, `COMPUTE_B`$(\boldsymbol{x_n})$ and `ENDLOOP`.

INIT

This section deals with the initial operations such as loading of samples and coefficients from memory and computing values which will be used by the later sections. It is described as follows:

1. First the input register $\boldsymbol{I_1}$ is loaded with the first four input samples and the quadword registers $\boldsymbol{B_0}$, $\boldsymbol{B_1}$, $\boldsymbol{B_2}$ and $\boldsymbol{A_1}$ are loaded with the respective coefficients as shown in Eqs. 5.140 to 5.144.

$$\boldsymbol{I_1} = [\boldsymbol{x_0}, \boldsymbol{x_1}, \boldsymbol{x_2}, \boldsymbol{x_3}] \tag{5.140}$$
$$\boldsymbol{B_0} = [\boldsymbol{b_{(0,H_{z_1})}}, \boldsymbol{b_{(0,H_{z_2})}}, \boldsymbol{b_{(0,H_{z_3})}}, \boldsymbol{b_{(0,H_{z_4})}}] \tag{5.141}$$
$$\boldsymbol{B_1} = [\boldsymbol{b_{(1,H_{z_1})}}, \boldsymbol{b_{(1,H_{z_2})}}, \boldsymbol{b_{(1,H_{z_3})}}, \boldsymbol{b_{(1,H_{z_4})}}] \tag{5.142}$$
$$\boldsymbol{B_2} = [\boldsymbol{b_{(2,H_{z_1})}}, \boldsymbol{b_{(2,H_{z_2})}}, \boldsymbol{b_{(2,H_{z_3})}}, \boldsymbol{b_{(2,H_{z_4})}}] \tag{5.143}$$
$$\boldsymbol{A_1} = [\boldsymbol{a_{(1,H_{z_1})}}, \boldsymbol{a_{(1,H_{z_2})}}, \boldsymbol{a_{(1,H_{z_3})}}, \boldsymbol{a_{(1,H_{z_4})}}] \tag{5.144}$$

2. While this load operation is carried out, $\boldsymbol{B_0} \odot \boldsymbol{x_0}$ is computed as shown in Eq. 5.145. At that point in the code, both the input samples and $\boldsymbol{B_0}$ coefficients of each filter have been loaded into their registers. Hence this computation and the remaining load operations for $\boldsymbol{B_1}$, $\boldsymbol{B_2}$ and $\boldsymbol{A_1}$ are performed in parallel.

$$\boldsymbol{T_5} = \boldsymbol{B_0} \odot \boldsymbol{x_0} \tag{5.145}$$

3. Then the quadword registers $\boldsymbol{A_2}$ and $\boldsymbol{S_1}$ are loaded with their respective coefficients.

$$\boldsymbol{A_2} = [\boldsymbol{a_{(2,H_{z_1})}}, \boldsymbol{a_{(2,H_{z_2})}}, \boldsymbol{a_{(2,H_{z_3})}}, \boldsymbol{a_{(2,H_{z_4})}}] \tag{5.146}$$
$$\boldsymbol{S_1} = [\boldsymbol{s_{(1,H_{z_1})}}, \boldsymbol{s_{(1,H_{z_2})}}, \boldsymbol{s_{(1,H_{z_3})}}, \boldsymbol{s_{(1,H_{z_4})}}] \tag{5.147}$$

4. Following this is an independent operation where $\boldsymbol{B_2} \odot \boldsymbol{x_0}$ is computed. At this point in the code, all the FIR coefficients have been loaded from memory and so any one of them could be multiplied with any one of the input samples to achieve the goal of inserting an independent instruction. Through trial and error, the operation in Eq. 5.148 was chosen because the subsequent sections are designed around this particular scheduling method, as explained in the subsections below. The intention was that the result of this operation is available at the right point in the code down the line.

$$\boldsymbol{T_4} = \boldsymbol{B_2} \odot \boldsymbol{x_0} \tag{5.148}$$

5. The register $\boldsymbol{S_2}$ is loaded with the respective $\boldsymbol{s_2}$ coefficients.

$$\boldsymbol{S_2} = [\boldsymbol{s_{(2,H_{z_1})}}, \boldsymbol{s_{(2,H_{z_2})}}, \boldsymbol{s_{(2,H_{z_3})}}, \boldsymbol{s_{(2,H_{z_4})}}] \tag{5.149}$$

COMPUTE_(x_n)

The COMPUTE section carries out the main computation of the filter response. It works on 8 input samples $(x_n, x_{(n+1)}, x_{(n+2)} \ldots x_{(n+7)})$ and is iterated over the entire buffer. It is divided into two parts, COMPUTE_A(x_n) and COMPUTE_B(x_n). They perform the following operations.

- Compute output $\hat{y}_{(n,H_h)}$ for previous sample $x_{(n-1)}$
- s_1 coefficients for the $x_{(n+1)}$ sample.
- s_2 coefficients for the $x_{(n+1)}$ sample.

This is akin to a pipeline stage, where each computation section finishes the computation of the previous working sample, does partial work on the current one and passes it on to the next computation unit. The only difference between COMPUTE_A(x_n) and COMPUTE_B(x_n) are the temporary registers used.

COMPUTE_A(x_n)

$$T_2 = B_1 \odot x_n \tag{5.150}$$
$$\hat{Y} = S_1 \oplus T_5 \tag{5.151}$$
$$T_5 = B_0 \odot x_{(n+1)} \tag{5.152}$$
$$S_2 = S_2 + (A_1 \odot \hat{Y}) \tag{5.153}$$
$$S_1 = A_2 \odot \hat{Y} \tag{5.154}$$
$$[O_0, O_1] = [(\hat{Y}_{1_0} + \hat{Y}_{1_3}), (\hat{Y}_{1_1} + \hat{Y}_{1_4})] \tag{5.155}$$
$$T_3 = B_2 \odot x_{(n+1)} \tag{5.156}$$
$$S_1 = S_1 + T_4 \tag{5.157}$$
$$S_2 = S_2 + T_2 \tag{5.158}$$

1. B_1 is multiplied with the current sample x_n as shown in Eq. 5.150. This is to be used for computation of the next sample $x_{(n+1)}$.

2. T_5 contains the result of the operation $B_0 \odot x_n$ performed in the preceding computation section. It is added with the coefficients $s_{(1,H_h)}$ in the quadword register S_1 as shown in Eq. 5.151 to obtain the outputs of the individual filter $\hat{y}_{(n,H_h)}$. After this operation, T_5 can now hold the result of the operation $B_0 \odot x_{(n+1)}$ as shown in Eq. 5.152. It will be used in the succeeding computation section. In this manner, register T_5 is dedicated for these two operations.

3. At this point in the code, the result $\hat{y}_{(n,H_h)}$ of Eq. 5.151 will be available. This is used to calculate the $s_{(1,H_h)}$ coefficients of the filters for $x_{(n+1)}$. So the S_2 register is added with $A_1 \odot \hat{Y}$ using a MAC instruction. Now S_2 register contains the value as shown in Eq. 5.159.

$$S_2 = (B_2 \odot x_{(n-1)}) + (A_2 \odot \hat{y}_{(n-1,H_h)}) + (A_1 \odot \hat{y}_{(n,H_h)}) \tag{5.159}$$

4. The $s_{(2,H_h)}$ coefficients can now be calculated at this point for the sample $x_{(n+1)}$. The operation $A_2 \odot \hat{Y}$ is scheduled here. Since the S_2 register is used for the previous operation, the result is stored in S_1.

5. The adjacent pairs of the register $\hat{Y}$ is added using the `VPADD` instruction as shown in 5.155. The result is stored in the upper half of the quadword register O.

6. The operation $B_2 \odot x_{(n+1)}$ is performed and the result is stored in register T_3. This will be later used by the succeeding section **COMPUTE_B(x_n)**.

7. The computation to obtain $s_{(2,H_h)}$ coefficients for the sample $x_{(n+1)}$ is completed here. The register S_1 is added with register T_4 as shown in 5.157. The operation $T_4 = B_2 \odot x_n$ is performed in the preceding computation section. **S_1 register now contains the $s_{(2,H_h)}$coefficients**. It contains the value as shown in 5.160

$$\begin{aligned} S_1 &= (A_2 \odot \hat{Y}) + T_4 \\ &= (A_2 \odot \hat{y}_{(n,H_h)}) + (B_2 \odot x_n) \end{aligned} \tag{5.160}$$

8. The computation for $s_{(1,H_h)}$ coefficients is completed with the operation as shown in 5.158. **S_2 register now contains the s_1 coefficients**.

9. Thus the roles of the registers S_1 and S_2 are reversed. S_1 now holds the $s_{(2,H_h)}$ coefficients and vice versa.

COMPUTE_B(x_n)

$$T_2 = B_1 \odot x_n \tag{5.161}$$
$$\hat{Y} = S_2 \oplus T_5 \tag{5.162}$$
$$T_5 = B_0 \odot x_{(n+1)} \tag{5.163}$$
$$S_1 = S_1 \oplus (A_1 \odot \hat{Y}) \tag{5.164}$$
$$S_2 = A_2 \odot \hat{Y} \tag{5.165}$$
$$[O_2, O_3] = [(\hat{Y}_{1_0} + \hat{Y}_{1_3}), (\hat{Y}_{1_1} + \hat{Y}_{1_4})] \tag{5.166}$$
$$T_4 = B_2 \odot x_{(n+1)} \tag{5.167}$$
$$S_2 = S_2 \oplus T_3 \tag{5.168}$$
$$S_1 = S_1 \oplus T_2 \tag{5.169}$$

This section is similar to to **COMPUTE_B(x_n)** with the following exceptions:

1. The roles of S_1 and S_2 register are reversed again and now S_1 holds the $s_{(1,H_h)}$ coefficients and vice versa.

2. The result of the operation $\boldsymbol{B_2} \odot \boldsymbol{x_{(n+1)}}$ is stored in $\boldsymbol{T_4}$ instead of $\boldsymbol{T_3}$. This will be used by the succeeding **COMPUTE_A**$(\boldsymbol{x_n})$ section in Eq. 5.157.

3. The adjacent pairs of $\hat{\boldsymbol{Y}}$ is stored in the lower half of register $\boldsymbol{O}$.

ENDLOOP

The looping mechanism is designed such that the last section in the loop is the **COMPUTE_A**$(\boldsymbol{x_n})$ section. Hence, the `ENDLOOP` section is similar to **COMPUTE_B**$(\boldsymbol{x_n})$ with the following exceptions:

1. There are no operations dealing with $\boldsymbol{x_{(n+1)}}$ elements.

2. The output samples and coefficients are stored back into memory.

Algorithm 9: Algorithm of SIMD Parallel Biquads Function for number of biquads is 4

```
INIT
i = 0

LOOP
   i = i+8
   COMPUTE_A(x_i)
   COMPUTE_B(x_(i+1))
   COMPUTE_A(x_(i+2))
   COMPUTE_B(x_(i+3))
   STORE y_i, y_(i+1), y_(i+2), y_(i+3)
   COMPUTE_A(x_(i+4))
   COMPUTE_B(x_(i+5))
   COMPUTE_A(x_(i+6))
   If (i = buffer_size)
      GOTO ENDLOOP
   COMPUTE_B(x_(i+7))
   STORE y_(i+4), y_(i+5), y_(i+6), y_(i+7)

ENDLOOP
   STORE y_(N-4), y_(N-3), y_(N-2), y_(N-1)
   STORE s_(1,H_h), s_(2,H_h)

END
```

5.7.4 Results

To compare the performance of the SIMD vectorized parallel realization, each biquad in the cascaded realization was vectorized with the techniques proposed by R. Kutil et al. (2008) and optimized with instruction scheduling techniques to improve performance. Figure 5.8 shows the probability distribution of execution times taken over 1 million iterations. It can be seen that the vectorized cascade realization and the auto-vectorized implementation have similar execution times. Though a SIMD vectorized individual biquad will be faster than its auto-vectorized counterpart, when cascading multiple biquads, the performance gains is not significant enough. This is because the compiler vectorizes the four biquads together rather than the individual one. The vectorized parallel biquad filters implementation performs significantly better than the other two and takes less than half the clock cycles to execute.

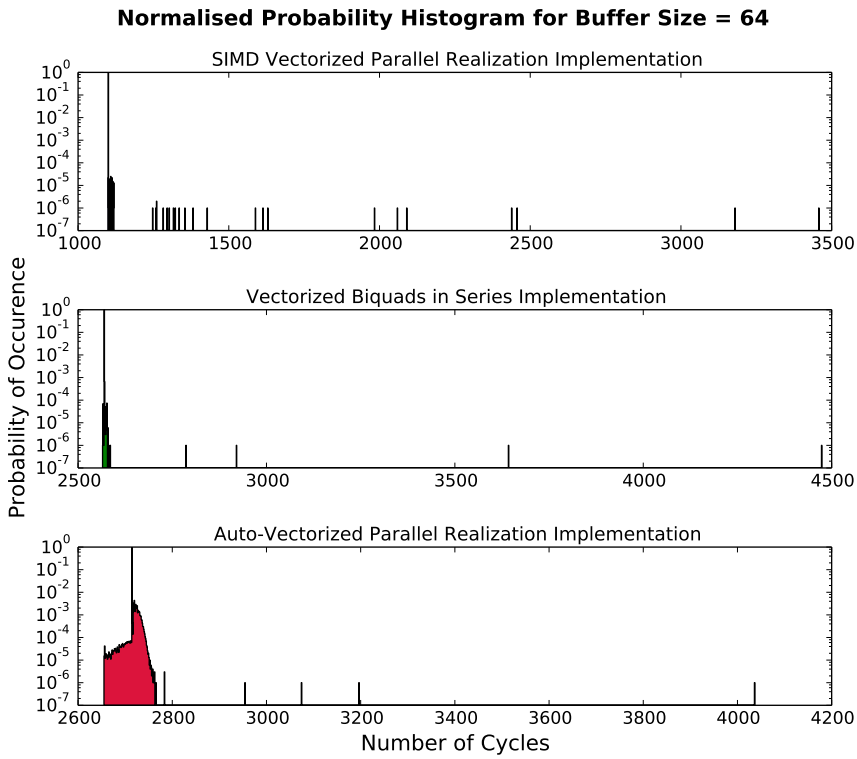

Figure 5.8: Normalized probability histograms for the vectorized parallel filter implementation, vectorized biquad implementations in cascade form and auto-vectorized C implementation.

Table 5.6: Peak program meter algorithm execution time results

Performance Metric	SIMD Vectorized Parallel Realization	Vectorized Biquads in series	Auto-Vectorized Parallel Realization
Minimum execution time (clock cycles)	1099	2565	2655
Maximum execution time (clock cycles)	3458	4474	4037
Median execution time (clock cycles)	1100	2569	2714
Speedup as compared with auto-vectorized parallel realization	**2.47**		
Speedup as compared with vectorized biquads in series	**2.33**		
Confidence Rate (1-α)	99%		

Table 5.6 shows the execution times obtained for the three implementations. The speedup of the SIMD vectorized parallel biquad implementation is 2.47 when compared to the auto-vectorized code and 2.33 when compared to vectorized biquads in series. It takes 1100 clock cycles (median) for a buffer size of 64 samples. A limitation to this approach is that the filter coefficients have to be converted to the corresponding parallel filter coefficients. This needs to be done only once and can be done in the initialization phase of the filter.

Chapter 6

Conclusion

This chapter presents the conclusions arrived after the thesis work has been carried out.

6.1 Conclusions

The main goals of the thesis was to improve the performance of the DSP algorithm and evaluate it by comparing it to techniques proposed by relevant work in the field. The required deliverable of this thesis was a high performance library for the ARM Cortex-A15 architecture, which was to be used in a real time audio processing system implemented as a part of an electric guitar. Another indirect requirement was to develop an accurate timing measurement system with high time granularity, which can be used to benchmark and schedule instructions optimally. First, an extensive study into each of the six algorithms were conducted. For each algorithm, a literature study was done as to what mathematical operations constitute each algorithm and what is the relevant work regarding their vectorization, including existing libraries such as NE10. Also, a study of the ARM Advanced SIMD unit and the NEON instruction set was conducted simultaneously. It was found that NE10 was aimed to be a rudimentary library comprised of basic DSP building blocks. It has implementations of the Gain, Mix and Gain and Mix algorithm only.

First, a simple vectorized implementation of the Gain algorithm and a FIR filter was developed to serve a test program. Initially the performance of the VFP unit and the Advanced SIMD unit was evaluated. It was found that the VFP unit was not feasible for vectorization and cannot be used to perform vector operations. Following this, a study into possible execution time measurement techniques was conducted and the different solutions were explored. The PMU cycle counter and the Chronos library were two suitable candidates and their performance was evaluated. It was found using data and instruction synchronization barriers along with the PMU timer yielded very good accuracy and high time granularity in the measurements whereas the Chronos library was unusable as it required a stable clock.

Hence, it was decided to use the modified version of the PMU cycle counter to aide in instruction scheduling and benchmarking. Using this accurate source of timing measurement, a guideline to aide in scheduling of SIMD instructions is presented. A literature study was conducted to find a way to present the performance gains in a meaningful way. It was decided that the Speedup using median execution times is the recommended way[47]. This was accompanied with a confidence rate $\mathbf{1} - \boldsymbol{\alpha}$, which represents the probability that the speedup obtained can be observed in any random run.

Following this, each algorithm of the was vectorized and its performance was evaluated. The Gain, Mix, Gain and Mix algorithms were optimized only with improved instruction scheduling. It was found that the speedup compared to that of the NE10 implementation was 1.73 and compared to the auto-vectorized code it was 2.635. Even for simple vectorized multiplication, pipeline stalls and dependencies severely hurt the performance of the NE10 and the auto-vectorized code. The Mix algorithm also had similar results. The speedup compared to that of the NE10 implementation was found to be 1.77 and compared to that of the auto-vectorized code was 3.51. The Gain and Mix had significant performance improvement compared to the other two implementations, as there were more availability of nondependent instructions and in turn, more room for instruction level parallelism. The speedup compared to that of the NE10 implementation was found to be 2.87 and compared to that of the auto-vectorized code was 3.87.

The Complex Number Multiple algorithm was vectorized by first deinterleaving the data and splitting the real and imaginary parts into separate SIMD registers. Then the numbers were multiplied and stored back into the memory in an interleaved fashion. This removed the dependencies and the speedup compared compared to that of the auto-vectorized code was found to be 1.72. The low speedup is due to fact that the SIMD vectorization calculates only 8 output samples per iteration, compared to 16 output samples in the previous mentioned algorithms. This can be improved by using more temporary registers and compute more output samples per iteration.

The Peak Program Meter Algorithm is based of an envelope detector and is comprised of a one pole filter, whose coefficient depends on the amplitude of the input signal[40]. This presented a problem when it comes to vectorization as it requires a branching condition. The branch was removed by calculating the responses of both the coefficients and masking the result using the `VGCT` instruction to choose the ones which are needed. The removal of the branch along with the reduction of pipeline stalls through instruction scheduling techniques yielded significant performance gains. The program finishes in a deterministic time of 1301 clock cycles and the speedup obtained was found to be 5.11.

The Cascaded IIR Filter was first converted to a parallel realization, so that the

number of filters can be vectorized as opposed to the traditional way of vectorizing with multiple input samples. To compare the performance, the individual biquads of the cascaded filter were vectorized with the techniques proposed by R. Kutil et. al (2008)[36]. The speedup compared to the vectorized biquad version was found to be 2.47, This was due to the fact that even though each biquad is vectorized, they are serially connected to each other, essentially needing sequential operations as the output of one is the input of the other. The parallel filter realization was written in C and auto-vectorized by the compiler. The speedup of vectorized parallel filter realization compared to this was 2.33. The compiler did vectorize certain operations in the outer loop well, but implemented many of the inner loop operations sequentially. Also, the instructions were not scheduled properly resulting in pipeline stalls.

To summarize, the following were inferred:

- Pipeline stalls significantly impair performance as shown in the results for the NE10 library implementations. Better scheduling along with removal of branches reduces the execution time by a significant amount.
- Since the ARM Advanced SIMD unit and the ARM Integer core operate in parallel, execution time measurement through operating system calls or hardware timers are inaccurate since the timer control instructions execute out of order with the Advanced SIMD unit. Barriers have to be placed to ensure that the NEON code has finished execution.
- The compiler's auto-vectorization feature does not produce optimal results and produces a lot of pipeline stalls. Even for simple operations such as vectorized multiplication (Gain) and vectorized addition (Mix), the auto-vectorized code was around 2 - 3 times slower.

6.2 Future Work

The DSP functions were vectorized using ARM NEON instructions. The availability of third party high performance DSP libraries is quite low. On the other hand, the Intel x_86/_64 platforms have the SSE instructions which are capable of performing operations on eight floating point values with one instruction. Also, the compilers for the x_86/amd_64 platforms are much better when it comes to auto-vectorization. It would be of interest to see how the algorithms presented in Chapter 5 compare against auto-vectorized code produced by Intel Compilers. Another possibility would be to use Multiple Instruction Multiple Data(MIMD) type of instructions to vectorize the algorithms.

Bibliography

[1] Woon-Seng and G. S. M. Kuo, *Introduction.* Hoboken : Wiley, 2007, ch. 1, pp. 6–7, section 1.2: Real Time Embedded Signal Processing.

[2] W.-S. Gan and S. M. Kuo, *Real-Time DSP Fundamentals and Implementation Considerations.* Hoboken : Wiley, 2007, ch. 6, pp. 250–251, section 6.3: Real Time Embedded Signal Processing.

[3] Woon-Seng and G. S. M. Kuo, *Code Optimization and Power Management.* Hoboken : Wiley, 2007, ch. 8, pp. 330 – 377.

[4] R. Chassaing, *Code Optimization.* John Wiley and Sons, Inc., 2005, ch. 8, pp. 284–303. [Online]. Available: http://dx.doi.org/10.1002/0471704075.ch8

[5] S. Dew, "Chapter 10 - Optimizing DSP Software - High-Level Languages and Programming Models," in *DSP for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 169 – 179. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B978012386535900010X

[6] R. Oshana, "Chapter 9 - Optimizing DSP Software - Benchmarking and Profiling DSP Systems," in *DSP for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 157–168. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B9780123865359000020

[7] ARM, "Introducing neon development article," 2009, accessed:15-Sep-2016. [Online]. Available: http://infocenter.arm.com/help/topic/com.arm.doc.dht0002a/DHT0002A_introducing_neon.pdf

[8] ARM, "ARM Infocenter: NEON and VFP Programming," 2010, accessed: 2016. [Online]. Available: http://infocenter.arm.com/help/topic/com.arm.doc.dui0204j/DUI0204J_rvct_assembler_guide.pdf.

[9] R. Oshana, "Chapter 12 - DSP Optimization - Memory Optimization," in *DSP for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 217 – 240. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B9780123865359000123

[10] S. Dew, "Chapter 11 - Optimizing DSP Software - Code Optimization," in *DSP for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 181 – 215. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B9780123865359000111

[11] ARM, "Using the PMU event counters in ds-5," accessed: 2016. [Online]. Available: https://developer.arm.com/products/software-development-tools/ds-5-development-studio/resources/tutorials/using-the-pmu-event-counters-in-ds-5

[12] M. Jang, K. Kim, and K. Kim, "The performance analysis of arm neon technology for mobile platforms," in *Proceedings of the 2011 ACM Symposium on Research in Applied Computation*, ser. RACS '11. New York, NY, USA: ACM, 2011, pp. 104–106. [Online]. Available: http://doi.acm.org/10.1145/2103380.2103401

[13] D. Melnik, A. Belevantsev, D. Plotnikov, and S. Lee, "A case study: optimizing gcc on arm for performance of libevas rasterization library," in *Proceedings of International Workshop on GCC Research Opportunities (GROW-2010)*, Pisa, Italy, 2010. [Online]. Available: http://ctuning.org/dissemination/grow10-03.pdf

[14] L. Kong and J. Jiang, *A Combined Hardware/Software Measurement for ARM Program Execution Time.* Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 185–201. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-35898-2_21

[15] R. Oshana, "Chapter 2 - overview of real-time and embedded systems," in *DSP for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 15 – 27. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B9780123865359000020

[16] H. Kopetz, *The Real-Time Environment.* Boston, MA: Springer US, 2011, pp. 1–28. [Online]. Available: http://dx.doi.org/10.1007/978-1-4419-8237-7_1

[17] ARM, "Arm architecture reference manual armv7-a and armv7-r edition," https://silver.arm.com/download/ARM_and_AMBA_Architecture/AR570-DA-70000-r0p0-00rel2/DDI0406C_C_arm_architecture_reference_manual.pdf, 2014, accessed: 28-Oct-2016. [Online].

[18] ARM, "Realview tools by arm : Arm architecture overview," National Center for Supercomputing Applications : http://scc.acad.bg/ncsa/articles/library/Library2016_Supercomputers-at-Work/Neuro_Computer/ARM_Architecture_Overview.pdf, 2010, accessed: 18-Oct-2016. [Online].

[19] J. Lemieux, "Introduction to arm thumb," https://www.cs.princeton.edu/courses/archive/fall12/cos375/ARMthumb.pdf, 2003, accessed: 14-Oct-2016. [Online].

[20] ARM, "Arm7tdmi technical reference manual," http://infocenter.arm.com/help/topic/com.arm.doc.ddi0210c/DDI0210B.pdf, 2004, accessed: 8-Oct-2016. [Online].

[21] ARM, "Realview compilation tools assembler guide : Vfp directives and vector notation," http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dui0204j/Chdehgeh.html, 2010, accessed: 22-Nov-2016. [Online].

[22] ARM, "Cortex-a15 : Technical reference manual," ARM Infocenter : http://infocenter.arm.com/help/topic/com.arm.doc.ddi0438c/DDI0438C_cortex_a15_r2p0_trm.pdf, 2011, accessed: 21-Nov-2016. [Online].

[23] T. Lanier, "Exploring the design of the cortex-a15 processor," ARM Infocenter : https://www.arm.com/files/pdf/AT-Exploring_the_Design_of_the_Cortex-A15.pdf, 2011, accessed: 22-Nov-2016. [Online].

[24] 7-cpu.com, "Arm cortex-a15 core," 7-cpu.com,"accessed: 25-Nov-2016. [Online]. http://www.7-cpu.com/cpu/Cortex-A15.html, accessed: 2016.

[25] ARM, "Realview compilation tools assembler guide : Neon and vfp programming," accessed: 25-Nov-2016. [Online]. http://infocenter.arm.com/help/topic/com.arm.doc.dui0204h/DUI0204H_rvct_assembler_guide.pdf, 2010.

[26] ARM, "Architecture and implementation of the arm cortex-a8 microprocessor," ARM Infocenter : accessed: 14-Oct-2017. [Online]. https://pixhawk.ethz.ch/_media/software/optimization/neon_whitepaper.pdf.

[27] B. Klug and A. L. Shimpi, "Qualcomm snapdragon s4 (krait) performance preview - 1.5 ghz msm8960 mdp and adreno 225 benchmarks," accessed: 2-Jan-2017. [Online]. http://www.anandtech.com/show/5559/qualcomm-snapdragon-s4-krait-performance-preview-msm8960-adreno-225-benchmarks/2, February 21, 2012.

[28] "Arm cortex-a8: Whats the difference between vfp and neon," accessed: 6-Oct-2016. [Online]. http://stackoverflow.com/questions/4097034/arm-cortex-a8-whats-the-difference-between-vfp-and-neon

[29] Hardkernel, "Odroid xu4," accessed: 12-Oct-2016. [Online]. http://www.hardkernel.com/main/products/prdt_info.php?g_code=G143452239825&tab_idx=1.

[30] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström, "The worst-case execution-time problem ;overview of methods and survey of tools," *ACM Trans. Embed. Comput. Syst.*, vol. 7, no. 3, pp. 36:1–36:53, May 2008. [Online]. Available: http://doi.acm.org.focus.lib.kth.se/10.1145/1347375.1347389

[31] J. A. Belloch, A. González, F. D. Igual, R. Mayo, and E. S. Quintana-Orti, "Vectorization of binaural sound virtualization on the arm cortex-a15 architecture," in *2015 23rd European Signal Processing Conference (EUSIPCO)*, Aug 2015, pp. 1601–1605.

[32] B. Mulgrew, P. M. Grant, and . Thompson, John, *Digital signal processing : concepts and applications*, 2nd ed. Basingstoke : Macmillan, 2003, previous edition: Basingstoke : Macmillan, 1999.

[33] J. O. Smith, *Introduction to Digital Filters with Audio Applications.* W3K Publishing, 2007.

[34] R. Oshana, "Chapter 7 - overview of {DSP} algorithms," in *{DSP} for Embedded and Real-Time Systems*, R. Oshana, Ed. Oxford: Newnes, 2012, pp. 113 – 131. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B978012386535900007X

[35] W.-S. Gan and S. M. Kuo, *Embedded signal processing with the micro signal architecture.* John Wiley & Sons, 2007, section 6.3: Real Time Embedded Signal Processing.

[36] R. Kutil, "Parallelization of IIR Filters Using SIMD Extensions," in *2008 15th International Conference on Systems, Signals and Image Processing*, June 2008, pp. 65–68.

[37] I. Astakhov, "Intel avx realization of infinite impulse response (iir) filter for complex float data," Intel Software Solutions Group, Intel, Tech. Rep., April 2008. [Online]. Available: https://software.intel.com/sites/default/files/m/d/4/1/d/8/Intel_AVX_Realization_of_IIR_Filter_for_Complex_Float_Data_WP.pdf

[38] A. CABRERA, "Audio metering and linux." Linux Audio Conference, 2007.

[39] "Linux audio downloads," accessed: 17-Dec-2016. [Online]. http://kokkinizita.linuxaudio.org/linuxaudio/downloads/index.html.

[40] U. Zolzer, *Dynamic Range Control.* John Wiley and Sons, Ltd, 2008, pp. 225–239. [Online]. Available: http://dx.doi.org/10.1002/9780470680018.ch7

[41] E. Sobole, "Cycle counter for cortex a8," accessed: 24-Oct-2016. [Online]. http://pulsar.webshaker.net/ccc/index.php?lng=us, 2010.

[42] A. Butko, R. Garibotti, L. Ost, and G. Sassatelli, "Accuracy evaluation of gem5 simulator system," in *7th International Workshop on Reconfigurable and Communication-Centric Systems-on-Chip (ReCoSoC)*, July 2012, pp. 1–7.

[43] cppreference.com, "Date and time utilities: Chronos library," accessed: 2.Feb-2017. [Online]. Available: http://en.cppreference.com/w/cpp/chrono

[44] P. Van Weert and M. Gregoire, *General Utilities.* Berkeley, CA: Apress, 2016, pp. 23–50. [Online]. Available: http://dx.doi.org/10.1007/978-1-4842-1876-1_2

[45] Elinux.org, "High resolution timers," accessed: 2-Feb-2017. [Online]. Available: http://elinux.org/High_Resolution_Timers

[46] L. Eeckhout, "Computer architecture performance evaluation methods," *Synthesis Lectures on Computer Architecture*, vol. 5, no. 1, pp. 1–145, 2010. [Online]. Available: http://dx.doi.org/10.2200/S00273ED1V01Y201006CAC010

[47] S.-A.-A. Touati, J. Worms, and S. Briais, "The speedup-test: a statistical methodology for programme speedup analysis and computation," *Concurrency and Computation: Practice and Experience*, vol. 25, no. 10, pp. 1410–1426, 2013. [Online]. Available: http://dx.doi.org/10.1002/cpe.2939

Appendix A

Build System used for Development

For this project, **CMake** was decided to be used to generate the makefiles. It is simple to use tool that manages the build procedure and quite easy to reconfigure for adding more files to the project. CMake works by fist including a text file named `CmakeLists.txt` in every folder of the project which has source files which specifies rules for compilation as shown in **figure A.1**.

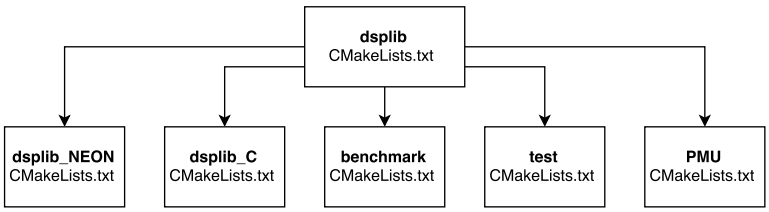

Figure A.1: `CmakeLists.txt` is placed in all directories having source code as well as the top directory.

As mentioned above CMake has a set of commands to set rules that define what should be done with the source files and what the output should be. For example, in the `dsplib_NEON` and the `dsplib_C` folders which deals with the vectorized and sequential implementations respectively, to create the respective library the following command is used.

```
add_library(dsplib_NEON file1_NEON.s file2_NEON.s
   .....)  //for library dsplib_NEON to be created


add_library(dsplib_C file1_C.s file2_C.s .....)
             //for library dsplib_C to be created
```

This creates a makefile to compile the source to object files and package then into `.a` files. To create output binaries, for example the benchmark program, the command `add_executable` is used. The benchmark program is a command line parser which

is linked to a static library named `bench`, which is in turn linked to the `dsplib_NEON` and the `dsplib_C` libraries. This is achieved with the following Cmake commands.

```
add_executable(Benchmark  run_bench.cpp)
target_link_libraries (Benchmark LINK_PUBLIC bench)
```

The command `target_link_libraries` link the necessary libraries to build the executable and `LINK_PUBLIC` makes the link interface publicly accessible. Compiler flags can also be specified using the command `set_target_properties` The top level `CMakeLists.txt` has the information which sub folders are part of the project as well as any dependencies it needs. To generate makefiles, navigate to the `build` folder and the command `cmake <path to top level CMakeLists.txt` should be used. This will check for dependencies and then generate. To summarize CMake is chosen because of its ease of use, low setup time and modularity.

Appendix B

Unit Testing with Google Test

The Google test framework can be used on the ARM platform locally. To invoke the Google test framework , the Google test header file `gtest.h` is included in the the file `test_dsplib.cpp` in the `test` folder.

- Google test provides a testing fixture class called `Test`, which is used to derive a customized class `dsplib_test`.
- Google test provides two functions, `SetUp` and `TearDown`, which deals which acts like the constructor and destructor respectively. Google Test sets up (initializes) and destructs the variables for each test case, in order to have complete isolation and independence between test cases and to prevent accidental manipulation of data.
- Google test provides functions for text fixtures `TEST_F(<text fixture class>, <text fixture name>`. Each time `TEST_F` is called, a new instance of test fixture class (in this case `dsplib_test`) is created. Using `TEST_F` unit testing can be performed.
- In `TEST_F` assertion and equality checks is placed. For example, if `x`, the expected value of a function and `y` is the output obtained from a function, they can be checked if they are equal with the statement `EXPECT_EQ(x,y)`. Google test stops the test program if this equality check fails and reports the reasons.
- In the case of this thesis, since floating point numbers are compared against each other it is better to use `EXPECT_FLOAT_EQ(x,y)`. This takes into effect rounding of noise.
- Finally in the main, the Google test program is started by calling function `testing::InitGoogleTest(&argc, argv)`. This parses command line arguments and passes options to the framework.

For functions such as `gain, mix, mix_with_gain and complex_multiply` the reference value is calculated in the `TEST_F` function itself such as shown below. For functions such as `peak_meter_NEON` and the filters, they are compared against the C implementation. The C implementation is in turn tested with reference data placed in text files which are obtained through MATLAB or python. In filters, the rounding of noise propagates through the system more as the buffer size increases especially through the use of single precision floating point numbers, so the test fixture function `EXPECT_NEAR(<expected>, <output>, <max error tolerated>)` is used. This allows the specification of a maximum permissible difference between the expected and original value.

Testing of `gain_NEON` function:

```
TEST_F(dsp_test,gain)
{
    float gain = 1.5;        //set gain value
    //calculate output
    gain_NEON(output,input,gain,BUFFER_SIZE);
    for(i = 0; i < BUFFER_SIZE; i++)
      EXPECT_FLOAT_EQ(output[i], input[i]*gain); //Check
}
```

Testing of `filter_4b_NEON` function:

```
TEST_F(dsp_test,filter_4b)
{
  //Initialize filter and filter coefficients
  filter4b_1CH_C(output_C, input, coefficients);

  //Re-Initialize filter coefficients
  filter4b_1CH_NEON(output_NEON, input, coefficients);

  for(i = 0; i < BUFFER_SIZE; i++)
    EXPECT_NEAR(output_C[i],output_NEON[i],max_error); //
        check
```

Psuedo code of testing program:

```
// Include gtest.h
// Include dsplib_NEON, dsplib_C and others
class dsp_test : public testing::Test
{
    // Declare buffers, pointers to buffers, etc
    virtual void SetUp( )
  {
    // Initialize the buffers etc
  }

  virtual void TearDown()
  {
  }
};

TEST_F(dsp_test,gain)
{
    // check correctness of gain function
}

TEST_F(dsp_test,filter_4b)
{
    // check correctness of filter_4b function
}
.
.
.
// call main from google test library and parse input
    arguments from command line
GTEST_API_ int main(int argc, char **argv) {
    testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

Appendix C

Execution Time Measurement with Chronos Library

Chronos is comprised of three elements as given below[44].

1. `Duration` is a type defined by the `std::chrono` library which measures a duration of time with number of ticks. The user can specify the time span for a tick and `Duration` stores the time taken in terms of ticks. It has two member types, `rep` which denotes the data type of the tick and `period` which is a fraction that denotes how many ticks represent one second. Arithmetic operations can also be performed on these. For example, `std::chrono::duration<double> elapsed_NEON` means that `elapsed_NEON` is of double type and can represent 1 second.

2. `Timepoint` represents a point in time as calculated with respect to the Epoch time in Unix. Its member types are `clocks` and `duration`. It gets the time from the clock type specified. Two `Timepoints` can be subtracted to give a `duration`.

3. To obtain the current timepoint from the clock, the operation `t_NEON_start = std::chronos::high_resolution_clock::now()` can be used.

C.1 Measuring code with Chronos High Precision Timer

```
    // Set up duration for one sec as tick and type as
       double
std::chrono::duration<double> elapsed_NEON;

    // initialize time point in system clock
std::chrono::time_point<std::chrono::system_clock>
   t_NEON_start;
std::chrono::time_point<std::chrono::system_clock>
   t_NEON_stop;

    //Start measurement
t_NEON_start = std::chronos::high_resolution_clock::now()
  <code to be measured>();
    //Stop Measurement
t_NEON_stop = std::chronos::high_resolution_clock::now()

    //Calculate duration
elapsed_NEON = t_NEON_stop - t_NEON_start;
```

Appendix D

Execution Time Measurement with Chronos Library

D.1 Enabling User Space Access to PMU Registers

It is necessary that the kernel module should run only on the core where benchmarks are run, to prevent unnecessary bugs. This can be achieved by using the function `on_each_cpu_mask` provided by the Linux header files. It takes a function pointer and a `cpumask` to identify which function should run on which core respectively. The PMU registers can be written to and read from using the instructions `mcr` and `mrc` respectively along with assembler intrinsics. Their syntaxes are[17]:

```
        // Write to co-processor register
    MCR <co_proc_name>, opcode1, Rd, CRn, CRm{, opcode2}

        // Read to co-processor register
    MCR <co_proc_name>, opcode1, Rd, CRn, CRm{, opcode2}

        // co_proc_name : the name of the co-processor
        // opcode1 : a code for the co-processor
        // Rd : Source register
        // CRn, CRm : The coprossors registers
        // opcode2 : optional code for co-processor
```

The ARMv7 technical manual gives the appropriate opcode and `CRn, CRm` values for the specific registers which have to be accessed.

The pseudo code of the kernel module for core number seven is shown below.

```
// kernel module to enable ARM PMU on core #7
enable_cycle_counter() {
// Write 1 to PMUSERENR, to get user access
asm volatile("mcr p15,0,%0,c9,c14,0" :: "r"(1));

// PCMR:0 set to 1 to enable all counters
// PCMR:1 resets event counters. Set to 0 for no action
// PCMR:2 resets clock counters. Set to 1
// PCMR:3 set to 0 to count every clock cycle
// PCMR:4,5 set to 0 (default reset value)
asm volatile("mcr p15,0,%0,c9,c12,0" :: "r"(0x00101f));

// Finally Set bit 31 of PMCNTENSET to enable cycle
   counter
asm volatile("mcr p15,0,%0,c9,c12,1" :: "r"(0x8000000f));

// Print cpu id in kernel message
printk("PMU enabled on CPU %d",smp_processor_id()) }

init() {
struct cpumask *mask; // to create a cpu mask
cpumask_clear(mask); // clear the mask
cpumask_set_cpu(7, mask); // set CPU to number 7
//run function on CPU
on_each_cpu,mask(mask,enable_cycle_counter,NULL, 1); }
// load module
module_init(init);
```

The function `enable_cycle_counter` first enables user access in `PMUSERENR`, then enables all counters, resets cycle counter to 0 and set it to increment every clock in the `PCMR` and finally enables cycle counters in `PMCNTENSET`. It also prints the CPU ID using the function `smp_processor_id()` which will be displayed as a kernel message when the module is being loaded. The funtion `init` creates a `cpumask` to specify the CPU and it is set to 7. Then finally the function `on_each_cpu_mask` enables the PMU cycle counter on the core specified by `mask`. This function along with `smp_processor_id()` is part of the Linux header `linux/smp.h`. To measure the execution time, the `PMCCNTR` register, which holds the cycle count value, is first reset to zero before the code and then its value is read at the end of the code.

D.2 Using PMU Cycle Counter with Barriers

```
MOV R10,#0 // Move 0 into R10 register
DSB  // Wait for previous data stores to be executed
ISB  // Wait for execution of previous arm instr.
MCR p15,0,R10,c9,c13,0 // Move 0 into cycle counter
ISB // Wait for MCR to finish.
// .............
// Code
// .............
// Last SIMD instruction
// Store operation on the result of last SIMD instruction
    here
ISB // halt execution until pipeline is flushed
DSB // wait for store
MRC p15,0,R0,c9,c13,0  // Read value
```

Measuring Latency of Measurement Technique

The latency is introduced from the code in line 5 to until line 11. This includes the amount of time taken for the barriers to execute as well as the time taken for the `mrc` to execute.

```
PUSH {R8}                                                    1
MOV R8,#0  // R8 = 0                                         2
dsb                                                          3
isb                                                          4
mcr p15, 0, R8, c9, c13, 0 //write 0 to cycle timer          5
isb                                                          6
// Timer has started, stop immediately                       7
// VST store instruction                                     8
isb                                                          9
dsb                                                          10
POP {R8}                                                     11
mrc p15, 0, R0, c9, c13, 0 // read value from cycle timer    12
```

TRITA TRITA-EE 2017:088

www.kth.se